

BUKU PRAKTIKUM

BASIS DATA

Berbasis Proyek dan Tersinkron SKKNI Data Management

MariaDB 10.4 · XAMPP · SQL · Git

Semester 3

[Logo UM Metro]

sisipkan file logo resmi

Penyusun:

Dedi Irawan, S.Kom., M.T.I.

Program Studi Ilmu Komputer

Fakultas Ilmu Komputer

Universitas Muhammadiyah Metro

2026

Halaman Hak Cipta

BUKU PRAKTIKUM BASIS DATA

Berbasis Proyek dan Tersinkron SKKNI Data Management

Penulis: Dedi Irawan, S.Kom., M.T.I.

Editor: [diisi penerbit]

Desain sampul dan tata letak: [diisi penerbit]

Diterbitkan oleh: [nama penerbit anggota IKAPI/terdaftar di Perpustnas]

[alamat lengkap penerbit, telepon, surel, laman]

Cetakan pertama, 2026

Ukuran: 17,6 × 25 cm (B5); xii + [jumlah halaman] halaman

ISBN: [diisi setelah surat keterangan ISBN terbit dari Perpustnas]

Hak cipta dilindungi Undang-Undang Republik Indonesia Nomor 28 Tahun 2014 tentang Hak Cipta. Dilarang memperbanyak sebagian atau seluruh isi buku ini dalam bentuk apa pun tanpa izin tertulis dari penerbit.

Katalog Dalam Terbitan (KDT)

[KDT resmi diterbitkan Perpustakaan Nasional RI; draf di bawah hanya acuan isian]

Irawan, Dedi

Buku praktikum basis data : berbasis proyek dan tersinkron
SKKNI data management / Dedi Irawan. -- Cetakan ke-1. --

[Kota] : [Penerbit], 2026.

xii, [jumlah] hlm. : illus. ; 25 cm.

ISBN [diisi setelah terbit]

1. Basis data -- Praktikum. 2. MariaDB. 3. SQL.
I. Judul.

Daftar Isi

Daftar Gambar.....	iii
Daftar Tabel.....	v
Kata Pengantar.....	viii
Pendahuluan: Cerita di Balik Buku Ini.....	ix
Identitas dan Tata Tertib Praktikum.....	xii
Peta Capaian Pembelajaran.....	xvi
Panduan Instalasi Lingkungan Kerja.....	xviii
Rubrik Penilaian Semester.....	xxi
Pedoman dan Checklist Laporan Praktikum.....	xxii
Modul 1: Lingkungan Kerja MariaDB di XAMPP dan Git.....	1
Modul 2: Analisis Kebutuhan Pengelolaan Data.....	14
Modul 3: Pemodelan Data dengan Entity-Relationship Diagram.....	26
Modul 4: Normalisasi dan Skema Relasional.....	34
Modul 5: Membangun Struktur dengan DDL dan Constraint.....	41
Modul 6: Mengisi, Mengubah, dan Menguji Data dengan DML.....	53
Modul 7: Menjawab Pertanyaan dengan SELECT dan Fungsi Skalar.....	59
Pertemuan 8: UTS Proyek Tahap 1.....	65
Modul 9: Meringkas Data dengan Fungsi Agregat dan GROUP BY.....	69
Modul 10: Menghubungkan Tabel dengan JOIN.....	75
Modul 11: Subquery dan Operasi Himpunan.....	80
Modul 12: CTE dan Window Function untuk Peringkat dan Tren.....	85
Modul 13: View dan Perancangan Kueri per Peran.....	91
Modul 14: Transaksi dan Integritas Data.....	97
Modul 15: Mengelola Kualitas Data: Validasi dan Pembersihan.....	103
Pertemuan 16: UAS Proyek Final dan Serah Terima.....	110
Daftar Pustaka.....	114
Glosarium.....	116
Lampiran A: Ringkasan Sintaks SQL.....	118
Lampiran B: Daftar Kode Galat.....	119
Lampiran C: Skema Kanonik Kopma dan Berkas Data.....	121
Tentang Penulis.....	124

Daftar Gambar

Gambar P.1 Pemilihan komponen pada installer XAMPP.....	xix
Gambar P.2 Keluaran verifikasi instalasi MariaDB dan Git.....	xx
Gambar 1.1 Arsitektur klien-server MariaDB.....	2
Gambar 1.2 Komponen XAMPP 8.2.12 yang dipakai di buku ini.....	3
Gambar 1.3 Alur kerja Git dari direktori kerja ke GitHub.....	4
Gambar 1.4 XAMPP Control Panel dengan MariaDB berjalan di port 3306.....	5
Gambar 1.5 Masuk ke MariaDB sebagai root dan memeriksa versi server.....	6
Gambar 1.6 Akun kerja hanya melihat basis data miliknya.....	7
Gambar 1.7 Menjalankan kueri melalui tab SQL phpMyAdmin.....	8
Gambar 1.8 Inisialisasi repositori dan push pertama ke GitHub.....	9
Gambar 1.9 Pesan log XAMPP ketika port 3306 sudah dipakai program lain.....	10
Gambar 1.10 Anatomi pesan ERROR 1045.....	11
Gambar 2.1 Posisi analisis kebutuhan dalam siklus perancangan basis data.....	15
Gambar 2.2 Dari aktivitas organisasi ke aset data dan tata kelolanya.....	15
Gambar 2.3 Pernyataan kebutuhan kabur dibandingkan pernyataan yang dapat diuji.....	17
Gambar 2.4 Peta proses bisnis Kopma per aktor.....	18
Gambar 2.5 Nota penjualan Kopma dan anotasi elemen datanya.....	19
Gambar 3.1 Notasi Crow's Foot untuk kardinalitas dan partisipasi.....	27
Gambar 3.2 ERD Kopma: seluruh tabel, kolom, tipe data, constraint, dan kardinalitas.....	30
Gambar 4.1 Spreadsheet lama Kopma dan tiga anomali.....	35
Gambar 4.2 Tahapan normalisasi UNF sampai 3NF.....	35
Gambar 5.1 Anatomi CREATE TABLE barang.....	42
Gambar 5.2 Constraint menolak kunci tamu tak dikenal dan stok negatif.....	44
Gambar 6.1 Mode aman menolak UPDATE tanpa syarat kolom kunci.....	54
Gambar 6.2 Format skenario uji positif dan negatif.....	54
Gambar 7.1 Urutan evaluasi logis pernyataan SELECT.....	60
Gambar 7.2 Hasil kueri KI-03 barang di bawah stok minimum.....	61
Gambar 8.1 Alur ujian proyek.....	66
Gambar 9.1 Cara kerja GROUP BY.....	70
Gambar 9.2 Laporan KI-01 omzet harian dengan baris total.....	71
Gambar 10.1 Perbedaan INNER JOIN dan LEFT JOIN.....	76
Gambar 10.2 Hasil KI-04 anggota dengan belanja terbesar.....	77
Gambar 11.1 Jebakan NOT IN ketika subquery mengandung NULL.....	81
Gambar 11.2 Operasi himpunan UNION, INTERSECT, dan EXCEPT.....	81
Gambar 12.1 Peringkat omzet per kategori dengan window function.....	86
Gambar 12.2 CTE rekursif menemukan hari tanpa penjualan.....	87
Gambar 13.1 View per peran menyembunyikan kolom data pribadi.....	92
Gambar 13.2 Kasir ditolak mengakses tabel dasar tetapi dapat membaca view.....	94

Gambar 14.1 Satu penjualan sebagai satu transaksi.....	98
Gambar 14.2 Galat constraint tidak menutup transaksi.....	99
Gambar 15.1 Siklus pengelolaan kualitas data.....	104
Gambar 15.2 Profil masalah data impor calon anggota.....	105
Gambar 16.1 Alur ujian proyek UAS.....	111

Daftar Tabel

Tabel P.1 Alur cerita Kopma di Buku 1.....	x
Tabel P.1 Anatomi setiap modul reguler.....	xii
Tabel P.2 Saran pembagian waktu satu sesi praktikum.....	xiii
Tabel P.3 Standar penamaan berkas dan objek basis data.....	xiv
Tabel P.4 Peta capaian pembelajaran 16 pertemuan.....	xvi
Tabel P.5 Pembagian tema proyek.....	xvii
Tabel P.6 Peta milestone proyek.....	xvii
Tabel P.7 Perangkat lunak yang digunakan.....	xviii
Tabel P.8 Daftar periksa kesiapan lingkungan.....	xx
Tabel P.9 Komponen nilai akhir praktikum.....	xxi
Tabel P.10 Rubrik ujian proyek (UTS dan UAS).....	xxi
Tabel P.11 Ketentuan umum laporan praktikum.....	xxii
Tabel P.12 Unsur Markdown untuk laporan.....	xxiii
Tabel P.13 Checklist umum laporan praktikum.....	xxiii
Tabel P.14 Checklist khusus laporan Pertemuan 1–16.....	xxiv
Tabel 1.1 Fungsi utama DBMS dan modul yang membahasnya.....	2
Tabel 1.2 Perbandingan klien CLI dan phpMyAdmin.....	3
Tabel 1.3 Ringkasan perintah yang dipakai hari ini.....	4
Tabel 1.4 Checklist khusus laporan Pertemuan 1.....	12
Tabel 1.5 Rubrik penilaian Modul 1.....	12
Tabel 1.6 Unit kompetensi yang dilatih di Modul 1.....	13
Tabel 2.1 Jenis kebutuhan yang dicatat dalam dokumen kebutuhan data.....	16
Tabel 2.2 Hasil identifikasi aktor dan proses bisnis Kopma.....	18
Tabel 2.3 Entitas kandidat Kopma.....	19
Tabel 2.4 Aturan bisnis Kopma.....	20
Tabel 2.5 Kebutuhan informasi Kopma.....	20
Tabel 2.6 Matriks CRUD proses bisnis terhadap entitas Kopma.....	20
Tabel 2.7 Cuplikan kamus data awal Kopma.....	21
Tabel 2.8 Titik awal analisis per tema proyek.....	23
Tabel 2.9 Checklist khusus laporan Pertemuan 2.....	24
Tabel 2.10 Rubrik penilaian Modul 2.....	24
Tabel 2.11 Unit kompetensi yang dilatih di Modul 2.....	25
Tabel 3.1 Unsur ERD dan contohnya di Kopma.....	27
Tabel 3.2 Relasi dan kardinalitas Kopma.....	28
Tabel 3.3 Standar kelengkapan ERD konseptual di buku ini.....	29
Tabel 3.4 Checklist khusus laporan Pertemuan 3.....	32
Tabel 3.5 Rubrik penilaian Modul 3.....	33
Tabel 3.6 Unit kompetensi yang dilatih di Modul 3.....	33

Tabel 4.1 Ringkasan bentuk normal.....	35
Tabel 4.2 Skema kanonik Kopma hasil normalisasi (acuan seluruh buku).....	37
Tabel 4.3 Checklist khusus laporan Pertemuan 4.....	39
Tabel 4.4 Rubrik penilaian Modul 4.....	40
Tabel 4.5 Unit kompetensi yang dilatih di Modul 4.....	40
Tabel 5.1 Pemetaan kamus data Kopma ke tipe data MariaDB.....	42
Tabel 5.2 Standar kelengkapan ERD fisik.....	45
Tabel 5.3 Pengelompokan tabel Kopma.....	46
Tabel 5.4 Perbedaan tabel penjualan dan pembelian.....	48
Tabel 5.5 Checklist khusus laporan Pertemuan 5.....	51
Tabel 5.6 Rubrik penilaian Modul 5.....	51
Tabel 5.7 Unit kompetensi yang dilatih di Modul 5.....	52
Tabel 6.1 Contoh skenario uji Kopma.....	56
Tabel 6.2 Checklist khusus laporan Pertemuan 6.....	57
Tabel 6.3 Rubrik penilaian Modul 6.....	58
Tabel 6.4 Unit kompetensi yang dilatih di Modul 6.....	58
Tabel 7.1 Fungsi skalar yang dipakai di modul ini.....	60
Tabel 7.2 Checklist khusus laporan Pertemuan 7.....	63
Tabel 7.3 Rubrik penilaian Modul 7.....	63
Tabel 7.4 Unit kompetensi yang dilatih di Modul 7.....	64
Tabel 8.1 Cakupan UTS.....	65
Tabel 8.2 Rincian tahap UTS.....	66
Tabel 8.3 Rubrik rinci UTS.....	67
Tabel 8.4 Checklist khusus laporan Pertemuan 8.....	67
Tabel 8.5 Unit kompetensi yang diujikan pada UTS.....	68
Tabel 9.1 Fungsi agregat dan perilakunya terhadap NULL.....	70
Tabel 9.2 Checklist khusus laporan Pertemuan 9.....	73
Tabel 9.3 Rubrik penilaian Modul 9.....	73
Tabel 9.4 Unit kompetensi yang dilatih di Modul 9.....	74
Tabel 10.1 Jenis JOIN yang dipakai di modul ini.....	76
Tabel 10.2 Checklist khusus laporan Pertemuan 10.....	79
Tabel 10.3 Rubrik penilaian Modul 10.....	79
Tabel 10.4 Unit kompetensi yang dilatih di Modul 10.....	79
Tabel 11.1 Jenis subquery.....	80
Tabel 11.2 Checklist khusus laporan Pertemuan 11.....	84
Tabel 11.3 Rubrik penilaian Modul 11.....	84
Tabel 11.4 Unit kompetensi yang dilatih di Modul 11.....	84
Tabel 12.1 Fungsi peringkat saat ada nilai kembar.....	86
Tabel 12.2 Checklist khusus laporan Pertemuan 12.....	89
Tabel 12.3 Rubrik penilaian Modul 12.....	89
Tabel 12.4 Unit kompetensi yang dilatih di Modul 12.....	90

Tabel 13.1 Rancangan view per peran Kopma.....	92
Tabel 13.2 Checklist khusus laporan Pertemuan 13.....	95
Tabel 13.3 Rubrik penilaian Modul 13.....	96
Tabel 13.4 Unit kompetensi yang dilatih di Modul 13.....	96
Tabel 14.1 Perilaku penting yang diuji di modul ini.....	98
Tabel 14.2 Checklist khusus laporan Pertemuan 14.....	101
Tabel 14.3 Rubrik penilaian Modul 14.....	102
Tabel 14.4 Unit kompetensi yang dilatih di Modul 14.....	102
Tabel 15.1 Dimensi kualitas data dan contoh aturannya di Kopma.....	103
Tabel 15.2 Checklist khusus laporan Pertemuan 15.....	108
Tabel 15.3 Rubrik penilaian Modul 15.....	108
Tabel 15.4 Unit kompetensi yang dilatih di Modul 15.....	109
Tabel 16.1 Cakupan UAS.....	110
Tabel 16.2 Rubrik rinci UAS.....	112
Tabel 16.3 Checklist khusus laporan Pertemuan 16.....	112
Tabel 16.4 Unit kompetensi yang diujikan pada UAS.....	112
Tabel G.1 Istilah penting di buku ini.....	116
Tabel A.1 Ringkasan sintaks SQL yang dipakai di buku ini.....	118
Tabel B.1 Kode galat MariaDB yang dibahas di buku ini.....	119
Tabel C.1 Berkas pendukung dari dosen.....	121

Kata Pengantar

Puji syukur penulis panjatkan ke hadirat Allah SWT atas rahmat dan karunia-Nya, sehingga *Buku Praktikum Basis Data* ini dapat diselesaikan. Buku ini disusun untuk mendampingi mahasiswa Program Studi Ilmu Komputer, Fakultas Ilmu Komputer, Universitas Muhammadiyah Metro dalam praktikum Basis Data di semester tiga.

Basis data adalah salah satu keterampilan yang paling cepat dipakai lulusan informatika di dunia kerja. Hampir setiap sistem informasi bertumpu pada data yang dirancang dengan benar, dijaga aturannya, dan dapat ditanyai dengan tepat. Karena itu, praktikum basis data tidak cukup berhenti pada menghafal perintah SQL. Mahasiswa perlu mengalami seluruh perjalanannya: memahami kebutuhan pengguna, merancang, membangun, menguji, lalu menyerahkan basis data yang dapat dipercaya.

Buku ini lahir dari kegelisahan bahwa praktikum sering tersusun dari latihan-latihan lepas yang tidak saling terhubung, sementara kesalahan yang paling merugikan justru tidak memunculkan pesan galat: kueri yang berjalan lancar tetapi hasilnya keliru. Untuk menjawabnya, buku ini memakai satu kisah yang berkesinambungan, yaitu Koperasi Mahasiswa Sejahtera, dan satu proyek pribadi yang tumbuh dari pertemuan pertama sampai ujian akhir. Setiap modul dipetakan ke elemen kompetensi SKKNI bidang Data Management dan dilengkapi Titik Analisis, panduan membaca pesan galat, serta laporan dan catatan kerja di Git.

Seluruh kode dalam buku ini telah diuji pada MariaDB 10.4.32, versi yang dipakai di laboratorium melalui XAMPP. Meski demikian, penulis menyadari buku ini masih memiliki kekurangan, sehingga kritik dan saran dari dosen, asisten, dan mahasiswa sangat diharapkan.

Penulis mengucapkan terima kasih kepada pimpinan Fakultas Ilmu Komputer dan Program Studi Ilmu Komputer Universitas Muhammadiyah Metro atas dukungannya, kepada rekan-rekan dosen dan asisten laboratorium atas masukan selama penyusunan, serta kepada mahasiswa yang menjadi alasan buku ini ditulis.

Kepada mahasiswa, penulis berpesan: jangan takut pada pesan galat. Bacalah dengan tenang, karena setiap galat adalah petunjuk. Biasakan menulis hasil yang Anda harapkan sebelum menjalankan kueri, dan kerjakan proyek Anda sendiri dengan jujur. Kemampuan yang Anda bangun sendiri, sedikit demi sedikit setiap pertemuan, itulah yang akan Anda bawa ke dunia kerja. Semoga buku ini bermanfaat.

Metro, September 2026

Penulis,

Dedi Irawan, S.Kom., M.T.I.

Pendahuluan: Cerita di Balik Buku Ini

Buku ini disusun di atas lima pertanyaan pondasi. Kelimanya dijawab di bawah, dan jawabannya menentukan bentuk setiap modul yang Anda kerjakan.

1. Mengapa Buku Ini Disusun

Basis data adalah mata kuliah yang hasilnya paling cepat dipakai lulusan di dunia kerja, tetapi praktikumnya sering berhenti pada latihan sintaks yang terlepas satu sama lain: minggu ini **CREATE TABLE**, minggu depan **SELECT**, tanpa satu persoalan utuh yang menyatukannya. Di sisi lain, dunia kerja di Indonesia sudah memiliki ukuran kompetensi yang jelas, yaitu SKKNI Data Management [1], yang menuntut kemampuan dari menganalisis kebutuhan sampai menguji basis data. Buku ini disusun untuk menjembatani keduanya.

Catatan: catatan penulis

Sebagai dosen di Program Studi Ilmu Komputer Universitas Muhammadiyah Metro, penulis ingin mahasiswa tidak hanya mampu menulis perintah SQL, tetapi juga memahami untuk siapa dan untuk apa sebuah basis data dibangun. Karena itu, buku ini disusun agar setiap pertemuan terasa sebagai satu langkah dalam pekerjaan nyata: dari mendengarkan kebutuhan pengguna, merancang, membangun, sampai menyerahkan basis data yang dapat dipercaya. Harapannya, ketika lulus, mahasiswa sudah pernah menjalani seluruh perjalanan itu setidaknya sekali, dengan proyeknya sendiri.

2. Kegelisahan yang Mendasarinya

Kegelisahan pertama: mahasiswa dapat menulis kueri yang **berjalan tanpa galat tetapi hasilnya salah**. Kesalahan semantik seperti ini lebih sulit dikenali daripada galat sintaks, sehingga sering lolos sampai ke laporan [2]. Kegelisahan kedua: mahasiswa jarang dilatih **membaca pesan galat**; galat dianggap musuh, bukan petunjuk. Kegelisahan ketiga: praktikum tidak meninggalkan jejak proses, sehingga sulit membedakan pemahaman sendiri dari hasil menyalin, terlebih setelah asisten AI mudah diakses.

3. Mereka yang Gelisah Lebih Dulu

Kegelisahan ini bukan hal baru. Taipalus meneliti penyebab di balik kesalahan perumusan kueri SQL dan kesalahan semantik yang tidak terdeteksi server [2], [3]. Hatami dan Wolcott mengamati bagaimana mahasiswa mengenali dan memakai pola ketika menulis kueri [4]. Borodzhieva menerapkan pembelajaran berbasis proyek pada mata kuliah basis data [5]. Di Indonesia, penelitian di jurnal nasional terakreditasi memperlihatkan bahwa analisis kebutuhan data dan pengukuran kualitas data adalah pekerjaan nyata di organisasi [6], [7], bukan sekadar bab di buku teks.

4. Cara Buku Ini Menjawab dan Kebaruannya

Buku ini tidak menambah jumlah latihan, tetapi mengubah susunannya:

- **Satu cerita, satu proyek.** Semua langkah percobaan memakai satu organisasi fiktif, Kopma, yang tumbuh dari pertemuan ke pertemuan. Mahasiswa menerapkan hal yang sama pada proyek pribadinya sejak Pertemuan 1 dan melanjutkannya di Buku 2 (Basis Data Lanjut).
- **Setiap modul terpetakan ke elemen SKKNI**, bukan hanya ke judul unit, sehingga capaian praktikum dapat ditelusuri ke standar kerja nasional.
- **Galat sebagai bahan belajar.** Setiap modul memiliki Titik Analisis dan Troubleshooting yang mengajarkan cara membaca pesan galat bagian demi bagian, serta kebiasaan menulis hasil yang diharapkan sebelum menjalankan kueri untuk menangkap kesalahan semantik.
- **Jejak proses yang dapat diperiksa.** Git, laporan wajib dengan checklist setiap pertemuan, personalisasi berbasis NIM, serta ujian berupa modifikasi langsung dan viva membuat pemahaman sendiri terlihat.
- **Kode yang diuji.** Setiap blok SQL dijalankan di server MariaDB sebelum dimasukkan ke naskah.

5. Cerita yang Akan Anda Jalani

Sepanjang semester, Anda mendampingi Koperasi Mahasiswa Sejahtera berpindah dari buku catatan dan spreadsheet ke basis data yang dapat dipercaya. Setiap modul dibuka dengan kotak **Cerita Pertemuan Ini** yang melanjutkan kisah sebelumnya.

Tabel P.1 Alur cerita Kopma di Buku 1

P	Babak cerita	Yang Anda bangun
1	Kopma menerima komputer untuk pencatatan	lingkungan kerja yang aman
2	Wawancara: harga lama hilang, stok minus	dokumen kebutuhan data
3	Pengurus minta “gambarnya saja”	ERD yang ditinjau klien
4	Spreadsheet lama yang berantakan	skema 3NF
5	“Jangan sampai stok minus lagi”	tabel dan constraint
6	Kasir mencoba mengetik stok -5	data dan skenario uji
7	Ketua mulai bertanya	kueri satu tabel
8	Serah terima tahap pertama	UTS proyek
9–12	Rapat anggota butuh laporan dan peringkat	agregasi, JOIN, subquery, CTE
13	Kasir tidak boleh melihat nomor HP anggota	view per peran
14	Listrik padam di tengah transaksi	transaksi

P	Babak cerita	Yang Anda bangun
15	NIM ganda dan nama yang berbeda ejaan	pembersihan data
16	Serah terima akhir kepada pengurus	UAS proyek

Di Buku 2, cerita berlanjut: Kopma tumbuh, servernya menua, datanya menumpuk, dan tuntutan keamanan data pribadi datang. Anda akan mengelola basis data yang sama pada tahap kehidupan berikutnya.

Identitas dan Tata Tertib Praktikum

Identitas Mata Kuliah

Mata kuliah	Basis Data (praktikum)
Kode mata kuliah	–
Semester	3
Dosen penyusun	Dedi Irawan, S.Kom., M.T.I.
Lingkungan praktik	XAMPP 8.2.12 (MariaDB 10.4.32, phpMyAdmin 5.2.1), Git for Windows, peramban web
Acuan kompetensi	SKKNI No. 268 Tahun 2020 Bidang Data Management [1] dan SKKNI No. 282 Tahun 2016 Subbidang Pemrograman [8]
Mata kuliah lanjutan	Basis Data Lanjut (Buku 2, MariaDB 12.3 LTS)

Cara Menggunakan Buku Ini

Buku ini disusun untuk 16 pertemuan. Pertemuan 8 adalah Ujian Tengah Semester dan pertemuan 16 adalah Ujian Akhir Semester, keduanya berbentuk evaluasi **proyek**, bukan ujian tertulis. Setiap modul reguler memakai susunan yang sama supaya Anda cepat terbiasa:

Tabel P.1 Anatomi setiap modul reguler

Bagian	Isi	Cara mengerjakan
A. Tujuan Praktikum	Kemampuan terukur yang ditargetkan	Baca sebelum praktik
B. Alat dan Bahan	Perangkat lunak dan versinya	Siapkan sebelum jam praktikum
C. Dasar Teori	Konsep, tabel ringkas, diagram	Pahami “mengapa”, bukan hanya “bagaimana”
D. Langkah Percobaan	Praktik terpandu dengan kasus contoh Koperasi Mahasiswa (Kopma)	Kerjakan di lab, jawab setiap Titik Analisis
E. Latihan/Modifikasi	Perubahan kecil dari langkah D	Semi-mandiri
F. Tugas Mandiri	Milestone proyek pribadi sesuai tema NIM	Mandiri, dikumpulkan lewat GitHub
Troubleshooting	Galat umum dan cara membacanya	Rujukan ketika menemui galat
Laporan dan Rubrik	Ketentuan laporan, aturan keaslian, bobot nilai	Pedoman pengumpulan

Setiap pertemuan dirancang untuk satu sesi praktikum. Pembagian waktu berikut adalah saran agar semua bagian sempat dikerjakan; dosen dapat menyesuaikannya dengan jadwal masing-masing.

Tabel P.2 Saran pembagian waktu satu sesi praktikum

Kegiatan	Porsi waktu	Keterangan
Pengantar dan membaca Dasar Teori	$\pm 15\%$	Dasar Teori sebaiknya sudah dibaca sebelum hadir
Langkah Percobaan (D) dan Titik Analisis	$\pm 50\%$	Inti sesi; dikerjakan di lab dengan pendampingan
Latihan dan Modifikasi (E)	$\pm 20\%$	Semi-mandiri; sisa waktu dilanjutkan di rumah
Commit, push, dan mulai menulis laporan	$\pm 15\%$	Pastikan pekerjaan tersimpan di GitHub sebelum pulang
Tugas Mandiri (F) dan laporan lengkap	di luar sesi	Batas waktu pukul 23.59 sehari sebelum pertemuan berikutnya

Kasus contoh di langkah D selalu memakai **Koperasi Mahasiswa Sejahtera (Kopma)**, organisasi fiktif yang sengaja berbeda dari semua tema proyek. Dengan begitu, hasil langkah D tidak bisa disalin mentah-mentah ke proyek Anda; Anda harus menerapkan konsepnya sendiri.

Label Versi: [10.4]

Seluruh kode di buku ini diuji pada server MariaDB 10.4.32, versi yang sama dengan yang dibawa XAMPP 8.2.12, termasuk pesan galat yang ditampilkan di setiap modul. Beberapa fitur MariaDB versi baru (misalnya **INSERT ... RETURNING** atau **JSON_TABLE**) sengaja tidak dipakai di buku ini dan baru dibahas di Buku 2 dengan label [12.3+].

Tata Tertib Laboratorium

1. Hadir paling lambat 15 menit setelah praktikum dimulai. Keterlambatan lebih dari itu dicatat sebagai tidak mengikuti langkah percobaan.
2. Gunakan akun kerja pribadi di komputer lab. Jangan menyimpan password di berkas yang di-commit ke repositori.
3. Dilarang makan dan minum di dekat perangkat komputer. Jaga kebersihan meja kerja.
4. Jangan menghentikan layanan MariaDB atau mengubah konfigurasi XAMPP di komputer lab tanpa instruksi dosen atau asisten, karena konfigurasi dipakai bersama.

5. Sebelum meninggalkan lab, pastikan seluruh pekerjaan sudah di-push ke repositori GitHub Anda. Berkas di komputer lab dapat terhapus sewaktu-waktu.

Standar Penamaan

Konvensi penamaan diberlakukan sejak pertemuan pertama, karena di dunia kerja nama objek basis data yang tidak konsisten menjadi sumber galat yang sulit dilacak. Setiap kolom diberi akhiran nama tabelnya (misalnya `nama_petugas`, bukan `nama`), sehingga saat membaca kueri yang menggabungkan banyak tabel, mahasiswa langsung tahu asal setiap kolom. Notasi `<NIM>` berarti NIM lengkap, `<3d>` berarti tiga digit terakhir NIM, dan `<2d>` berarti dua digit terakhir NIM.

Tabel P.3 Standar penamaan berkas dan objek basis data

Objek	Format	Contoh (NIM 2301010123)
Repositori GitHub	<code>basisdata-<NIM></code>	<code>basisdata-2301010123</code>
Berkas skrip SQL	<code>pNN_<topik>_<NIM>.sql</code>	<code>p05_ddl_2301010123.sql</code>
Basis data praktik (Kopma)	<code>kopma_<3d></code>	<code>kopma_123</code>
Basis data proyek	<code><kode_tema>_<3d></code>	<code>perpus_123</code>
Akun MariaDB praktik / proyek	<code>mhs_<3d> / dev_<3d></code>	<code>mhs_123 / dev_123</code>
Tabel	huruf kecil, <code>snake_case</code> , kata benda tunggal	<code>anggota</code> , <code>detail_penjualan</code>
Kolom (field)	<code><atribut>_<nama_tabel></code> , agar setiap kolom langsung menunjukkan tabel asalnya	<code>nama_petugas</code> , <code>stok_barang</code> , <code>qty_detail_penjualan</code>
Kunci primer	<code>id_<nama_tabel></code>	<code>id_anggota</code>
Kunci tamu	sama persis dengan kunci primer yang dirujuk	<code>id_petugas</code> di tabel penjualan
Kode alami	<code>kode_<tabel></code> atau <code>no_<tabel></code> , sudah memuat nama tabel	<code>kode_barang</code> , <code>no_anggota</code>
Kata kunci SQL	HURUF KAPITAL	<code>SELECT</code> , <code>CREATE TABLE</code>
Pesan commit	<code>pNN: <deskripsi singkat></code>	<code>p05: membuat tabel inti proyek</code>

Aturan Keaslian dan Plagiarisme

Menyalin skrip SQL, dokumen, atau laporan milik mahasiswa lain, baik sebagian maupun seluruhnya, adalah pelanggaran akademik. Setiap modul memuat **aturan personalisasi** berbasis NIM (nama objek, parameter angka, dan tema proyek), sehingga pekerjaan yang identik antarmahasiswa mudah dikenali. Dosen berhak melakukan tanya jawab singkat (*viva*) dan pemeriksaan kemiripan kapan saja.

Kebijakan Penggunaan Asisten AI

- **Diizinkan** untuk menjelaskan pesan galat, membandingkan sintaks, dan memahami konsep.
- **Tidak diizinkan** untuk menghasilkan skrip atau dokumen Tugas Mandiri dan milestone proyek secara utuh.
- Setiap laporan wajib memuat bagian **Pernyataan Penggunaan AI**: alat yang dipakai, untuk apa, dan bagian mana yang terbantu. Laporan tanpa pernyataan dianggap tidak memakai AI; jika saat *viva* terbukti sebaliknya, nilai milestone tersebut dibatalkan.
- Saat UTS dan UAS proyek, Anda akan diminta melakukan **modifikasi langsung** di depan dosen. Pemahaman yang tidak dimiliki sendiri akan terlihat pada tahap ini.

Standar Kerja: Git dan GitHub

Seluruh tugas dikumpulkan melalui repositori GitHub pribadi, bukan berkas zip atau cetakan kertas [9]. Riwayat commit dipakai sebagai bukti proses pengerjaan proyek dari minggu ke minggu, sehingga commit yang menumpuk di akhir semester dinilai lebih rendah daripada commit yang rutin tiap pertemuan.

- Repositori diinisialisasi di Modul 1 dan dipakai sampai Modul 16.
- Minimal satu commit bermakna di setiap pertemuan, dengan format pesan **pNN: deskripsi**.
- Tautan repositori dikirim ke dosen sekali di awal semester; akses dosen sebagai *collaborator* atau repositori publik sesuai arahan dosen.

Peta Capaian Pembelajaran

Tabel berikut merangkum apa yang akan Anda kuasai di setiap pertemuan beserta unit kompetensi SKKNI yang dilatih. Kode **DMS** merujuk ke unit J.62DMS00.xxx.1 pada SKKNI Data Management [1], sedangkan **PRG** merujuk ke unit J.620100.020.02 *Menggunakan SQL* pada SKKNI Pemrograman [8].

Tabel P.4 Peta capaian pembelajaran 16 pertemuan

Pert.	Topik	Keluaran yang dikuasai	Unit SKKNI
1	Lingkungan kerja MariaDB di XAMPP dan Git	Server berjalan, akun root aman, akun kerja, repositori proyek	DMS 017, 019
2	Analisis kebutuhan pengelolaan data	Dokumen kebutuhan data proyek	DMS 001
3	Pemodelan data (ERD)	Diagram ER proyek	DMS 006
4	Normalisasi	Skema relasional 3NF	DMS 006
5	DDL dan constraint	Skrip pembuatan tabel proyek	DMS 006, 010
6	DML dan pengujian basis data	Data sampel dan skenario uji	DMS 010
7	SELECT dan fungsi skalar	Kueri kebutuhan informasi dasar	DMS 019, PRG
8	UTS: Proyek Tahap 1	Demo basis data berjalan, modifikasi langsung, viva	DMS 001, 006, 010, 019
9	Agregasi dan GROUP BY	Laporan ringkasan	DMS 019, PRG
10	JOIN	Kueri lintas tabel	DMS 019, PRG
11	Subquery dan operasi himpunan	Kueri analitis	DMS 019, PRG
12	CTE dan window function	Laporan peringkat dan tren	DMS 006, PRG
13	View dan perancangan kueri	View per peran pengguna	DMS 006, 019
14	Transaksi dan integritas data	Transaksi bisnis utama proyek	DMS 006, 010
15	Validasi dan pembersihan data	Audit dan perbaikan kualitas data	DMS 012
16	UAS: Proyek Final	Basis data OLTP lengkap dan terdokumentasi	Komprehensif

Tema Proyek Berdasarkan NIM

Proyek dikerjakan **individu** sejak Pertemuan 1 dan terus tumbuh sampai Pertemuan 16. Tema ditentukan oleh dua digit terakhir NIM, sehingga tidak dapat dipilih sendiri. Proyek yang sama akan Anda lanjutkan di mata kuliah Basis Data Lanjut.

Tabel P.5 Pembagian tema proyek

Dua digit akhir NIM	Tema	Kode tema	Proses inti
00–24	Perpustakaan	perpus	keanggotaan, katalog, peminjaman, pengembalian, denda
25–49	Klinik	klinik	pendaftaran pasien, kunjungan, tindakan, resep, pembayaran
50–74	Toko daring	toko	katalog, keranjang, pesanan, pembayaran, pengiriman
75–99	Akademik	akad	mahasiswa, mata kuliah, jadwal, KRS, nilai

Tabel P.6 Peta milestone proyek

Pert.	Milestone yang dikumpulkan
1	Repositori, basis data dan akun proyek, README identitas proyek
2	Dokumen kebutuhan data
3	ERD proyek
4	Skema relasional 3NF
5	Skrip DDL dan ERD fisik proyek
6	Data sampel dan skenario uji
7	Sepuluh kueri satu tabel pendukung KI
8	UTS: Proyek Tahap 1 (milestone 1–7 terintegrasi)
9–12	Laporan ringkasan, kueri relasional, kueri analitis, laporan peringkat/tren
13	View per peran pengguna
14	Transaksi bisnis utama
15	Audit dan perbaikan kualitas data
16	UAS: Proyek Final

Panduan Instalasi Lingkungan Kerja

Bagian ini dikerjakan **sekali** sebelum Pertemuan 1, baik di laptop pribadi maupun untuk memeriksa komputer lab. Modul-modul berikutnya menganggap lingkungan ini sudah siap.

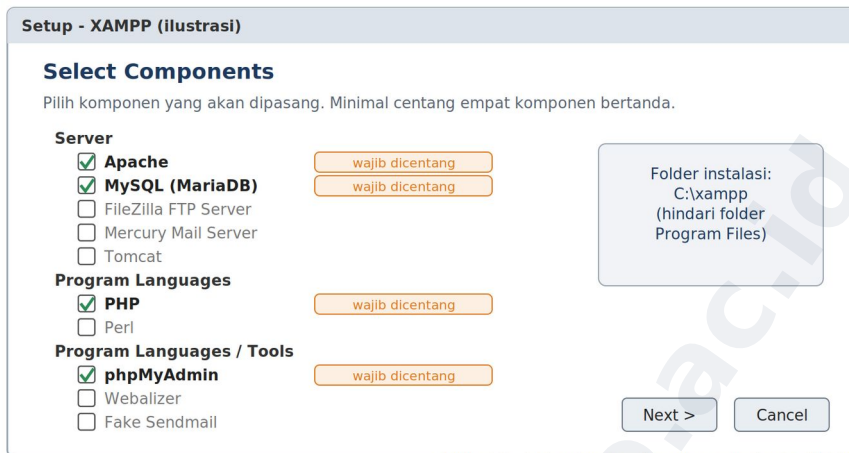
Tabel P.7 Perangkat lunak yang digunakan

Perangkat lunak	Versi	Fungsi
XAMPP untuk Windows	8.2.12	Paket berisi Apache 2.4.58, MariaDB 10.4.32, PHP 8.2.12, dan phpMyAdmin 5.2.1 [10]
Git for Windows	versi stabil terbaru	Pencatatan versi dan pengumpulan tugas [9]
Peramban web	Chrome, Edge, atau Firefox terbaru	Mengakses phpMyAdmin dan GitHub
Editor teks (disarankan)	Visual Studio Code terbaru	Menulis berkas <code>.sql</code> dan <code>.md</code>
draw.io (diagrams.net)	versi web atau desktop terbaru	Menggambar ERD mulai Pertemuan 3
Akun GitHub	–	Repositori pengumpulan tugas

Catatan: spesifikasi minimum
Windows 10/11 64-bit, RAM 4 GB, ruang kosong 2 GB. XAMPP untuk macOS dan Linux tersedia, tetapi seluruh contoh di buku ini ditulis untuk Windows.

Langkah 1: Memasang XAMPP

1. Unduh *installer* XAMPP untuk Windows versi 8.2.12 dari laman resmi Apache Friends [10].
2. Jalankan *installer*. Jika muncul peringatan UAC (User Account Control), pilih **OK**; peringatan ini hanya mengingatkan agar XAMPP tidak dipasang di folder `C:\Program Files`.
3. Pada layar *Select Components*, centang minimal **Apache**, **MySQL**, **PHP**, dan **phpMyAdmin** seperti pada Gambar P.1. Komponen lain boleh dilepas.
4. Biarkan folder instalasi `C:\xampp`, lalu lanjutkan sampai selesai.



ilustrasi disederhanakan; tampilan sebenarnya dapat sedikit berbeda.

Gambar P.1 Pemilihan komponen pada installer XAMPP

Catatan: label “MySQL”

Sejak versi 5.5.30 dan 5.6.14, XAMPP memakai MariaDB sebagai pengganti MySQL, tetapi label di installer dan Control Panel tetap bertuliskan “MySQL”. Server yang sebenarnya berjalan adalah MariaDB 10.4.32 [10].

Langkah 2: Menambahkan MariaDB ke PATH (disarankan)

Agar perintah `mysql` bisa dipanggil dari Command Prompt mana pun, tambahkan folder biner MariaDB ke variabel lingkungan PATH:

1. Buka **Start**, ketik *environment variables*, pilih **Edit the system environment variables**, lalu klik **Environment Variables**.
2. Pada bagian *User variables*, pilih **Path**, klik **Edit**, lalu **New**, dan isi `C:\xampp\mysql\bin`.
3. Klik **OK** di semua jendela, lalu tutup dan buka kembali Command Prompt.

Tips

Di komputer lab yang tidak mengizinkan perubahan PATH, gunakan tombol **Shell** di XAMPP Control Panel. Shell tersebut sudah mengenali perintah `mysql`.

Langkah 3: Memasang dan Mengatur Git

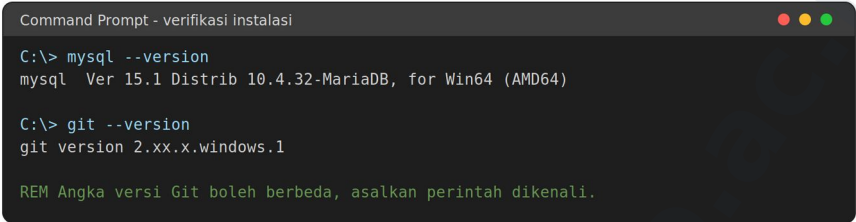
Unduh Git for Windows dari laman resmi git-scm.com, lalu pasang dengan pilihan bawaan. Setelah selesai, atur identitas yang akan tercatat di setiap commit:

```
git config --global user.name "Nama Lengkap Anda"
git config --global user.email "email_anda@contoh.ac.id"
git config --global init.defaultBranch main
```

Langkah 4: Verifikasi

Buka Command Prompt baru, lalu jalankan dua perintah berikut. Keluaran yang benar tampak seperti Gambar P.2.

```
mysql --version
git --version
```



Ilustrasi disederhanakan; tampilan sebenarnya dapat sedikit berbeda.

Gambar P.2 Keluaran verifikasi instalasi MariaDB dan Git

Tabel P.8 Daftar periksa kesiapan lingkungan

Pemeriksaan	Hasil yang diharapkan
mysql --version	Menampilkan Distrib 10.4.32-MariaDB
git --version	Menampilkan nomor versi Git
XAMPP Control Panel	Tombol Start pada Apache dan MySQL dapat diklik
Akun GitHub	Dapat masuk dan membuat repositori baru

Rubrik Penilaian Semester

Penilaian menekankan **proses dan pemahaman**, bukan sekadar produk akhir. Karena itu, milestone mingguan dan riwayat commit ikut dinilai, dan kedua ujian dilakukan dengan demo proyek. Pendekatan berbasis proyek seperti ini terbukti membantu mahasiswa mengaitkan konsep basis data dengan persoalan nyata [5].

Tabel P.9 Komponen nilai akhir praktikum

Komponen	Bobot	Indikator
Pelaksanaan praktikum (langkah D dan latihan E)	20%	Kehadiran, jawaban Titik Analisis, hasil latihan
Milestone proyek dan laporan mingguan	20%	Kelengkapan milestone, kualitas laporan, commit rutin
UTS: Proyek Tahap 1 (Pertemuan 8)	25%	Lihat rubrik ujian proyek
UAS: Proyek Final (Pertemuan 16)	35%	Lihat rubrik ujian proyek

Tabel P.10 Rubrik ujian proyek (UTS dan UAS)

Aspek	Bobot	Indikator
Produk berjalan sesuai milestone	40%	Skema dan kueri berjalan tanpa galat, sesuai dokumen kebutuhan
Demo, modifikasi langsung, dan viva	30%	Mampu mengubah objek atau kueri saat diminta dan menjelaskan alasan desain
Dokumentasi dan riwayat commit	20%	README, kamus data, dan commit rutin setiap pertemuan
Ketepatan waktu	10%	Dikumpulkan sebelum batas waktu

Bobot di atas dapat disesuaikan dosen pengampu dengan tetap mempertahankan porsi ujian berbasis proyek.

Pedoman dan Checklist Laporan Praktikum

Setiap pertemuan, dari Pertemuan 1 sampai Pertemuan 16 termasuk UTS dan UAS, wajib diakhiri dengan laporan praktikum. Pertemuan tanpa laporan dianggap belum selesai, dan nilai milestone proyek pertemuan tersebut tidak diberikan meskipun berkasnya ada di repositori.

Tabel P.11 Ketentuan umum laporan praktikum

Hal	Ketentuan
Format	Satu berkas Markdown <code>laporan/pNN_laporan_<NIM>.md</code> di repositori <code>basisdata-<NIM></code> , dengan gambar di folder <code>laporan/img/</code>
Batas waktu	Pukul 23.59 sehari sebelum pertemuan berikutnya; laporan UAS mengikuti jadwal dosen
Panjang	Secukupnya; ringkasan teori maksimal setengah halaman, pembahasan fokus pada hasil sendiri
Pengumpulan	Cukup di- <i>commit</i> dan di- <i>push</i> ; dosen memeriksa langsung dari repositori
Keterlambatan	Commit setelah batas waktu tercatat otomatis oleh Git dan memengaruhi nilai laporan

Struktur Baku Laporan

Salin kerangka berikut setiap pertemuan. Urutan bagian tidak boleh diubah agar pemeriksaan konsisten.

laporan/pNN_laporan_<NIM>.md

```
# Laporan Praktikum Basis Data - Pertemuan NN
**Nama:** ...   **NIM:** ...   **Kelas:** ...   **Tanggal:** ...
## 1. Tujuan Praktikum
## 2. Ringkasan Dasar Teori
## 3. Hasil Langkah Percobaan      (tangkapan layar + keterangan)
## 4. Jawaban Titik Analisis
## 5. Hasil Latihan dan Modifikasi
## 6. Tugas Mandiri: Milestone Proyek NN
## 7. Pembahasan dan Kendala
## 8. Kesimpulan
## 9. Pernyataan Penggunaan AI
## 10. Bukti Git                  (tautan repositori + hash commit)
## Checklist                      (salin dari buku, centang)
```

Menulis Laporan dengan Markdown

Markdown adalah format teks sederhana yang langsung tampil rapi di GitHub. Hanya enam unsur berikut yang dibutuhkan untuk laporan praktikum:

Tabel P.12 Unsur Markdown untuk laporan

Kebutuhan	Cara menulis	Catatan
Judul bagian	## 3. Hasil Langkah Percobaan	Satu tanda # untuk judul laporan, dua untuk bagian
Huruf tebal	**Titik Analisis 1**	Untuk menandai butir penting
Daftar	- butir atau 1. butir	Beri baris kosong sebelum daftar
Kode SQL	tiga backtick diikuti <code>sql</code> , lalu kode, lalu tiga backtick	Hasil kueri disalin sebagai teks di blok kode yang sama
Gambar	<code>![keterangan](img/p05_constraint.png)</code>	Simpan gambar di <code>laporan/img/</code> , nama tanpa spasi
Tabel	<code> Kolom Kolom </code> lalu <code>--- ---</code>	Untuk skenario uji dan checklist

Sebelum commit, buka pratinjau Markdown di Visual Studio Code (**Ctrl+Shift+V**) untuk memastikan gambar dan tabel tampil. Jika gambar tidak muncul di GitHub, periksa huruf besar-kecil nama berkas, karena GitHub membedakannya.

Checklist Umum (Berlaku di Semua Pertemuan)

Tabel P.13 Checklist umum laporan praktikum

	Butir	Yang harus ada
☐	Identitas	Nama, NIM, kelas, pertemuan ke-N, tanggal pelaksanaan, nama asisten/dosen
☐	Tujuan	Tujuan praktikum ditulis ulang dengan bahasa sendiri (maksimal 5 baris)
☐	Ringkasan teori	Pemahaman sendiri atas Dasar Teori, maksimal setengah halaman, tanpa menyalin buku
☐	Langkah percobaan (D)	Tangkapan layar hasil langkah kunci sesuai checklist khusus, masing-masing diberi keterangan singkat
☐	Titik Analisis	Semua Titik Analisis dijawab lengkap dengan alasan, bukan sekadar ya/tidak
☐	Latihan (E)	Skrip/dokumen hasil latihan beserta bukti berjalan

	Butir	Yang harus ada
☐	Tugas Mandiri (F)	Milestone proyek pertemuan ini: berkas, bukti, dan penjelasan keputusan desain
☐	Pembahasan dan kendala	Galat yang ditemui, cara membacanya, dan cara mengatasinya
☐	Kesimpulan	Dua sampai empat kalimat dengan bahasa sendiri
☐	Pernyataan Penggunaan AI	Alat yang dipakai, untuk apa, bagian mana; tulis “tidak menggunakan” bila tidak memakai
☐	Bukti Git	Tautan repositori dan kode <i>commit</i> (hash) pertemuan ini dengan pesan pNN: ...
☐	Keaslian	Setiap tangkapan layar memperlihatkan akun ber-NIM dan jam sistem; nama objek memuat NIM/inisial

Checklist Khusus per Pertemuan

Selain checklist umum, setiap pertemuan memiliki berkas, bukti, dan analisis wajib berikut. Checklist yang sama juga dicetak di akhir setiap modul.

Tabel P.14 Checklist khusus laporan Pertemuan 1–16

P	Berkas wajib di repositori	Bukti tangkapan layar	Analisis wajib
1	p01_lingkungan_<NIM>.sql (dapat dijalankan ulang), README.md berisi Identitas Proyek, .gitignore	SELECT VERSION() , CURRENT_USER(); SELECT @@sql_mode; SHOW DATABASES sebagai mhs_<3d> dan dev_<3d> ; galat 1044 dan 1142; halaman masuk phpMyAdmin mode cookie;git push pertama	Titik Analisis 1–4; perbedaan kode galat 1044, 1045, dan 1142
2	p02_kebutuhan_data_kopma_<NIM>.md dan p02_kebutuhan_data_<NIM>.md (proyek)	Tabel PB, AB, KI, matriks CRUD, dan kamus data di berkas proyek (tangkapan tampilan GitHub)	Titik Analisis 1–3; perhitungan parameter P; perbaikan tiga pernyataan kebutuhan kabur
3	p03_erd_kopma_<NIM> dan p03_erd_<NIM> (gambar + berkas sumber diagram)	ERD di draw.io dengan kardinalitas terbaca; catatan tinjauan klien	Pemetaan setiap entitas ke PB/KI dari Pertemuan 2; alasan kardinalitas relasi utama

P	Berkas wajib di repositori	Bukti tangkapan layar	Analisis wajib
4	p04_normalisasi_<NIM>.md (proyek)	Tabel tahapan UNF → 1NF → 2NF → 3NF	Daftar dependensi fungsional; keputusan menyimpan atau menghitung nilai turunan beserta alasannya
5	p05_ddl_<NIM>.sql (dapat dijalankan ulang dari basis data kosong) dan ERD fisik p05_erd_fisik_<NIM>	SHOW CREATE TABLE tabel utama; galat saat constraint dilanggar (FK, UNIQUE, CHECK)	Alasan pemilihan tipe data dan constraint; kaitan constraint dengan AB-xx
6	p06_data_<NIM>.sql (data sampel berbasis parameter P) dan p06_uji_<NIM>.md (skenario uji)	Hasil skenario uji positif dan negatif; jumlah baris per tabel	Tabel skenario uji: masukan, hasil yang diharapkan, hasil nyata, status
7	p07_kueri_<NIM>.sql berisi minimal 10 kueri satu tabel, masing-masing diberi komentar KI dan hasil yang diharapkan	Hasil setiap kueri	Pemetaan kueri → KI; alasan pemilihan fungsi skalar yang dipakai
8 UTS	p08_uts_<NIM>.md (ringkasan milestone 1-7) dan install_uts_<NIM>.sql	Bukti basis data berjalan dari skrip DDL dan data (instalasi bersih)	Catatan modifikasi langsung saat demo; refleksi keputusan desain yang akan diperbaiki
9	p09_agregasi_<NIM>.sql berisi kueri laporan ringkasan	Hasil laporan ringkasan (misalnya omzet per hari/bulan)	Kaitan setiap laporan dengan KI; penjelasan pemakaian HAVING
10	p10_join_<NIM>.sql berisi kueri relasional	Hasil INNER dan LEFT JOIN, termasuk kasus data tanpa pasangan	Perbedaan hasil antarjenis JOIN pada data proyek
11	p11_subquery_<NIM>.sql	Hasil subquery berkorelasi dan operasi himpunan	Perbandingan subquery dengan JOIN untuk kebutuhan yang sama
12	p12_cte_window_<NIM>.sql	Hasil laporan peringkat dan tren	Penjelasan PARTITION BY dan ORDER BY pada window function yang dipakai

P	Berkas wajib di repositori	Bukti tangkapan layar	Analisis wajib
13	p13_view_<NIM>.sql (view per peran) dan p13_role_<NIM>.sql (role dan akun, tanpa password asli)	Akses view berhasil dan akses tabel dasar ditolak untuk setiap peran	Matriks peran × view; alasan pembatasan kolom data pribadi
14	p14_transaksi_<NIM> .sql	Skenario COMMIT dan ROLLBACK , termasuk transaksi yang dibatalkan karena melanggar aturan	Pemetaan transaksi ke AB-xx; alasan batas awal dan akhir transaksi
15	p15_kualitas_<NIM>. sql dan p15_audit_kualitas_ <NIM>.md	Profil masalah sebelum dan sesudah, daftar baris yang ditolak	Tabel temuan: dimensi kualitas, jumlah kasus, tindakan perbaikan
16 UAS	p16_uas_<NIM>.md, install_uas_<NIM>.s ql, README.md dan kamus_data_<NIM>.md final	Instalasi bersih dari nol dan demo seluruh KI	Catatan modifikasi langsung; refleksi perjalanan proyek P1–P16

Catatan: cara dosen memeriksa

Dosen mencocokkan checklist di akhir laporan dengan isi repositori. Butir yang dicentang tetapi tidak ditemukan dianggap tidak ada, dan pencilangan palsu diperlakukan sebagai pelanggaran keaslian.

Modul 1: Lingkungan Kerja MariaDB di XAMPP dan Git

Cerita Pertemuan Ini

Koperasi Mahasiswa Sejahtera (Kopma) selama ini mencatat penjualan di buku tulis dan spreadsheet. Pengurus baru saja menerima satu komputer dan meminta Anda menyiapkan “dapur” pencatatan yang rapi dan aman. Sebelum satu data pun disimpan, ruang kerjanya harus siap: server berjalan, pintu masuk dikunci, dan setiap pekerjaan tercatat. Hari ini Anda menyiapkan semuanya.

A. Tujuan Praktikum

Setelah menyelesaikan modul ini, mahasiswa mampu:

1. menjalankan dan menghentikan layanan MariaDB melalui XAMPP Control Panel serta membaca status dan port-nya;
2. terhubung ke server MariaDB melalui klien baris perintah (CLI) dan phpMyAdmin, lalu menjelaskan perbedaan keduanya;
3. mengamankan akun **root** dengan password dan membuat akun kerja yang hak aksesnya terbatas pada satu basis data;
4. membuat basis data pertama dan mencatat pekerjaan ke repositori Git yang terhubung ke GitHub.

B. Alat dan Bahan

- Komputer Windows 10/11 64-bit yang sudah disiapkan sesuai Panduan Instalasi.
- XAMPP 8.2.12 berisi MariaDB 10.4.32 dan phpMyAdmin 5.2.1.
- Git for Windows dan akun GitHub.
- Command Prompt atau tombol **Shell** di XAMPP Control Panel.
- Peramban web untuk membuka <http://localhost/phpmyadmin>.

C. Dasar Teori

C.1 Basis Data dan Sistem Manajemen Basis Data

Basis data adalah kumpulan data yang saling berhubungan dan disimpan secara terorganisasi agar dapat dipakai bersama oleh banyak pengguna dan aplikasi. Perangkat lunak yang mengelolanya disebut **sistem manajemen basis data** (*database management system*, DBMS) [11]. Tanpa DBMS, setiap aplikasi harus mengurus sendiri cara menyimpan berkas, mencegah dua orang mengubah data yang sama bersamaan, dan memulihkan data setelah listrik padam. DBMS mengambil alih pekerjaan tersebut sehingga pengembang cukup menyatakan *apa* yang diinginkan melalui bahasa SQL [12].

Fungsi utama DBMS yang akan Anda temui sepanjang semester dirangkum pada Tabel 1.1. Tiap fungsi dibahas mendalam di modul-modul berikutnya; tugas modul ini hanya memastikan Anda dapat “masuk” ke DBMS dan bekerja dengan aman.

Tabel 1.1 Fungsi utama DBMS dan modul yang membahasnya

Fungsi DBMS	Makna sederhana	Dibahas di
Definisi data	Membuat struktur tabel dan aturannya	Modul 5
Manipulasi dan kueri	Menambah, mengubah, menghapus, dan mencari data	Modul 6, 7, 9–13
Integritas	Menolak data yang melanggar aturan	Modul 5, 14, 15
Konkurensi dan pemulihan	Banyak pengguna sekaligus tanpa merusak data	Modul 14 (lanjut di Buku 2)
Keamanan	Membatasi siapa boleh melakukan apa	Modul 1 (dasar), Buku 2

C.2 Arsitektur Klien-Server MariaDB

MariaDB bekerja dengan pola **klien-server**. Program server (proses `mysqld`) berjalan terus di latar belakang dan “mendengarkan” permintaan pada port TCP **3306**. Program lain, seperti klien baris perintah `mysql` atau phpMyAdmin, bertindak sebagai klien: mengirim perintah SQL, menunggu, lalu menampilkan hasil atau pesan galat yang dikirim balik oleh server [13]. Alurnya diperlihatkan pada Gambar 1.1.



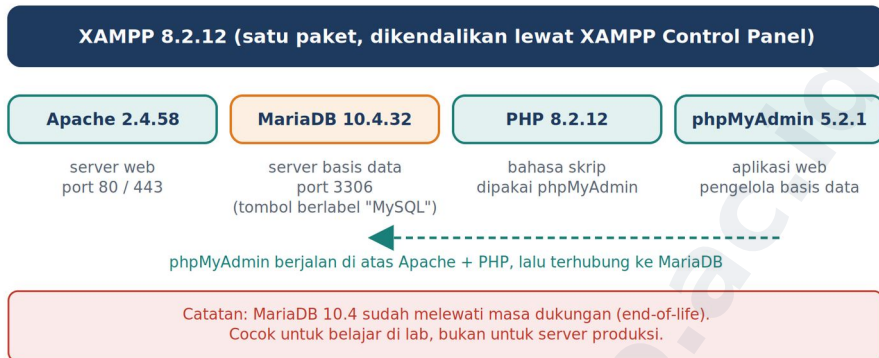
Gambar 1.1 Arsitektur klien-server MariaDB

Pemahaman ini penting untuk membaca galat. Jika klien menampilkan pesan “tidak dapat terhubung”, masalahnya ada di jalur koneksi atau server belum berjalan. Sebaliknya, jika pesan diawali **ERROR** disertai kode angka, berarti koneksi berhasil dan server sendiri yang menolak perintah Anda.

C.3 XAMPP sebagai Lingkungan Belajar

XAMPP adalah paket gratis yang menggabungkan server web Apache, server basis data MariaDB, bahasa PHP, dan aplikasi phpMyAdmin dalam satu pemasangan [10]. Semua

komponen dinyalakan dan dimatikan lewat XAMPP Control Panel (Gambar 1.2). Kepraktisan inilah yang membuat XAMPP banyak dipakai di laboratorium.



Gambar 1.2 Komponen XAMPP 8.2.12 yang dipakai di buku ini

Perhatian: bukan untuk produksi

MariaDB 10.4.32 adalah rilis terakhir seri 10.4 [14], dan seri tersebut sudah tidak menerima pembaruan keamanan. Versi ini aman untuk belajar SQL di jaringan lab, tetapi jangan dipakai untuk menyimpan data sungguhan di server yang terhubung ke internet. Di Buku 2, Anda akan memakai MariaDB 12.3 LTS yang dipasang mandiri.

C.4 Dua Cara Berbicara dengan Server: CLI dan phpMyAdmin

Buku ini mewajibkan Anda menguasai **CLI terlebih dahulu**, dengan phpMyAdmin sebagai pendamping. Alasannya dirangkum pada Tabel 1.2: perintah yang diketik bisa disimpan sebagai skrip, diulang persis sama, dikumpulkan ke Git, dan diuji dalam sertifikasi kompetensi, sedangkan rangkaian klik tidak meninggalkan jejak.

Tabel 1.2 Perbandingan klien CLI dan phpMyAdmin

Aspek	Klien CLI <code>mysql</code>	phpMyAdmin
Bentuk	Teks di Command Prompt	Aplikasi web di peramban
Kecepatan untuk pemula	Lebih lambat di awal	Cepat, berbasis klik
Dapat diulang dan disimpan	Ya, sebagai skrip <code>.sql</code>	Terbatas (harus disalin manual dari tab SQL)
Pesan galat	Lengkap: kode, SQLSTATE, teks	Lengkap, tetapi kadang terselip di antara elemen tampilan
Dipakai di dunia kerja	Hampir selalu tersedia di server	Tidak selalu dipasang di server produksi

C.5 Akun, Host, dan Hak Akses

Di MariaDB, akun tidak hanya ditentukan oleh nama pengguna, tetapi oleh pasangan **pengguna@host**. `'root'@'localhost'` dan `'root'@'127.0.0.1'` adalah dua akun berbeda, masing-masing dengan password dan hak aksesnya sendiri [13]. Bawaan XAMPP, akun `root` dibuat **tanpa password** agar mudah dipakai. Praktik seperti ini berbahaya di luar lab, karena siapa pun yang dapat menjangkau server langsung memegang kendali penuh.

Prinsip yang dipakai di buku ini adalah **hak akses minimum** (*least privilege*): akun `root` hanya untuk administrasi, sedangkan pekerjaan harian memakai akun kerja yang hanya berhak atas satu basis data. Prinsip ini sejalan dengan elemen kompetensi *mengimplementasikan aturan autentikasi pengguna* pada unit Mengadministrasi Basis Data [1].

C.6 Git sebagai Catatan Kerja

Git adalah sistem kendali versi yang mencatat setiap perubahan berkas dalam bentuk *commit* [9]. Di buku ini, Git dipakai untuk dua hal: menyimpan skrip SQL dan dokumen proyek, serta membuktikan bahwa proyek dikerjakan bertahap. Alur kerja setiap pertemuan tampak pada Gambar 1.3.



Gambar 1.3 Alur kerja Git dari direktori kerja ke GitHub

Tabel 1.3 Ringkasan perintah yang dipakai hari ini

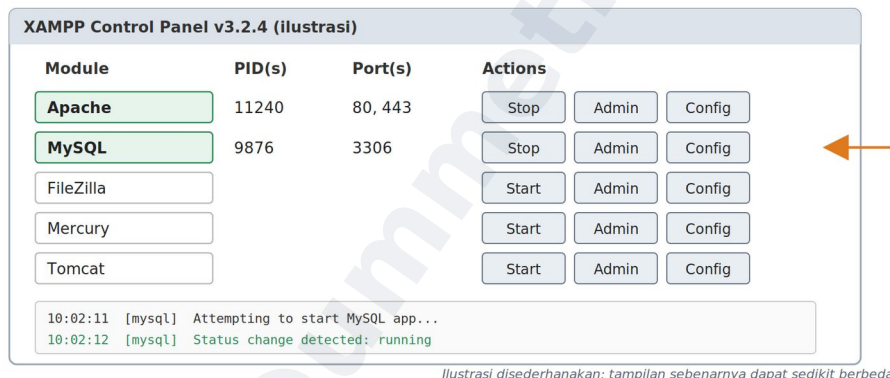
Perintah	Fungsi
<code>mysql -u root -p</code>	Masuk ke server sebagai root, meminta password
<code>SELECT VERSION(), CURRENT_USER();</code>	Menampilkan versi server dan akun yang sedang dipakai
<code>SHOW DATABASES;</code>	Menampilkan basis data yang boleh dilihat akun ini
<code>CREATE DATABASE nama;</code>	Membuat basis data baru
<code>CREATE USER / GRANT</code>	Membuat akun dan memberi hak akses
<code>exit</code> atau <code>\q</code>	Keluar dari klien <code>mysql</code>
<code>git init / add / commit / push</code>	Membuat repositori, menyiapkan, mencatat, dan mengirim perubahan

D. Langkah Percobaan

Seluruh langkah memakai kasus contoh Koperasi Mahasiswa (Kopma). Ganti 123 dengan tiga digit terakhir NIM Anda di setiap perintah.

D.1 Menjalankan Layanan MariaDB

1. Buka **XAMPP Control Panel**. Di komputer lab, jalankan dengan klik kanan → *Run as administrator* bila diminta.
2. Klik **Start** pada baris **MySQL**. Tunggu sampai nama modul berlatar hijau dan kolom *Port(s)* menampilkan 3306, seperti pada Gambar 1.4.
3. Perhatikan panel log di bagian bawah. Baris **Status change detected: running** menandakan server sudah siap menerima koneksi.



Ilustrasi disederhanakan; tampilan sebenarnya dapat sedikit berbeda.

Gambar 1.4 XAMPP Control Panel dengan MariaDB berjalan di port 3306

Titik Analisis: 1

Label tombol bertuliskan “MySQL”, tetapi yang berjalan adalah MariaDB. Apa konsekuensinya ketika Anda mencari solusi galat di internet? Kapan dokumentasi MySQL masih relevan, dan kapan Anda harus merujuk ke dokumentasi MariaDB?

D.2 Masuk Melalui Klien Baris Perintah

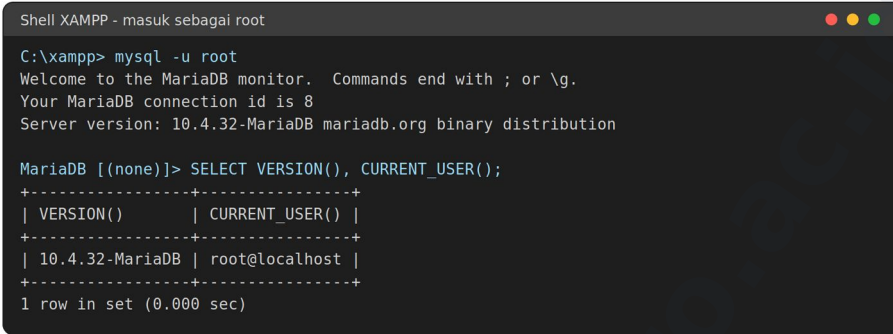
1. Klik tombol **Shell** di XAMPP Control Panel, atau buka Command Prompt jika PATH sudah diatur.
2. Masuk sebagai **root**. Pada pemasangan baru, **root** belum memiliki password sehingga opsi **-p** belum diperlukan.

```
mysql -u root
```

Setelah prompt **MariaDB [(none)]>** muncul, jalankan kueri pertama Anda. Tanda titik koma (;) mengakhiri perintah; tanpa itu, klien akan menunggu kelanjutan baris.

```
SELECT VERSION(), CURRENT_USER();
SHOW DATABASES;
```

Keluaran kueri pertama tampak pada Gambar 1.5. Perintah **SHOW DATABASES** pada XAMPP baru biasanya menampilkan **information_schema**, **mysql**, **performance_schema**, **phpmyadmin**, dan **test**. Empat yang pertama adalah basis data sistem; jangan mengubah isinya.



```

Shell XAMPP - masuk sebagai root

C:\xampp> mysql -u root
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 8
Server version: 10.4.32-MariaDB mariadb.org binary distribution

MariaDB [(none)]> SELECT VERSION(), CURRENT_USER();
+-----+-----+
| VERSION() | CURRENT_USER() |
+-----+-----+
| 10.4.32-MariaDB | root@localhost |
+-----+-----+
1 row in set (0.000 sec)
  
```

Ilustrasi disederhanakan; tampilan sebenarnya dapat sedikit berbeda.

Gambar 1.5 Masuk ke MariaDB sebagai root dan memeriksa versi server

Periksa juga mode SQL server. Beberapa modul (terutama Pertemuan 9 dan 15) mengandalkan **mode ketat**, yaitu server menolak data yang tidak valid alih-alih diam-diam mengubahnya.

```
SELECT @@sql_mode;
```

Catatan: mode ketat

Hasilnya harus memuat **STRICT_TRANS_TABLES**. Nilai bawaan MariaDB 10.4.32 adalah **STRICT_TRANS_TABLES,ERROR_FOR_DIVISION_BY_ZERO,NO_AUTO_CREATE_USER,NO_ENGINE_SUBSTITUTION**. Jika **STRICT_TRANS_TABLES** tidak ada, lihat Masalah 5 di bagian Troubleshooting.

D.3 Mengamankan Akun root

1. Periksa akun **root** yang tersedia di server Anda:

```
SELECT User, Host FROM mysql.user WHERE User = 'root';
```

Pada XAMPP umumnya muncul tiga baris dengan host **localhost**, **127.0.0.1**, dan **:::1**. Beri password yang sama pada **setiap** baris yang muncul. Ganti **PasswordKuat#2026** dengan password Anda sendiri, dan jangan menuliskannya di berkas yang akan di-commit.

```

ALTER USER 'root'@'localhost' IDENTIFIED BY 'PasswordKuat#2026';
ALTER USER 'root'@'127.0.0.1' IDENTIFIED BY 'PasswordKuat#2026';
ALTER USER 'root'@':::1' IDENTIFIED BY 'PasswordKuat#2026';
exit
  
```

2. Masuk kembali. Kali ini opsi **-p** wajib dipakai agar klien meminta password.

```
mysql -u root -p
```

Titik Analisis: 2

Keluar lagi, lalu coba masuk dengan `mysql -u root tanpa -p`. Catat pesan galat yang muncul. Bagian mana dari pesan itu yang memberi tahu bahwa klien tidak mengirim password? Bandingkan jawaban Anda dengan Gambar 1.10 di bagian Troubleshooting.

Perhatian: phpMyAdmin setelah root diberi password

Konfigurasi bawaan phpMyAdmin di XAMPP masuk otomatis sebagai `root` tanpa password. Setelah langkah ini, phpMyAdmin akan menampilkan galat `#1045 Access denied`. Ini wajar dan diperbaiki pada langkah D.5.

D.4 Membuat Basis Data dan Akun Kerja

1. Masih sebagai `root`, buat basis data praktik dan akun kerja yang hanya berhak atas basis data tersebut. Pilihan `utf8mb4` memastikan huruf beraksen dan emoji tersimpan dengan benar.

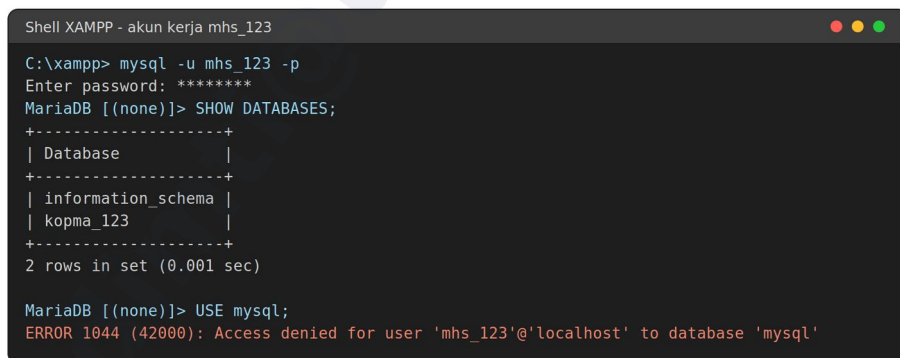
```
CREATE DATABASE kopma_123
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

CREATE USER 'mhs_123'@'localhost' IDENTIFIED BY 'PasswordKerja#123';
GRANT ALL PRIVILEGES ON kopma_123.* TO 'mhs_123'@'localhost';
SHOW GRANTS FOR 'mhs_123'@'localhost';
```

Baris `GRANT ALL PRIVILEGES ON kopma_123.*` berarti: semua hak, tetapi **hanya** pada objek di dalam `kopma_123`. Tanda `.*` menunjuk semua tabel di basis data tersebut.

2. Keluar, lalu masuk dengan akun kerja dan periksa apa yang dapat dilihatnya.

```
mysql -u mhs_123 -p
SHOW DATABASES;
USE mysql;
```



```
Shell XAMPP - akun kerja mhs_123

C:\xampp> mysql -u mhs_123 -p
Enter password: *****
MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| kopma_123 |
+-----+
2 rows in set (0.001 sec)

MariaDB [(none)]> USE mysql;
ERROR 1044 (42000): Access denied for user 'mhs_123'@'localhost' to database 'mysql'
```

Ilustrasi disederhanakan; tampilan sebenarnya dapat sedikit berbeda.

Gambar 1.6 Akun kerja hanya melihat basis data miliknya

Titik Analisis: 3

Mengapa `information_schema` tetap terlihat oleh `mhs_123`, sedangkan `mysql` ditolak dengan `ERROR 1044`? Apa bedanya kode galat 1044 dengan 1045?

D.5 Menggunakan phpMyAdmin dengan Aman

1. Tutup semua jendela peramban, lalu buka berkas `C:\xampp\phpMyAdmin\config.inc.php` dengan editor teks.
2. Cari baris `$cfg['Servers'][$i]['auth_type'] = 'config';` dan ubah nilainya menjadi `'cookie'`. Simpan berkas.
3. Buka `http://localhost/phpmyadmin`. Kini muncul halaman masuk. Masuk sebagai `mhs_123`, buka tab **SQL**, lalu jalankan kueri yang sama seperti di CLI (Gambar 1.7).

`config.inc.php` (baris yang diubah)

```
$cfg['Servers'][$i]['auth_type'] = 'cookie';
```



Ilustrasi disederhanakan; tampilan sebenarnya dapat sedikit berbeda.

Gambar 1.7 Menjalankan kueri melalui tab SQL phpMyAdmin

Titik Analisis: 4

Mode `cookie` meminta pengguna mengetik password setiap kali masuk. Mengapa mode ini lebih aman daripada mode `config` yang menyimpan password di dalam berkas konfigurasi?

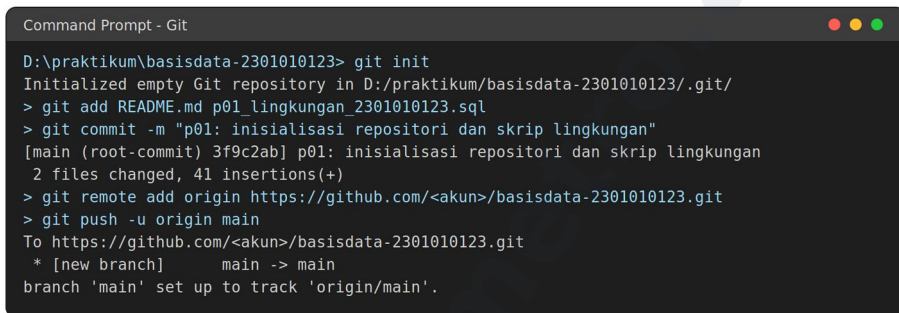
D.6 Menyiapkan Repositori Git

1. Buat folder kerja, misalnya `D:\praktikum\basisdata-<NIM>`, lalu buat dua berkas: `README.md` (identitas: nama, NIM, kelas) dan `p01_lingkungan_<NIM>.sql`.
2. Isi berkas SQL dengan perintah yang Anda jalankan hari ini. Ganti password dengan penanda, bukan password asli.
3. Inisialisasi repositori, commit, lalu kirim ke GitHub. Keluaran yang diharapkan tampak pada Gambar 1.8.

p01_lingkungan_<NIM>.sql

```
-- p01_lingkungan_2301010123.sql
-- Password sengaja diganti penanda. JANGAN commit password asli.
CREATE DATABASE kopma_123
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'mhs_123'@'localhost' IDENTIFIED BY '<password_kerja>';
GRANT ALL PRIVILEGES ON kopma_123.* TO 'mhs_123'@'localhost';

git init
git add README.md p01_lingkungan_2301010123.sql
git commit -m "p01: inisialisasi repositori dan skrip lingkungan"
git remote add origin https://github.com/<akun>/basisdata-2301010123.git
git push -u origin main
```



```
Command Prompt - Git
D:\praktikum\basisdata-2301010123> git init
Initialized empty Git repository in D:\praktikum\basisdata-2301010123\.git/
> git add README.md p01_lingkungan_2301010123.sql
> git commit -m "p01: inisialisasi repositori dan skrip lingkungan"
[main (root-commit) 3f9c2ab] p01: inisialisasi repositori dan skrip lingkungan
 2 files changed, 41 insertions(+)
> git remote add origin https://github.com/<akun>/basisdata-2301010123.git
> git push -u origin main
To https://github.com/<akun>/basisdata-2301010123.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Ilustrasi disederhanakan; tampilan sebenarnya dapat sedikit berbeda.

Gambar 1.8 Inisialisasi repositori dan push pertama ke GitHub

E. Latihan dan Modifikasi

1. Buat akun `tamu_<3d>` yang hanya memiliki hak `SELECT` pada `kopma_<3d>`. Masuk dengan akun tersebut, lalu coba `CREATE TABLE uji (id INT);`. Catat kode galat yang muncul dan jelaskan maknanya.
2. Ubah skrip `p01_lingkungan_<NIM>.sql` agar dapat dijalankan berulang kali tanpa galat. Petunjuk: MariaDB 10.4 mendukung `CREATE DATABASE IF NOT EXISTS` dan `CREATE USER IF NOT EXISTS`. Uji dengan menjalankan skrip dua kali melalui `mysql -u root -p < p01_lingkungan_<NIM>.sql`.

F. Tugas Mandiri: Milestone Proyek 1

Mulai hari ini Anda membangun proyek pribadi yang akan tumbuh sampai UAS. Tentukan tema berdasarkan dua digit terakhir NIM (lihat Peta Capaian Pembelajaran), lalu kerjakan:

1. Buat basis data proyek `<kode_tema>_<3d>` dengan `utf8mb4`, misalnya `perpus_123`.
2. Buat akun `dev_<3d>` yang hanya berhak atas basis data proyek. Buktikan bahwa akun ini tidak dapat membuka `kopma_<3d>`.

3. Tambahkan ke **README.md** bagian **Identitas Proyek**: tema, nama organisasi fiktif yang memuat inisial Anda (contoh: “Perpustakaan Cendekia DI”), serta satu paragraf lingkup layanan organisasi tersebut.
4. Tambahkan berkas **.gitignore** yang mengecualikan berkas berisi kredensial, misalnya ***.env** dan **kredensial*.txt**.
5. Commit dengan pesan **p01: milestone proyek 1** dan push ke GitHub.

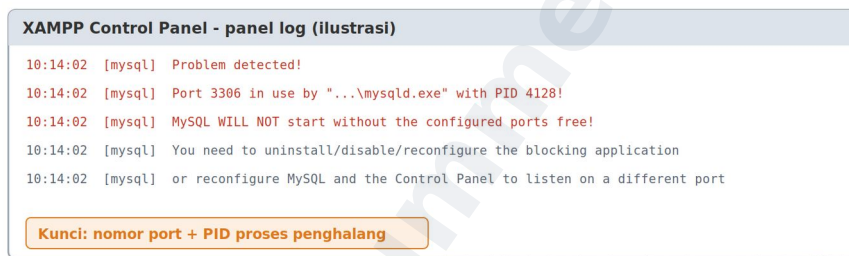
Catatan: hubungan ke pertemuan berikutnya

Organisasi fiktif yang Anda tulis di README menjadi dasar analisis kebutuhan data di Modul 2. Tulis lingkupnya secukupnya, tetapi konkret.

Troubleshooting Umum

Masalah 1: MariaDB gagal dijalankan karena port 3306 terpakai

Gejala. Tombol Start pada MySQL tidak menghasilkan warna hijau, dan panel log menampilkan pesan seperti Gambar 1.9.



Ilustrasi disederhanakan; tampilan sebenarnya dapat sedikit berbeda.

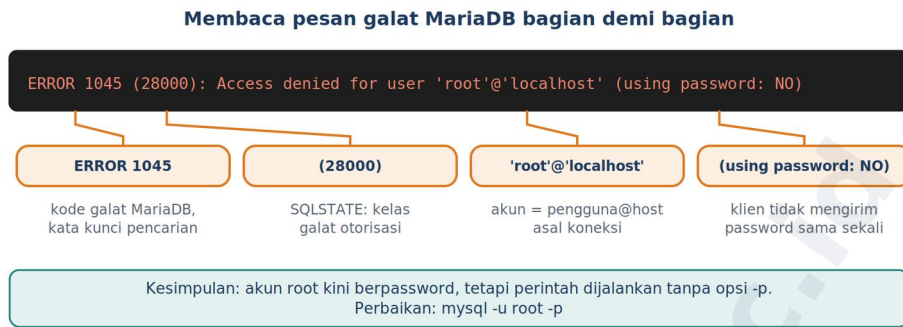
Gambar 1.9 Pesan log XAMPP ketika port 3306 sudah dipakai program lain

Cara membaca. Dua informasi kunci ada di baris kedua: nomor port (**3306**) dan identitas proses penghalang (nama program dan PID). Artinya MariaDB XAMPP sebenarnya sehat, tetapi “kursinya” sudah diduduki program lain, biasanya MySQL atau XAMPP lain yang pernah dipasang sebagai layanan Windows.

Perbaiki. Pastikan pemakai port dengan **netstat -ano | findstr :3306**, lalu hentikan layanan tersebut lewat **Services Windows**. Bila layanan itu memang diperlukan, ubah **port=3306** menjadi **port=3307** pada **C:\\xampp\\mysql\\bin\\my.ini** (bagian **[mysqld]** dan **[client]**), lalu masuk dengan **mysql -u root -p -P 3307**.

Masalah 2: ERROR 1045 Access denied

Cara membaca. Pesan galat MariaDB selalu terdiri atas kode, SQLSTATE, dan teks penjelasan. Gambar 1.10 memecah pesan ini bagian demi bagian. Frasa penentu ada di akhir: **using password: NO** berarti klien tidak mengirim password, sedangkan **using password: YES** berarti password dikirim tetapi salah.



Gambar 1.10 Anatomi pesan ERROR 1045

Perbaikan. Untuk **NO**, tambahkan opsi **-p**. Untuk **YES**, periksa kembali password, termasuk tombol Caps Lock, dan pastikan akun dengan host yang dimaksud memang ada (jalankan kueri `mysql.user` dari langkah D.3 sebagai **root**).

Masalah 3: phpMyAdmin menampilkan #1045 setelah root diberi password

Gejala. Halaman phpMyAdmin menampilkan `mysqli::real_connect(): (HY000/1045): Access denied for user 'root'@'localhost' (using password: NO)`.

Cara membaca. Pesan ini identik dengan Masalah 2, hanya saja “klien”-nya adalah phpMyAdmin. phpMyAdmin masih memakai konfigurasi lama yang masuk sebagai root tanpa password. **Perbaikan:** ubah `auth_type` menjadi `'cookie'` seperti pada langkah D.5. Jangan menuliskan password root ke dalam `config.inc.php`.

Masalah 4: “MySQL shutdown unexpectedly” setelah komputer mati mendadak

Cara membaca. Pesan di Control Panel hanya ringkasan. Penyebab sebenarnya ada di berkas log server: klik tombol **Logs** pada baris MySQL, pilih `mysql_error.log`, lalu baca baris-baris terakhir yang mengandung `[ERROR]`. Biasanya log menyebut berkas InnoDB atau Aria yang rusak.

Perbaikan. Jangan menghapus berkas di folder `C:\xampp\mysql\data` secara acak. Salin dulu seluruh folder tersebut sebagai cadangan, lalu laporkan isi log kepada asisten. Kejadian ini menjadi alasan mengapa skrip SQL Anda wajib selalu tersimpan di GitHub: basis data dapat dibangun ulang dari skrip.

Masalah 5: Mode SQL tidak ketat

Gejala. `SELECT @@sql_mode;` tidak memuat `STRICT_TRANS_TABLES`, sehingga data yang salah (misalnya tanggal 31 September) disimpan dengan peringatan saja, bukan ditolak. **Perbaikan:** buka `C:\xampp\mysql\bin\my.ini`, cari bagian `[mysqld]`, lalu tambahkan atau ubah baris berikut, kemudian Stop dan Start ulang MySQL di XAMPP Control Panel.

my.ini (bagian [mysqld])

```
[mysqld]
sql_mode=STRICT_TRANS_TABLES,ERROR_FOR_DIVISION_BY_ZERO,NO_AUTO_CREATE_
USER,NO_ENGINE_SUBSTITUTION
```

Checklist Laporan Pertemuan 1

Laporan disimpan sebagai `laporan/p01_laporan_<NIM>.md` dengan struktur baku, lalu di-push sebelum batas waktu. Pastikan semua **checklist umum** (lihat Pedoman dan Checklist Laporan Praktikum) dan checklist khusus berikut terpenuhi.

Tabel 1.4 Checklist khusus laporan Pertemuan 1

	Jenis	Yang harus ada
☐	Berkas wajib	<code>p01_lingkungan_<NIM>.sql</code> (dapat dijalankan ulang), <code>README.md</code> berisi Identitas Proyek, <code>.gitignore</code>
☐	Bukti tangkapan layar	<code>SELECT VERSION(), CURRENT_USER(); SELECT @@sql_mode; SHOW DATABASES</code> sebagai <code>mhs_<3d></code> dan <code>dev_<3d></code> ; galat 1044 dan 1142; halaman masuk phpMyAdmin mode <code>cookie</code> ; <code>git push</code> pertama
☐	Analisis wajib	Titik Analisis 1–4; perbedaan kode galat 1044, 1045, dan 1142

Perhatian: aturan keaslian

Setiap tangkapan layar wajib memperlihatkan nama akun `mhs_<3d>` atau `dev_<3d>` dan jam sistem Windows. Tangkapan layar tanpa identitas tidak dinilai.

Nama basis data, akun, dan organisasi fiktif wajib memuat NIM atau inisial Anda. Laporan dengan kalimat pembahasan yang sama persis dengan laporan lain akan diperiksa melalui viva.

Rubrik Penilaian Modul

Tabel 1.5 Rubrik penilaian Modul 1

Aspek	Bobot	Indikator
Pelaksanaan langkah D	25%	Server berjalan, root berpassword, akun kerja dan hak akses benar
Titik Analisis dan latihan E	20%	Jawaban tepat dan beralasan; skrip dapat dijalankan ulang
Milestone Proyek 1	35%	Basis data dan akun proyek benar, <code>README</code> lengkap, <code>.gitignore</code> ada
Laporan dan bukti Git	20%	Lengkap, rapi, commit dengan format yang benar

Sinkronisasi Kompetensi

Tabel 1.6 Unit kompetensi yang dilatih di Modul 1

Kode unit	Judul unit	Elemen yang dilatih
J.62DMS00.017.1	Mengadministrasi Basis Data	Menyiapkan perangkat lunak pengelola basis data; mengimplementasikan aturan autentikasi pengguna
J.62DMS00.019.1	Menggunakan Data	Menetapkan tools yang akan digunakan

Modul 2: Analisis Kebutuhan Pengelolaan Data

Cerita Pertemuan Ini

Server Kopma sudah siap, tetapi Anda belum tahu apa yang harus disimpan. Hari ini Anda duduk bersama ketua, kasir, dan petugas gudang. Keluhan mereka terdengar sederhana: harga di nota lama tidak bisa dicek lagi, stok di buku catatan kadang minus, dan anggota sering lupa membawa kartu. Di balik setiap keluhan tersembunyi kebutuhan data yang harus Anda temukan sebelum menggambar apa pun.

A. Tujuan Praktikum

Setelah menyelesaikan modul ini, mahasiswa mampu:

1. mengidentifikasi aktivitas organisasi, aktor, dan proses bisnis dari narasi, wawancara, dan dokumen sumber;
2. menurunkan elemen data, entitas kandidat, dan aturan bisnis dari dokumen sumber;
3. menyusun matriks proses-data (CRUD) dan kamus data awal yang mencantumkan penanggung jawab data;
4. menulis pernyataan kebutuhan data dan kebutuhan informasi yang spesifik dan dapat diuji.

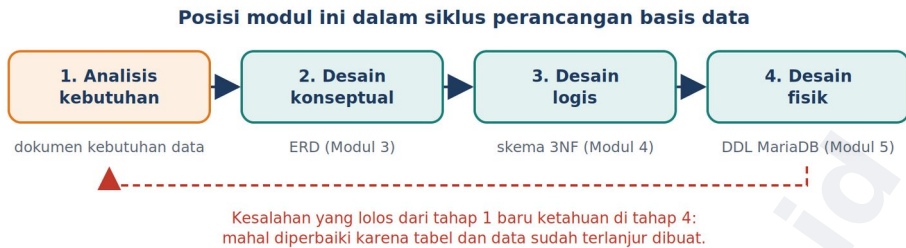
B. Alat dan Bahan

- Narasi studi kasus Kopma, kutipan wawancara, dan nota penjualan di modul ini.
- Editor teks untuk menulis berkas Markdown (`.md`), misalnya Visual Studio Code.
- Repositori `basisdata-<NIM>` dari Modul 1.
- Tidak memerlukan XAMPP; modul ini sepenuhnya kerja analisis.

C. Dasar Teori

C.1 Mengapa Analisis Kebutuhan Didahulukan

Perancangan basis data umumnya berjalan dalam empat tahap: analisis kebutuhan, desain konseptual, desain logis, dan desain fisik [11], [15]. Gambar 2.1 memperlihatkan posisi modul ini sekaligus urutan tiga modul berikutnya. Setiap tahap mengambil keluaran tahap sebelumnya sebagai bahan.



Gambar 2.1 Posisi analisis kebutuhan dalam siklus perancangan basis data

Kesalahan di tahap analisis adalah yang paling mahal. Contohnya, jika harga barang ternyata perlu disimpan per transaksi tetapi hal ini baru disadari setelah ribuan transaksi tercatat, harga lama sudah hilang dan tidak dapat dipulihkan. Karena itu, sebelum menggambar satu entitas pun, Anda harus paham **aktivitas organisasi** dan **data apa yang lahir dari aktivitas itu**.

C.2 Data sebagai Aset Organisasi

Unit kompetensi pertama pada SKKNI Data Management adalah *Mengidentifikasi Kebutuhan Pengelolaan Data untuk Proses Bisnis* [1]. Unit ini meminta Anda memandang data sebagai **aset**: sesuatu yang lahir dari aktivitas organisasi, memiliki penanggung jawab, dan harus dijaga mutunya. Alur berpikirnya diringkas pada Gambar 2.2.



Gambar 2.2 Dari aktivitas organisasi ke aset data dan tata kelolanya

Kajian tata kelola data merumuskan pertanyaan yang sama: siapa yang berwenang atas suatu data, aturan apa yang berlaku, dan bagaimana mutunya dijaga [16]. Pada tingkat praktikum, ketiga pertanyaan ini diwujudkan dalam kolom **penanggung jawab data** (*data steward*) pada kamus data, daftar **aturan bisnis**, dan catatan **isu kualitas** untuk setiap aset. Unit yang sama juga membedakan model tata kelola terpusat, tereplikasi, dan terdistribusi. Konsep tersebut cukup dikenali di sini dan akan dipakai lagi di Buku 2.

C.3 Sumber Kebutuhan dan Jenis Kebutuhan

Kebutuhan data digali dari empat sumber: **wawancara** dengan pengguna, **observasi** proses kerja, **dokumen sumber** (formulir, nota, laporan), dan **sistem lama** jika ada [15]. Dokumen sumber paling berharga bagi pemula, karena setiap isian pada formulir

hampir selalu mewakili satu elemen data. Hasil penggalan kemudian dikelompokkan ke dalam empat jenis kebutuhan berikut.

Tabel 2.1 Jenis kebutuhan yang dicatat dalam dokumen kebutuhan data

Jenis	Pertanyaan pemandu	Contoh (Kopma)
Kebutuhan data	Apa yang harus disimpan?	Nomor nota, tanggal-jam, kasir, barang, qty, harga saat transaksi
Kebutuhan informasi	Pertanyaan apa yang harus bisa dijawab?	Lima barang terlaris per bulan
Aturan bisnis	Batasan apa yang tidak boleh dilanggar?	Stok tidak boleh negatif
Kebutuhan non-fungsional data	Berapa banyak, berapa lama, siapa boleh melihat?	±150 nota per hari, disimpan 5 tahun, nomor HP anggota bersifat pribadi

C.4 Matriks CRUD dan Kamus Data

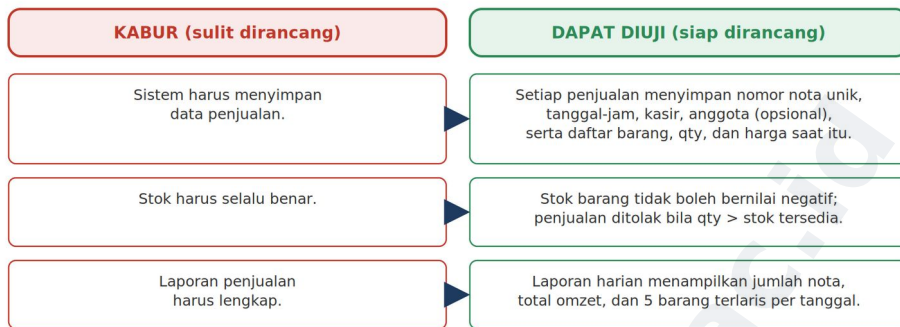
Matriks CRUD memetakan setiap proses bisnis terhadap setiap kelompok data dengan huruf **C** (*create*, membuat), **R** (*read*, membaca), **U** (*update*, mengubah), dan **D** (*delete*, menghapus). Matriks ini alat pemeriksa yang sangat efektif: data yang tidak pernah di-*create* oleh proses mana pun menandakan ada proses yang terlewat, sedangkan data yang dibuat tetapi tidak pernah dibaca patut dipertanyakan manfaatnya [15].

Kamus data mendokumentasikan setiap elemen data: nama, arti, contoh nilai, sumber, aturan, dan penanggung jawab. Kamus data menjadi “bahasa bersama” antara analis, pemrogram, dan pengguna. Praktik ini juga digunakan dalam penelitian sistem informasi di Indonesia, misalnya pada perancangan sistem validasi data pembangunan jaringan fiber optik yang dimulai dari analisis kebutuhan data sebelum perancangan [6].

C.5 Pernyataan Kebutuhan yang Baik

Pernyataan kebutuhan yang baik bersifat **spesifik** (menyebut datanya), **terukur** (ada angka atau kondisi), dan **dapat diuji** (bisa dibuktikan terpenuhi atau tidak). Gambar 2.3 membandingkan pernyataan kabur dengan versi yang siap dirancang. Kualitas pernyataan ini nantinya menentukan kualitas data itu sendiri: penelitian pengukuran kualitas data di instansi pemerintah menunjukkan bahwa dimensi kelengkapan dan akurasi sangat dipengaruhi oleh kejelasan aturan pengisian sejak awal [7].

Pernyataan kebutuhan: kabur vs dapat diuji



Gambar 2.3 Pernyataan kebutuhan kabur dibandingkan pernyataan yang dapat diuji

D. Langkah Percobaan

D.1 Membaca Studi Kasus

Baca narasi berikut dengan teliti. Seluruh langkah di bawah bersumber dari narasi, kutipan wawancara, dan nota ini.

Catatan: Studi kasus Koperasi Mahasiswa Sejahtera (fiktif)

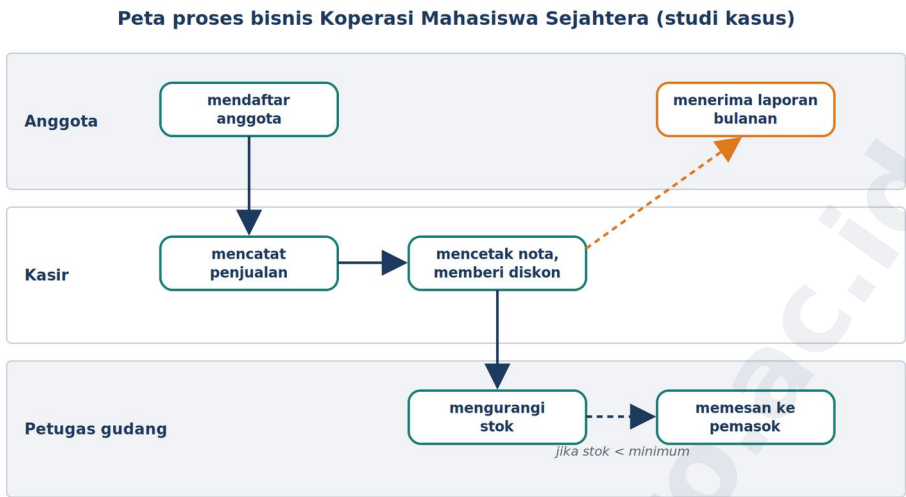
Kopma menjual alat tulis, makanan ringan, dan minuman di lingkungan kampus. Pembeli dapat berupa anggota atau umum. Mahasiswa mendaftar sebagai anggota dengan NIM, nama, program studi, dan nomor HP, lalu memperoleh nomor anggota berformat A-xxxx. Anggota aktif memperoleh diskon 5% untuk setiap nota.

Tiga kasir bekerja bergantian per sif. Kasir mencatat penjualan dan mencetak nota. Setiap sore petugas gudang memeriksa stok; bila stok suatu barang di bawah batas minimum, ia membuat pesanan pembelian ke pemasok. Ketika barang datang, stok bertambah sesuai faktur pemasok. Setiap awal bulan, ketua koperasi menerima laporan omzet, barang terlaris, barang dengan stok menipis, dan anggota paling aktif.

Kutipan wawancara. Ketua: "Harga barang sering naik, jadi kami bingung saat melihat nota lama." Petugas gudang: "Kadang di buku catatan stoknya malah minus." Kasir: "Anggota sering lupa membawa kartu, jadi kami mencarinya lewat NIM."

D.2 Mengidentifikasi Aktor dan Proses Bisnis

1. Tandai setiap kata kerja pada narasi yang menunjukkan kegiatan (mendaftar, mencatat, memeriksa, memesan, menerima).
2. Kelompokkan kegiatan menurut pelakunya, lalu gambarkan alurnya seperti pada peta proses berikut.



Gambar 2.4 Peta proses bisnis Kopma per aktor

Tabel 2.2 Hasil identifikasi aktor dan proses bisnis Kopma

Kode	Proses bisnis	Aktor	Pemicu
PB-01	Mendaftarkan anggota	Kasir (atas permintaan mahasiswa)	Mahasiswa ingin menjadi anggota
PB-02	Mencatat penjualan	Kasir	Pembeli membayar di kasir
PB-03	Memesan barang ke pemasok	Petugas gudang	Stok di bawah batas minimum
PB-04	Menerima barang dari pemasok	Petugas gudang	Barang datang bersama faktur
PB-05	Menyusun laporan bulanan	Ketua koperasi	Awal bulan

D.3 Membedah Dokumen Sumber

1. Amati nota penjualan pada Gambar 2.5. Setiap isian adalah kandidat elemen data.
2. Tandai mana yang **disimpan** dan mana yang **dihitung** (nilai turunan). Nilai turunan seperti subtotal dan total bisa dihitung ulang dari qty, harga, dan diskon.

Membedah dokumen sumber: nota penjualan

KOPERASI MAHASISWA SEJAHTERA			
Nota Penjualan			
No. Nota	:	PJ-2609-0142	
Tanggal	:	24-09-2026 10:15	
Kasir	:	Rina (K03)	
Anggota	:	A-0457 / Budi S.	
Barang	Qty	Harga	Subtotal
Pulpen Gel 0.5	3	4.000	12.000
Buku Tulis 58	2	6.500	13.000
Air Mineral 600	1	4.000	4.000
Jumlah			29.000
Diskon anggota 5%			1.450
TOTAL			27.550
Bayar tunai			30.000
Kembali			2.450

identitas transaksi

relasi ke Kasir & Anggota

barang, qty, harga saat transaksi

nilai turunan (dihitung)

data pembayaran

Gambar 2.5 Nota penjualan Kopma dan anotasi elemen datanya

Titik Analisis: 1

Harga barang sudah tersimpan di data barang. Mengapa nota tetap perlu menyimpan harga **saat transaksi**? Hubungkan jawaban Anda dengan keluhan ketua koperasi dalam kutipan wawancara.

Titik Analisis: 2

Subtotal dan total adalah nilai turunan. Sebutkan satu alasan untuk **tidak** menyimpannya dan satu alasan yang mungkin membuat total tetap disimpan. Keputusan akhirnya akan dibahas di Modul 4.

D.4 Menyusun Entitas Kandidat dan Aturan Bisnis

1. Kelompokkan elemen data yang menggambarkan “hal” yang sama menjadi entitas kandidat.
2. Tulis aturan bisnis dengan kode unik agar dapat dirujuk di modul-modul berikutnya.

Tabel 2.3 Entitas kandidat Kopma

Entitas kandidat	Elemen data utama	Sumber
Anggota	nomor anggota, NIM, nama, program studi, nomor HP, status aktif	Formulir pendaftaran
Barang	kode, nama, kategori, harga jual, stok, batas minimum stok	Daftar barang, faktur
Penjualan	nomor nota, tanggal-jam, kasir, anggota (opsional), bayar	Nota penjualan
Detail penjualan	nomor nota, barang, qty, harga saat transaksi	Nota penjualan

Entitas kandidat	Elemen data utama	Sumber
Petugas	kode petugas, nama, peran (kasir/gudang/ketua)	Wawancara
Pemasok	kode, nama, telepon, alamat	Faktur pemasok
Pembelian dan detailnya	nomor faktur, tanggal, pemasok, barang, qty, harga beli	Faktur pemasok

Tabel 2.4 Aturan bisnis Kopma

Kode	Aturan bisnis
AB-01	Setiap nota memiliki nomor unik dan minimal satu baris barang.
AB-02	Penjualan boleh tanpa anggota (pembeli umum); jika ada, anggota harus berstatus aktif untuk memperoleh diskon 5%.
AB-03	Stok barang tidak boleh negatif; penjualan ditolak bila qty melebihi stok tersedia.
AB-04	Harga jual yang dipakai pada nota disimpan per baris dan tidak berubah meski harga barang kemudian naik.
AB-05	NIM anggota unik; pencarian anggota dapat dilakukan lewat nomor anggota atau NIM.
AB-06	Pesanan pembelian dibuat bila stok kurang dari batas minimum barang tersebut.

D.5 Kebutuhan Informasi dan Matriks CRUD

1. Tulis setiap laporan yang diminta ketua koperasi sebagai kebutuhan informasi, lengkap dengan data yang diperlukan.
2. Susun matriks CRUD proses terhadap entitas.

Tabel 2.5 Kebutuhan informasi Kopma

Kode	Kebutuhan informasi	Data yang diperlukan
KI-01	Omzet dan jumlah nota per hari dan per bulan	Penjualan, detail penjualan
KI-02	Lima barang terlaris per bulan berdasarkan qty	Detail penjualan, barang
KI-03	Barang dengan stok di bawah batas minimum	Barang
KI-04	Sepuluh anggota dengan belanja terbesar per bulan	Penjualan, detail penjualan, anggota

Tabel 2.6 Matriks CRUD proses bisnis terhadap entitas Kopma

Proses	Anggota	Barang	Penjualan	Detail	Pemasok	Pembelian
PB-01 Daftar anggota	C					

Proses	Anggota	Barang	Penjualan	Detail	Pemasok	Pembelian
PB-02 Catat penjualan	R	R, U	C	C		
PB-03 Pesan ke pemasok		R			R	C
PB-04 Terima barang		U			R	U
PB-05 Laporan bulanan	R	R	R	R		R

Titik Analisis: 3

Perhatikan kolom **Pemasok**. Tidak ada proses yang memberi huruf **C**. Apa artinya, dan proses apa yang perlu ditambahkan ke daftar proses bisnis? Periksa juga: siapa yang mengubah status aktif anggota?

D.6 Kamus Data Awal dan Kebutuhan Non-Fungsional

Kamus data awal cukup memuat elemen penting dari setiap entitas. Kolom **penanggung jawab** mewujudkan prinsip tata kelola dari bagian C.2. Contoh sebagian isinya ada pada Tabel 2.7.

Tabel 2.7 Cuplikan kamus data awal Kopma

Elemen	Arti	Contoh	Aturan	Penanggung jawab
no_anggota	Nomor anggota koperasi	A-0457	Unik, format A-4 digit	Ketua
nim_anggota	NIM anggota	2301010123	Unik, 10 digit	Ketua
no_hp_anggota	Nomor HP anggota	0812xxxx	Data pribadi, akses terbatas	Ketua
no_nota_penjualan	Nomor nota penjualan	PJ-2609-0142	Unik per nota	Kasir
harga_satuan_detail_penjualan	Harga jual saat transaksi	4000	Bilangan bulat ≥ 0 (rupiah)	Kasir
stok_barang	Jumlah barang tersedia	35	Bilangan bulat ≥ 0 (AB-03)	Petugas gudang

Kebutuhan non-fungsional dicatat singkat: perkiraan ± 150 nota per hari, data transaksi disimpan minimal lima tahun, dan nomor HP anggota hanya boleh dilihat oleh ketua. Pembatasan akses data pribadi seperti ini sejalan dengan kewajiban pengendali data dalam Undang-Undang Pelindungan Data Pribadi [17].

D.7 Menulis Dokumen Kebutuhan Data

1. Pada repositori Anda, buat berkas `p02_kebutuhan_data_kopma_<NIM>.md` dengan kerangka berikut, lalu salin hasil langkah D.2–D.6 ke dalamnya.
2. Commit dengan pesan `p02: dokumen kebutuhan data kopma`.

Kerangka berkas dokumen kebutuhan data

```
# Dokumen Kebutuhan Data - <Nama Organisasi>
## 1. Latar belakang dan aktivitas organisasi
## 2. Aktor dan proses bisnis (tabel PB-xx)
## 3. Dokumen sumber yang dianalisis
## 4. Entitas kandidat dan elemen data
## 5. Aturan bisnis (tabel AB-xx)
## 6. Kebutuhan informasi (tabel KI-xx)
## 7. Matriks CRUD
## 8. Kamus data awal (dengan penanggung jawab)
## 9. Kebutuhan non-fungsional data (volume, retensi, privasi)
## 10. Isu kualitas data yang diantisipasi
```

E. Latihan dan Modifikasi

1. Kopma ingin memberi **poin loyalitas**: setiap kelipatan Rp10.000 belanja anggota bernilai 1 poin, dan 50 poin dapat ditukar dengan potongan Rp5.000. Tambahkan elemen data, aturan bisnis (AB-07 dan seterusnya), kebutuhan informasi, dan perbarui matriks CRUD.
2. Perbaiki tiga pernyataan kebutuhan kabur berikut agar dapat diuji: (a) “data anggota harus aman”; (b) “sistem harus cepat mencari barang”; (c) “laporan stok harus akurat”.

F. Tugas Mandiri: Milestone Proyek 2

Susun dokumen kebutuhan data untuk organisasi fiktif proyek Anda dengan kerangka pada langkah D.7. Berkas diberi nama `p02_kebutuhan_data_<NIM>.md`. Ketentuan minimum:

- minimal 4 proses bisnis, 6 entitas kandidat, 8 aturan bisnis, dan 5 kebutuhan informasi;
- matriks CRUD lengkap tanpa entitas yang tidak pernah di-*create* (kecuali dijelaskan alasannya);
- kamus data awal minimal 20 elemen dengan kolom penanggung jawab;
- kebutuhan non-fungsional, termasuk identifikasi **data pribadi** dan siapa yang boleh mengaksesnya;
- minimal satu dokumen sumber fiktif yang Anda rancang sendiri (formulir, nota, atau laporan) beserta pembedahannya.

Tabel 2.8 Titik awal analisis per tema proyek

Tema	Dokumen sumber yang disarankan	Perhatian khusus
Perpustakaan	Formulir anggota, slip peminjaman, kuitansi denda	Satu judul buku bisa memiliki banyak eksemplar
Klinik	Formulir pendaftaran pasien, lembar kunjungan, resep	Data kesehatan termasuk data pribadi yang bersifat spesifik; akses sangat terbatas
Toko daring	Halaman pesanan, bukti pembayaran, resi pengiriman	Harga, ongkos kirim, dan alamat disimpan per pesanan
Akademik	KRS, jadwal kuliah, kartu hasil studi	Mata kuliah berbeda dengan kelas yang dibuka pada semester tertentu

Perhatian: parameter personal berbasis NIM

Hitung $P = (2 \text{ digit terakhir NIM mod } 9) + 1$. Gunakan P untuk: batas maksimal item per transaksi = $P + 2$; persentase diskon atau denda harian (dalam ribu rupiah) = P ; perkiraan volume transaksi harian = $40 + 5 \times P$. Cantumkan perhitungan P di bagian 9 dokumen Anda. Nilai P dipakai kembali di Modul 6 untuk data sampel.

Troubleshooting Umum

Pada modul analisis, “galat” tidak muncul di layar, tetapi di dalam dokumen. Kesalahan berikut paling sering ditemukan saat pemeriksaan milestone.

Masalah 1: Kebutuhan terlalu umum

Gejala. Dokumen berisi kalimat seperti “sistem menyimpan data transaksi”. **Cara mendeteksi:** coba turunkan kalimat itu menjadi kolom tabel. Jika Anda tidak bisa menyebut satu pun nama kolom, pernyataannya terlalu umum. **Perbaikan:** sebutkan elemen datanya satu per satu dan beri aturan yang dapat diuji, seperti pada contoh di bagian C.5.

Masalah 2: Proses tertulis sebagai entitas

Gejala. Muncul entitas bernama kata kerja, seperti “Mencatat Peminjaman” atau “Mengecek Stok”. **Cara mendeteksi:** entitas adalah benda atau kejadian yang dicatat, bukan tindakan. **Perbaikan:** ubah menjadi kata benda (“Peminjaman”) dan pindahkan kata kerjanya ke daftar proses bisnis.

Masalah 3: Data historis terhapus tanpa sadar

Gejala. Harga, alamat, atau tarif hanya disimpan di entitas induk. **Cara mendeteksi:** tanyakan “jika nilai ini berubah bulan depan, apakah catatan lama harus tetap menampilkan nilai lama?” Jika ya, nilai tersebut harus disimpan di entitas transaksinya, seperti aturan AB-04.

Masalah 4: Entitas yatim di matriks CRUD

Gejala. Ada kolom entitas tanpa huruf C, atau baris proses yang kosong. **Perbaikan:** tambahkan proses pemeliharaan data (misalnya “mengelola data pemasok”) atau hapus entitas yang ternyata tidak diperlukan. Pemeriksaan ini adalah jawaban Titik Analisis 3.

Checklist Laporan Pertemuan 2

Laporan disimpan sebagai `laporan/p02_laporan_<NIM>.md` dengan struktur baku, lalu di-push sebelum batas waktu. Pastikan semua **checklist umum** (lihat Pedoman dan Checklist Laporan Praktikum) dan checklist khusus berikut terpenuhi.

Tabel 2.9 Checklist khusus laporan Pertemuan 2

	Jenis	Yang harus ada
☐	Berkas wajib	<code>p02_kebutuhan_data_kopma_<NIM>.md</code> dan <code>p02_kebutuhan_data_<NIM>.md</code> (proyek)
☐	Bukti tangkapan layar	Tabel PB, AB, KI, matriks CRUD, dan kamus data di berkas proyek (tangkapan tampilan GitHub)
☐	Analisis wajib	Titik Analisis 1–3; perhitungan parameter P; perbaikan tiga pernyataan kebutuhan kabur

Perhatian: aturan keaslian

Nama organisasi, dokumen sumber fiktif, dan parameter P bersifat personal. Dua dokumen dengan aturan bisnis atau kamus data yang sama persis akan diperiksa melalui viva, dan mahasiswa wajib menjelaskan asal setiap aturan dari proses bisnisnya.

Rubrik Penilaian Modul

Tabel 2.10 Rubrik penilaian Modul 2

Aspek	Bobot	Indikator
Pelaksanaan langkah D	20%	Tabel proses, entitas, aturan, dan matriks Kopma benar
Titik Analisis dan latihan E	20%	Alasan tepat; pernyataan kebutuhan hasil perbaikan dapat diuji
Milestone Proyek 2	45%	Memenuhi ketentuan minimum, konsisten, dan menggunakan parameter P
Laporan dan bukti Git	15%	Lengkap, rapi, commit berformat benar

Sinkronisasi Kompetensi

Tabel 2.11 Unit kompetensi yang dilatih di Modul 2

Kode unit	Judul unit	Elemen yang dilatih
J.62DMS00.001.1	Mengidentifikasi Kebutuhan Pengelolaan Data untuk Proses Bisnis	Menentukan bentuk aset data organisasi; menentukan aspek tata kelola aset data; mengidentifikasi unsur pengelolaan aset data (dokumentasi aturan dan isu kualitas data). Tipe model operasional tata kelola baru diperkenalkan.

Modul 3: Pemodelan Data dengan Entity-Relationship Diagram

Cerita Pertemuan Ini

Dokumen kebutuhan data dari Pertemuan 2 sudah disetujui ketua Kopma. Namun saat Anda menjelaskannya, pengurus kebingungan membaca tabel yang panjang. Ketua bertanya, “Bisakah digambar saja, supaya kami tahu data apa berhubungan dengan apa?” Hari ini Anda mengubah daftar kebutuhan menjadi satu gambar yang dapat dibaca bersama: ERD.

A. Tujuan Praktikum

Setelah menyelesaikan modul ini, mahasiswa mampu:

1. mengidentifikasi entitas, atribut, dan kunci dari dokumen kebutuhan data;
2. menentukan relasi beserta kardinalitas dan partisipasinya dengan alasan dari aturan bisnis;
3. memecah relasi banyak-ke-banyak menjadi entitas asosiatif;
4. menggambar ERD notasi Crow's Foot dan memvalidasinya terhadap kebutuhan informasi (KI).

B. Alat dan Bahan

- Dokumen `p02_kebutuhan_data_kopma_<NIM>.md` dan dokumen kebutuhan data proyek dari Pertemuan 2.
- draw.io (diagrams.net) versi web atau desktop, gratis, untuk menggambar ERD konseptual.
- phpMyAdmin Designer (XAMPP 8.2.12) baru dipakai pada Modul 5 untuk mencocokkan ERD dengan tabel nyata.
- Repositori `basisdata-<NIM>`.

C. Dasar Teori

C.1 Unsur-unsur ERD

Model Entity-Relationship menggambarkan data sebagai **entitas** (hal yang dicatat), **atribut** (sifat entitas), dan **relasi** (hubungan antarentitas) [11]. ERD adalah hasil tahap desain konseptual: belum memikirkan tipe data atau DBMS tertentu, tetapi sudah memastikan semua kebutuhan data tercakup [15].

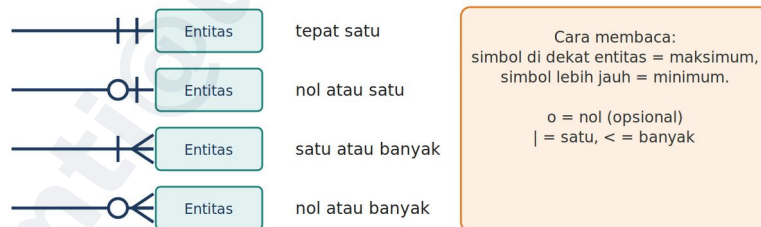
Tabel 3.1 Unsur ERD dan contohnya di Kopma

Unsur	Arti	Contoh Kopma
Entitas	Benda atau kejadian yang datanya disimpan	anggota, barang, penjualan
Atribut kunci	Atribut yang membedakan setiap anggota entitas	id_anggota ; kunci alternatif nim_anggota
Atribut turunan	Dapat dihitung dari atribut lain	total nota = jumlah qty × harga_satuan
Relasi	Hubungan bermakna antarentitas, dibaca sebagai kata kerja	petugas melayani penjualan
Entitas asosiatif	Pengganti relasi banyak-ke-banyak yang punya atribut sendiri	detail_penjualan (qty_detail_penjualan, harga_satuan_detail_penjualan)

C.2 Kardinalitas dan Partisipasi

Kardinalitas menjawab “berapa banyak paling banyak”, sedangkan partisipasi menjawab “wajib atau boleh tidak ada”. Notasi Crow's Foot menggabungkan keduanya di ujung garis (Gambar 3.1). Kunci membacanya: simbol yang dekat dengan kotak entitas menyatakan maksimum, sedangkan simbol yang lebih jauh menyatakan minimum.

Notasi Crow's Foot: membaca kardinalitas dari ujung garis



Contoh Kopma: penjualan — anggota bernilai “nol atau satu” (pembeli umum boleh).

Gambar 3.1 Notasi Crow's Foot untuk kardinalitas dan partisipasi

Kardinalitas tidak boleh ditebak. Setiap ujung garis harus dapat ditelusuri ke aturan bisnis. Misalnya, aturan AB-02 (penjualan boleh tanpa anggota) menjadikan sisi anggota pada relasi penjualan bernilai **nol atau satu**.

C.3 Relasi Banyak-ke-Banyak

Satu nota berisi banyak barang, dan satu barang muncul di banyak nota. Relasi seperti ini tidak dapat disimpan langsung dalam tabel relasional, dan lagi pula relasinya memiliki atribut sendiri (qty dan harga saat transaksi, AB-04). Solusinya adalah **entitas asosiatif**

`detail_penjualan` yang berhubungan satu-ke-banyak dengan kedua entitas asalnya [12].

D. Langkah Percobaan

D.1 Dari Entitas Kandidat ke Entitas

- 1. Buka tabel entitas kandidat Kopma dari Pertemuan 2. Tandai atribut kunci, kunci alternatif (misalnya `nim_anggota` dan `no_nota_penjualan`), dan atribut turunan.
- 2. Pecah entitas kandidat “Pembelian dan detailnya” menjadi `pembelian` dan `detail_pembelian`, dengan pola yang sama seperti penjualan.

Titik Analisis: 1
`no_anggota` dan `nim_anggota` sama-sama unik. Mengapa buku ini tetap memakai `id_anggota` sebagai kunci primer? Sebutkan satu risiko jika NIM dijadikan kunci primer.

D.2 Menentukan Relasi dan Kardinalitas

- 3. Untuk setiap pasangan entitas yang berhubungan, tulis kalimat relasi dengan kata kerja, lalu jawab dua pertanyaan dari kedua arah: “minimal berapa?” dan “maksimal berapa?”.
- 4. Isikan hasilnya ke tabel seperti Tabel 3.2, lengkap dengan kode aturan bisnis yang menjadi dasarnya.

Tabel 3.2 Relasi dan kardinalitas Kopma

Relasi	Sisi kiri	Sisi kanan	Dasar
petugas melayani penjualan	tepat satu petugas	nol atau banyak penjualan	wawancara kasir
anggota melakukan penjualan	nol atau satu anggota	nol atau banyak penjualan	AB-02
penjualan berisi detail_penjualan	tepat satu penjualan	satu atau banyak detail	AB-01
barang tercantum di detail_penjualan	tepat satu barang	nol atau banyak detail	AB-04
pemasok menerima pembelian	tepat satu pemasok	nol atau banyak pembelian	AB-06
pembelian berisi detail_pembelian	tepat satu pembelian	satu atau banyak detail	faktur pemasok

Titik Analisis: 2

Sisi penjualan terhadap detail penjualan bernilai **satu atau banyak**, bukan nol atau banyak. Aturan bisnis mana yang memaksanya? Apakah aturan “minimal satu” ini dapat dijaga oleh struktur tabel saja? Simpan jawaban Anda; Modul 14 akan menjawabnya dengan transaksi.

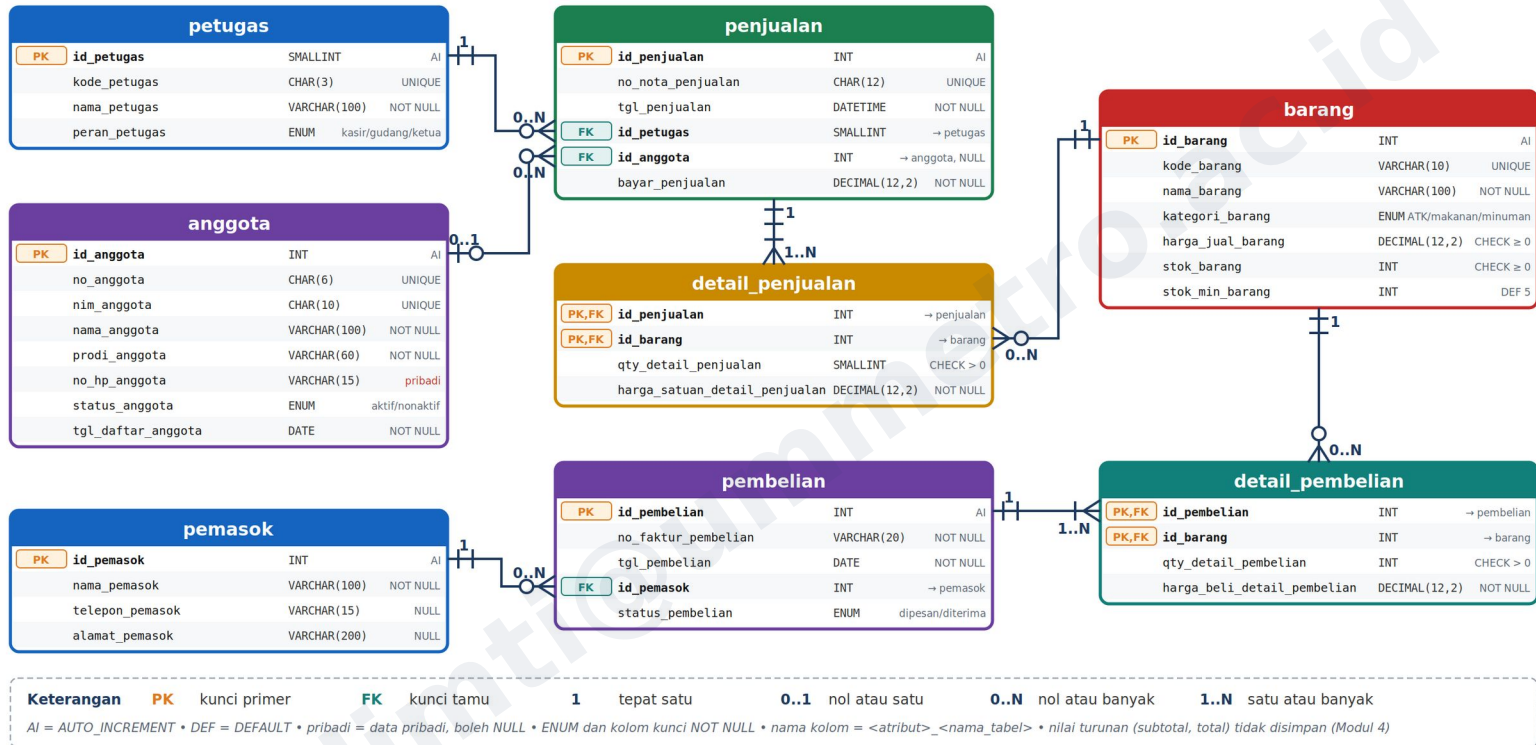
D.3 Menggambar ERD

5. Buka draw.io, pilih pustaka bentuk **Entity Relation**, lalu gambar delapan entitas beserta atributnya. Tandai kunci primer (PK) dan kunci tamu (FK). Bandingkan hasilnya dengan ERD Kopma pada Gambar 3.2. Pada tahap ini cukup gambarkan tabel, nama kolom, kunci, dan kardinalitas; tipe data dan constraint yang juga tampak pada gambar itu baru ditetapkan di Modul 5.
6. Pastikan ERD memenuhi standar kelengkapan pada tabel berikut, lalu simpan dua berkas: `p03_erd_kopma_<NIM>.drawio` (sumber) dan `p03_erd_kopma_<NIM>.png` (gambar).

Tabel 3.3 Standar kelengkapan ERD konseptual di buku ini

Unsur	Ketentuan
Entitas	Nama tabel huruf kecil tunggal di kepala kotak, satu warna per entitas
Atribut	Semua atribut dari kamus data tercantum, bukan hanya kunci
Kunci	Kunci primer diberi label PK, kunci tamu FK, kunci primer komposit PK,FK
Kardinalitas	Kedua ujung garis memakai simbol Crow's Foot dan label angka (1, 0..1, 0..N, 1..N)
Garis relasi	Ditarik dari kunci primer ke kunci tamu yang dirujuknya, tidak saling menumpuk
Keterangan	Kotak legenda simbol dan label di bagian bawah diagram

ERD fisik Kopma: setiap kolom beserta tipe data dan constraint (acuan DDL Modul 5)



Gambar 3.2 ERD Kopma: seluruh tabel, kolom, tipe data, constraint, dan kardinalitas

D.4 Memvalidasi ERD terhadap Kebutuhan Informasi

7. Untuk setiap KI dari Pertemuan 2, telusuri jalur di ERD yang menjawabnya. Contoh KI-04 (anggota dengan belanja terbesar): anggota – penjualan – detail_penjualan.
8. Jika ada KI yang tidak dapat ditelusuri, berarti ERD kurang entitas, atribut, atau relasi. Perbaiki sebelum lanjut.
9. Kirim ERD kepada “klien” untuk ditinjau. Di kelas, pasangan sebelah Anda berperan sebagai ketua Kopma dan memberi minimal dua umpan balik. Catat umpan balik dan keputusan Anda (diterima/ditolak beserta alasan) di bagian akhir berkas ERD.
10. Commit dengan pesan `p03: erd kopma`.

Titik Analisis: 3

Ketua ingin mengetahui “petugas gudang mana yang menerima barang dari pemasok”. Apakah ERD saat ini dapat menjawabnya? Jika belum, relasi apa yang perlu ditambahkan, dan apakah keputusan itu perlu dicatat sebagai aturan bisnis baru?

E. Latihan dan Modifikasi

1. Tambahkan ke ERD Kopma kebutuhan poin loyalitas dari latihan Pertemuan 2. Tentukan apakah poin cukup menjadi atribut anggota, atau perlu entitas riwayat poin. Tuliskan alasannya.
2. Gambar ulang relasi penjualan–barang sebagai banyak-ke-banyak langsung (tanpa entitas asosiatif), lalu jelaskan data apa yang hilang.

F. Tugas Mandiri: Milestone Proyek 3

- Gambar ERD proyek (`p03_erd_<NIM>.drawio` dan `.png`) dari dokumen kebutuhan data Pertemuan 2.
- Buat tabel relasi dan kardinalitas seperti pada langkah D.2, masing-masing dengan kode AB yang mendasarinya.
- Buat tabel penelusuran KI – jalur ERD untuk semua KI proyek.
- Minimal satu entitas asosiatif dengan atribut sendiri.
- ERD memenuhi seluruh standar kelengkapan ERD konseptual (Tabel 3.3).
- Catatan tinjauan klien: minimal dua umpan balik dari teman sekelas dan keputusan Anda atas masing-masing.

Catatan: hubungan ke pertemuan berikutnya

ERD proyek akan diuji dengan normalisasi di Pertemuan 4. Jika normalisasi menemukan masalah, perbaiki ERD dan catat perubahannya di commit, jangan dihapus diam-diam.

Troubleshooting Umum

Masalah 1: Garis relasi tanpa kata kerja

Gejala. ERD penuh garis tetapi tidak jelas maknanya. **Cara mendeteksi:** coba bacakan setiap garis sebagai kalimat. Jika tidak bisa, relasinya belum dipahami. **Perbaikan:** beri label kata kerja dan pastikan setiap label berasal dari proses bisnis PB-xx.

Masalah 2: Kardinalitas terbalik

Gejala. Crow's foot berada di sisi anggota, sehingga terbaca “satu penjualan memiliki banyak anggota”. **Perbaikan:** baca dari entitas di seberang: “untuk **satu** penjualan, berapa anggota paling banyak?”. Jawaban itulah yang digambar di ujung dekat entitas anggota.

Masalah 3: Atribut turunan digambar sebagai atribut biasa

Gejala. Entitas penjualan memiliki atribut **total**. **Perbaikan:** tandai sebagai atribut turunan dan tunda keputusan menyimpan atau menghitungnya ke Pertemuan 4. Buku ini memutuskan total **dihitung**.

Checklist Laporan Pertemuan 3

Laporan disimpan sebagai `laporan/p03_laporan_<NIM>.md` dengan struktur baku, lalu di-push sebelum batas waktu. Pastikan semua **checklist umum** (lihat Pedoman dan Checklist Laporan Praktikum) dan checklist khusus berikut terpenuhi.

Tabel 3.4 Checklist khusus laporan Pertemuan 3

	Jenis	Yang harus ada
☐	Berkas wajib	<code>p03_erd_kopma_<NIM></code> dan <code>p03_erd_<NIM></code> (gambar + berkas sumber diagram)
☐	Bukti tangkapan layar	ERD di draw.io dengan kardinalitas terbaca; catatan tinjauan klien
☐	Analisis wajib	Pemetaan setiap entitas ke PB/KI dari Pertemuan 2; alasan kardinalitas relasi utama

Perhatian: aturan keaslian

Nama berkas, nama organisasi pada judul diagram, dan tanda tangan digital di sudut ERD (nama + NIM) wajib ada. ERD proyek dengan susunan dan nama atribut identik dengan mahasiswa lain akan diperiksa melalui *viva*.

Rubrik Penilaian Modul

Tabel 3.5 Rubrik penilaian Modul 3

Aspek	Bobot	Indikator
Pelaksanaan langkah D	25%	ERD Kopma lengkap, notasi benar, KI tertelusuri
Titik Analisis dan latihan E	20%	Alasan kunci dan kardinalitas tepat
Milestone Proyek 3	40%	ERD proyek lengkap, kardinalitas berdasar AB, penelusuran KI
Laporan dan bukti Git	15%	Lengkap sesuai checklist, commit berformat benar

Sinkronisasi Kompetensi

Tabel 3.6 Unit kompetensi yang dilatih di Modul 3

Kode unit	Judul unit	Elemen yang dilatih
J.62DMS00.006.1	Merancang Basis Data	Elemen 1: model konseptual dibuat, ditinjau klien, dan umpan balik dievaluasi (KUK 1.3–1.5); Elemen 2: ERD dibuat untuk memperjelas kardinalitas relasi (KUK 2.3)

Modul 4: Normalisasi dan Skema Relasional

Cerita Pertemuan Ini

Ketua Kopma menyerahkan berkas spreadsheet yang dipakai selama ini. “Semua ada di sini,” katanya. Namun Anda segera menemukan nama anggota yang sama ditulis dua cara, nama pemasok yang ejaannya berubah-ubah, dan harga pulpen yang berbeda pada bulan yang sama. ERD dari Pertemuan 3 perlu diuji: apakah rancangan itu benar-benar bebas dari masalah seperti ini? Normalisasi adalah alat ujinya.

A. Tujuan Praktikum

Setelah menyelesaikan modul ini, mahasiswa mampu:

1. mengidentifikasi anomali ubah, sisip, dan hapus pada data yang belum dinormalisasi;
2. menuliskan dependensi fungsional dari aturan bisnis;
3. menormalisasi relasi hingga bentuk normal ketiga (3NF) dan memeriksa BCNF;
4. memutuskan penyimpanan nilai turunan dan mendokumentasikan skema relasional akhir.

B. Alat dan Bahan

- Spreadsheet lama Kopma pada modul ini.
- ERD Kopma dan ERD proyek dari Pertemuan 3.
- Editor Markdown dan repositori [basisdata-<NIM>](#).

C. Dasar Teori

C.1 Anomali: Mengapa Data Perlu Dinormalisasi

Data yang disimpan berulang dalam satu tabel besar menimbulkan tiga **anomali** [11]. Gambar 4.1 memperlihatkanannya pada spreadsheet Kopma: anomali ubah (satu fakta tertulis di banyak baris dan bisa tidak konsisten), anomali sisip (fakta tidak dapat dicatat sebelum ada fakta lain), dan anomali hapus (menghapus satu fakta ikut menghapus fakta lain).

Spreadsheet lama Kopma dan tiga anomali							
no_nota	tgl	kasir	anggota	barang	qty	harga	pemasok_brg
PJ-0142	24/09	Rina	Budi S.	Pulpen Gel 0.5	3	4000	CV Sumber ATK
PJ-0142	24/09	Rina	Budi S.	Buku Tulis 58	2	6500	CV Sumber ATK
PJ-0143	24/09	Rina	-	Roti Cokelat	2	5000	UD Segar
PJ-0150	25/09	Rina	Budi Santoso	Pulpen Gel 0.5	1	4500	CV. Sumber A.T.K

Anomali ubah
"Budi S." vs "Budi Santoso";
pemasok ditulis berbeda

Anomali sisip
barang baru tak bisa dicatat
sebelum ada penjualannya

Anomali hapus
hapus nota PJ-0143 ikut
menghapus info pemasok UD Segar

Gambar 4.1 Spreadsheet lama Kopma dan tiga anomali

C.2 Dependensi Fungsional

Atribut B **bergantung secara fungsional** pada A (ditulis $A \rightarrow B$) jika setiap nilai A selalu menentukan tepat satu nilai B. Dependensi fungsional bukan ditebak dari contoh data, tetapi diturunkan dari aturan bisnis [12]. Contohnya, `no_nota_penjualan` → `tgl_penjualan` benar karena satu nota hanya punya satu tanggal. Sebaliknya, `kode_barang` → `harga_satuan_detail_penjualan` **salah** untuk Kopma, karena AB-04 menyatakan harga di nota dapat berbeda dari harga barang saat ini.

C.3 Bentuk Normal

Normalisasi dilakukan bertahap (Gambar 4.2). Setiap tahap menghilangkan satu jenis ketergantungan yang tidak diinginkan [15].



Gambar 4.2 Tahapan normalisasi UNF sampai 3NF

Tabel 4.1 Ringkasan bentuk normal

Bentuk	Syarat	Pertanyaan pemeriksa
1NF	Semua nilai atomik; tidak ada grup berulang	Adakah sel berisi daftar atau kolom barang1, barang2, ...?
2NF	1NF dan tidak ada atribut bukan kunci yang bergantung pada sebagian kunci komposit	Adakah atribut yang cukup ditentukan oleh sebagian kunci?
3NF	2NF dan tidak ada atribut bukan kunci yang bergantung pada atribut bukan kunci lain	Adakah rantai $A \rightarrow B \rightarrow C$ di dalam satu tabel?

Bentuk	Syarat	Pertanyaan pemeriksa
BCNF	Setiap penentu adalah kunci kandidat	Adakah penentu yang bukan kunci kandidat?

D. Langkah Percobaan

D.1 Dari UNF ke 1NF

1. Tulis relasi spreadsheet lama dalam bentuk UNF. Satu nota menyimpan grup berulang barang:

```
NOTA_UNF (no_nota_penjualan, tgl_penjualan, kode_petugas, nama_petugas,
no_anggota, nama_anggota,
        { kode_barang, nama_barang, qty_detail_penjualan,
harga_satuan_detail_penjualan, subtotal_detail_penjualan },
jumlah_penjualan, diskon_penjualan, total_penjualan)
```

2. Hilangkan grup berulang: satu baris untuk setiap barang di setiap nota. Kunci primer menjadi komposit (**no_nota_penjualan, kode_barang**).

```
NOTA_1NF (no_nota_penjualan, kode_barang, tgl_penjualan, kode_petugas,
nama_petugas, no_anggota,
        nama_anggota, nama_barang, qty_detail_penjualan,
harga_satuan_detail_penjualan, subtotal_detail_penjualan,
jumlah_penjualan, diskon_penjualan, total_penjualan)
```

D.2 Menuliskan Dependensi Fungsional

3. Turunkan dependensi fungsional dari aturan bisnis Pertemuan 2, bukan dari isi spreadsheet.

```
FD1: no_nota_penjualan -> tgl_penjualan, kode_petugas, no_anggota
(AB-01)
FD2: kode_petugas      -> nama_petugas
FD3: no_anggota       -> nama_anggota
(AB-05)
FD4: kode_barang      -> nama_barang
FD5: (no_nota_penjualan, kode_barang)
      -> qty_detail_penjualan,
harga_satuan_detail_penjualan (AB-04)
FD6: qty_detail_penjualan, harga_satuan_detail_penjualan
      -> subtotal_detail_penjualan
(turunan)
```

Titik Analisis: 1

Mengapa FD5 menempatkan **harga_satuan_detail_penjualan** pada kunci (**no_nota_penjualan, kode_barang**), bukan pada **kode_barang** saja? Apa yang terjadi pada laporan omzet tahun lalu jika **harga_satuan_detail_penjualan** dipindah ke tabel barang?

D.3 Menuju 2NF dan 3NF

4. **2NF.** FD1 dan FD4 hanya bergantung pada sebagian kunci komposit. Pisahkan menjadi relasi sendiri: penjualan (kunci **no_nota_penjualan**) dan barang (kunci **kode_barang**). Sisanya menjadi detail penjualan.
5. **3NF.** Di relasi penjualan terdapat rantai **no_nota_penjualan** → **kode_petugas** → **nama_petugas** dan **no_nota_penjualan** → **no_anggota** → **nama_anggota**. Pisahkan petugas dan anggota menjadi relasi sendiri.
6. **Nilai turunan.** **subtotal_detail_penjualan**, **jumlah_penjualan**, dan **total_penjualan** dapat dihitung dari detail dan status anggota. Buku ini memutuskan **tidak menyimpannya** di tabel operasional, sehingga tidak mungkin tidak konsisten.

```
PENJUALAN (no_nota_penjualan, tgl_penjualan, kode_petugas*,
no_anggota*, bayar_penjualan)
DETAIL_PENJUALAN (no_nota_penjualan*, kode_barang*,
qty_detail_penjualan, harga_satuan_detail_penjualan)
BARANG (kode_barang, nama_barang, kategori_barang, harga_jual_barang,
stok_barang, stok_min_barang)
PETUGAS (kode_petugas, nama_petugas, peran_petugas)
ANGGOTA (no_anggota, nim_anggota, nama_anggota, prodi_anggota,
no_hp_anggota, status_anggota, tgl_daftar_anggota)
-- garis bawah = kunci primer, * = kunci tamu
```

Titik Analisis: 2

Keputusan “total tidak disimpan” akan ditinjau ulang di Buku 2 Modul 2, ketika tabel **arsip** penjualan justru menyimpan total. Mengapa keputusan yang berbeda itu tetap konsisten? Petunjuk: bedakan data operasional yang masih dapat berubah dari potret historis yang tidak boleh berubah.

D.4 Mencocokkan dengan Skema Kanonik

7. Bandingkan hasil Anda dengan skema kanonik Kopma yang dipakai di seluruh buku (Tabel 4.2). Perbedaan utamanya adalah kunci pengganti (*surrogate key*) **id_*** yang dipakai sebagai kunci primer, sedangkan kode alami seperti **no_nota_penjualan** menjadi kunci unik.
8. Periksa BCNF: apakah ada penentu yang bukan kunci kandidat? Tuliskan hasil pemeriksaan untuk setiap tabel.
9. Simpan semua tahap di **p04_normalisasi_kopma_<NIM>.md**, lalu commit dengan pesan **p04: normalisasi kopma**.

Tabel 4.2 Skema kanonik Kopma hasil normalisasi (acuan seluruh buku)

Tabel	Kunci primer	Kunci unik	Kunci tamu
petugas	id_petugas	kode_petugas	–
anggota	id_anggota	no_anggota, nim_anggota	–

Tabel	Kunci primer	Kunci unik	Kunci tamu
barang	id_barang	kode_barang	–
pemasok	id_pemasok	–	–
penjualan	id_penjualan	no_nota_penjualan	id_petugas, id_anggota (boleh NULL)
detail_penjualan	(id_penjualan, id_barang)	–	id_penjualan, id_barang
pembelian	id_pembelian	–	id_pemasok
detail_pembelian	(id_pembelian, id_barang)	–	id_pembelian, id_barang

Titik Analisis: 3

Kolom `pemasok_brg` di spreadsheet lama tidak muncul di tabel barang. Di mana informasi “barang ini dipasok oleh siapa” kini disimpan? Apa keuntungannya dibanding menyimpan satu nama pemasok di tabel barang?

E. Latihan dan Modifikasi

1. Kopma ingin mencatat beberapa nomor telepon untuk setiap pemasok. Normalisasikan kebutuhan ini dan tuliskan relasi barunya.
2. Tunjukkan satu contoh data yang melanggar 3NF jika `nama_anggota` disimpan di tabel penjualan, lalu tulis kueri ubah yang memperlihatkan anomali ubahnya.

F. Tugas Mandiri: Milestone Proyek 4

- Ambil satu dokumen sumber proyek (dari Pertemuan 2) sebagai titik awal UNF.
- Tulis dependensi fungsional beserta kode AB yang mendasarinya.
- Tunjukkan tahapan 1NF, 2NF, 3NF, dan hasil pemeriksaan BCNF.
- Putuskan penyimpanan setiap nilai turunan beserta alasannya.
- Perbarui ERD proyek bila berubah, dan catat perubahannya di `p04_normalisasi_<NIM>.md`.

Catatan: hubungan ke pertemuan berikutnya

Skema 3NF proyek menjadi dasar skrip DDL pada Pertemuan 5. Setiap kunci dan aturan pada dokumen ini harus muncul sebagai constraint.

Troubleshooting Umum

Masalah 1: Dependensi disimpulkan dari contoh data

Gejala. Karena semua contoh data harga pulpen Rp4.000, mahasiswa menulis `kode_barang` → `harga_satuan_detail_penjualan`. **Perbaikan:** kembali ke aturan bisnis. Contoh data hanya dapat membantah dependensi, tidak dapat membuktikannya.

Masalah 2: Tabel terpecah berlebihan

Gejala. Setiap atribut dijadikan tabel sendiri. **Cara mendeteksi:** gabungkan kembali tabel-tabel itu dengan kuncinya. Jika tidak ada anomali yang hilang karena pemisahan, pemisahan itu tidak perlu. Normalisasi bertujuan menghilangkan anomali, bukan memperbanyak tabel.

Masalah 3: Kunci pengganti dianggap menyelesaikan normalisasi

Gejala. Setiap tabel diberi `id` otomatis lalu dianggap sudah 3NF. **Perbaikan:** kunci pengganti tidak menghapus dependensi fungsional. Periksa tetap dengan kunci alami, misalnya `nim_anggota` di anggota.

Checklist Laporan Pertemuan 4

Laporan disimpan sebagai `laporan/p04_laporan_<NIM>.md` dengan struktur baku, lalu di-push sebelum batas waktu. Pastikan semua **checklist umum** (lihat Pedoman dan Checklist Laporan Praktikum) dan checklist khusus berikut terpenuhi.

Tabel 4.3 Checklist khusus laporan Pertemuan 4

	Jenis	Yang harus ada
☐	Berkas wajib	<code>p04_normalisasi_<NIM>.md</code> (proyek)
☐	Bukti tangkapan layar	Tabel tahapan UNF → 1NF → 2NF → 3NF
☐	Analisis wajib	Daftar dependensi fungsional; keputusan menyimpan atau menghitung nilai turunan beserta alasannya

Perhatian: aturan keaslian

Dokumen sumber, dependensi fungsional, dan keputusan nilai turunan proyek harus berasal dari aturan bisnis Anda sendiri. Salinan yang sama dengan mahasiswa lain, termasuk urutan FD dan kalimatnya, akan diperiksa melalui viva.

Rubrik Penilaian Modul

Tabel 4.4 Rubrik penilaian Modul 4

Aspek	Bobot	Indikator
Pelaksanaan langkah D	25%	Tahapan UNF sampai 3NF Kopma benar dan cocok dengan skema kanonik
Titik Analisis dan latihan E	20%	Alasan FD dan nilai turunan tepat
Milestone Proyek 4	40%	FD berbasis AB, tahapan normalisasi benar, BCNF diperiksa
Laporan dan bukti Git	15%	Lengkap sesuai checklist

Sinkronisasi Kompetensi

Tabel 4.5 Unit kompetensi yang dilatih di Modul 4

Kode unit	Judul unit	Elemen yang dilatih
J.62DMS00.006.1	Merancang Basis Data	Elemen 2: atribut diidentifikasi, normalisasi dilakukan, dan hasilnya didokumentasikan (KUK 2.1, 2.2, 2.4)

Modul 5: Membangun Struktur dengan DDL dan Constraint

Cerita Pertemuan Ini

Rancangan Kopma sudah lolos uji normalisasi. Sekarang saatnya membangun “lemari berlaci” yang sesungguhnya di MariaDB. Pengurus mengajukan satu permintaan penting: “Jangan sampai stok minus lagi, dan jangan sampai ada nota tanpa kasir.” Hari ini aturan bisnis dari Pertemuan 2 berubah dari kalimat di dokumen menjadi aturan yang dijaga langsung oleh server.

A. Tujuan Praktikum

Setelah menyelesaikan modul ini, mahasiswa mampu:

1. menerjemahkan skema relasional 3NF menjadi perintah **CREATE TABLE** dengan tipe data yang tepat;
2. menerapkan constraint **PRIMARY KEY**, **FOREIGN KEY**, **UNIQUE**, **NOT NULL**, **DEFAULT**, dan **CHECK** sesuai aturan bisnis;
3. mengubah dan menghapus struktur dengan **ALTER TABLE** dan **DROP TABLE** dalam urutan yang benar;
4. membuktikan bahwa constraint menolak data yang melanggar aturan, lalu membandingkan desain fisik dengan ERD.

B. Alat dan Bahan

- XAMPP 8.2.12 (MariaDB 10.4.32), akun `mhs_<3d>`, basis data `kopma_<3d>` dari Modul 1.
- Dokumen normalisasi dan ERD dari Pertemuan 3–4.
- phpMyAdmin Designer untuk membandingkan desain fisik dengan ERD.
- Editor teks untuk menulis berkas `.sql`.

C. Dasar Teori

C.1 DDL dan Tipe Data

Data Definition Language (DDL) adalah bagian SQL untuk membuat, mengubah, dan menghapus struktur basis data [12]. Pemilihan tipe data adalah keputusan desain: tipe yang terlalu besar memboroskan ruang, sedangkan tipe yang terlalu kecil menolak data yang sah [13].

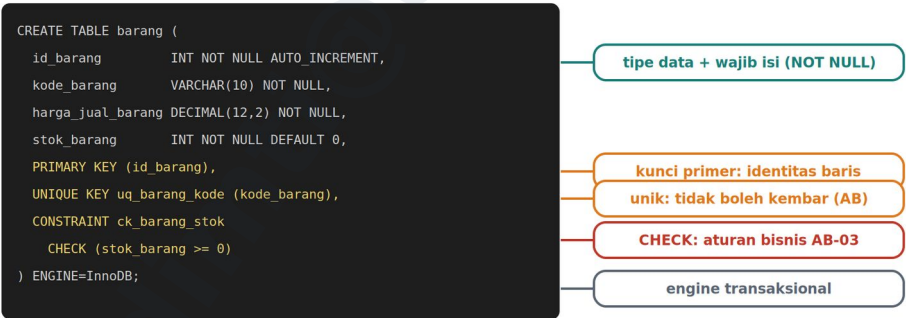
Tabel 5.1 Pemetaan kamus data Kopma ke tipe data MariaDB

Elemen	Tipe	Alasan
nim_anggota	CHAR(10)	Panjang selalu 10; berupa kode, bukan angka untuk dihitung
no_nota_penjualan	CHAR(12)	Format tetap PJ - 2609 - 0142
harga_jual_barang, harga_satuan_detail_penjualan	DECIMAL(12,2)	Uang wajib tepat; FLOAT menimbulkan galat pembulatan
stok_barang, qty_detail_penjualan	INT, SMALLINT	Bilangan bulat; qty per baris tidak mungkin puluhan ribu
tgl_penjualan	DATETIME	Perlu tanggal dan jam (KI-01 per hari)
status_anggota, peran_petugas	ENUM(...)	Nilai terbatas dan diketahui

C.2 Constraint sebagai Aturan Bisnis

Constraint memindahkan aturan bisnis dari “disiplin pengguna” ke “jaminan server” [11]. Gambar 5.1 menunjukkan bagaimana satu perintah **CREATE TABLE** memuat tipe data, kunci, dan aturan sekaligus. Sejak MariaDB 10.2, constraint **CHECK** benar-benar ditegakkan, sehingga AB-03 (stok tidak boleh negatif) dapat dijaga server [13].

Anatomi **CREATE TABLE**: dari kamus data ke aturan yang dijaga server



Gambar 5.1 Anatomi **CREATE TABLE** `barang`

C.3 Urutan Membuat dan Menghapus Tabel

Tabel induk harus dibuat sebelum tabel anak yang merujuknya, dan dihapus setelahnya. Untuk Kopma, urutan pembuatannya adalah: petugas, anggota, barang, pemasok, penjualan, detail_penjualan, pembelian, detail_pembelian. Urutan penghapusannya kebalikannya.

Catatan: kebijakan charset tabel (sinkron dengan Buku 2)

Tabel **tidak** diberi klausa **CHARACTER SET** sendiri, sehingga mewarisi collation basis data **utf8mb4_unicode_ci** dari Modul 1. Kebijakan ini mencegah campuran collation ketika basis data dipindah ke MariaDB 12.3 di Buku 2.

D. Langkah Percobaan

D.1 Membuat Tabel Induk

1. Buat berkas **p05_ddl_kopma_<NIM>.sql**. Awali dengan **USE kopma_<3d>;**, lalu tulis tabel tanpa kunci tamu terlebih dahulu.

Cuplikan p05_ddl_kopma_<NIM>.sql

```
CREATE TABLE petugas (
  id_petugas      SMALLINT          NOT NULL AUTO_INCREMENT,
  kode_petugas    CHAR(3)           NOT NULL,
  nama_petugas    VARCHAR(100)      NOT NULL,
  peran_petugas   ENUM('kasir','gudang','ketua') NOT NULL,
  PRIMARY KEY (id_petugas),
  UNIQUE KEY uq_petugas_kode (kode_petugas)
) ENGINE=InnoDB;

CREATE TABLE barang (
  id_barang      INT                NOT NULL
  AUTO_INCREMENT,
  kode_barang     VARCHAR(10)       NOT NULL,
  nama_barang     VARCHAR(100)      NOT NULL,
  kategori_barang ENUM('ATK','makanan','minuman') NOT NULL,
  harga_jual_barang DECIMAL(12,2)  NOT NULL,
  stok_barang     INT               NOT NULL DEFAULT 0,
  stok_min_barang INT               NOT NULL DEFAULT 5,
  PRIMARY KEY (id_barang),
  UNIQUE KEY uq_barang_kode (kode_barang),
  CONSTRAINT ck_barang_stok CHECK (stok_barang >= 0),
  CONSTRAINT ck_barang_harga CHECK (harga_jual_barang >= 0)
) ENGINE=InnoDB;
```

2. Lanjutkan tabel **anggota** dan **pemasok** sesuai skema kanonik (Modul 4). **nim_anggota** dan **no_anggota** masing-masing diberi **UNIQUE** untuk AB-05.

D.2 Membuat Tabel dengan Kunci Tamu

3. Buat tabel transaksi. Perhatikan: kolom kunci tamu harus bertipe **sama persis** dengan kolom yang dirujuk.

```
CREATE TABLE penjualan (
  id_penjualan     INT                NOT NULL AUTO_INCREMENT,
  no_nota_penjualan CHAR(12)          NOT NULL,
  tgl_penjualan    DATETIME           NOT NULL,
  id_petugas       SMALLINT           NOT NULL,
  id_anggota       INT                NULL,
  bayar_penjualan  DECIMAL(12,2)      NOT NULL,
  PRIMARY KEY (id_penjualan),
  UNIQUE KEY uq_penjualan_nota (no_nota_penjualan),
```

```

CONSTRAINT fk_penjualan_petugas FOREIGN KEY (id_petugas) REFERENCES
petugas (id_petugas),
CONSTRAINT fk_penjualan_anggota FOREIGN KEY (id_anggota) REFERENCES
anggota (id_anggota)
) ENGINE=InnoDB;

CREATE TABLE detail_penjualan (
    id_penjualan          INT          NOT NULL,
    id_barang             INT          NOT NULL,
    qty_detail_penjualan  SMALLINT     NOT NULL,
    harga_satuan_detail_penjualan DECIMAL(12,2) NOT NULL,
    PRIMARY KEY (id_penjualan, id_barang),
    CONSTRAINT fk_detail_penjualan FOREIGN KEY (id_penjualan)
        REFERENCES penjualan (id_penjualan) ON DELETE CASCADE,
    CONSTRAINT fk_detail_barang FOREIGN KEY (id_barang) REFERENCES barang
(id_barang),
    CONSTRAINT ck_detail_qty CHECK (qty_detail_penjualan > 0)
) ENGINE=InnoDB;

```

4. Selesaikan **pembelian** dan **detail_pembelian** dengan pola yang sama, lalu periksa hasilnya dengan **SHOW TABLES;** dan **SHOW CREATE TABLE penjualan;**.

Titik Analisis: 1

Mengapa **fk_detail_penjualan** memakai **ON DELETE CASCADE**, sedangkan **fk_detail_barang** tidak? Apa yang terjadi pada laporan penjualan lama jika barang boleh dihapus beserta seluruh detailnya?

D.3 Membuktikan Constraint Bekerja

5. Isi data induk secukupnya (dua petugas, dua barang), lalu coba masukkan data yang **melanggar** aturan. Semua perintah berikut harus ditolak (Gambar 5.2).

```

INSERT INTO penjualan (no_nota_penjualan, tgl_penjualan, id_petugas,
id_anggota, bayar_penjualan)
VALUES ('PJ-2609-0199', NOW(), 99, NULL, 5000);    -- petugas 99 tidak
ada
UPDATE barang SET stok_barang = -1 WHERE id_barang = 1;    -- melanggar
AB-03
INSERT INTO barang (kode_barang, nama_barang, kategori_barang,
harga_jual_barang)
VALUES ('ATK-001', 'Duplikat', 'ATK', 1000);        -- kode kembar

```

Shell XAMPP - constraint menolak data yang salah

```

MariaDB [kopma_123]> INSERT INTO penjualan (no_nota_penjualan, tgl_penjualan,
-> id_petugas, id_anggota, bayar_penjualan)
-> VALUES ('PJ-2609-0199', NOW(), 99, NULL, 5000);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint
fails ('kopma_123`.`penjualan`, CONSTRAINT `fk_penjualan_petugas` FOREIGN KEY
(`id_petugas`) REFERENCES `petugas` (`id_petugas`))

MariaDB [kopma_123]> UPDATE barang SET stok_barang = -1 WHERE id_barang = 1;
ERROR 4025 (23000): CONSTRAINT `ck_barang_stok` failed for `kopma_123`.`barang`

```

Ilustrasi disederhanakan; tampilan sebenarnya dapat sedikit berbeda.

Gambar 5.2 Constraint menolak kunci tamu tak dikenal dan stok negatif

Titik Analisis: 2

Kode galat 1452, 4025, dan 1062 muncul pada tiga percobaan di atas. Petakan setiap kode ke aturan bisnis (AB-xx) yang dilindunginya. Aturan bisnis Kopma mana yang **belum** dapat dijaga oleh constraint apa pun?

D.4 Mengubah Struktur dan Membandingkan dengan ERD

6. Ketua meminta pencatatan tanggal kedaluwarsa untuk barang makanan. Tambahkan kolom tanpa membuat ulang tabel:

```
ALTER TABLE barang ADD COLUMN tgl_kedaluwarsa DATE NULL AFTER
stok_min_barang;
ALTER TABLE barang DROP COLUMN tgl_kedaluwarsa; -- dibatalkan setelah
dicoba
```

7. Lengkapi ERD konseptual Pertemuan 3 menjadi **ERD fisik**: setiap atribut diberi tipe data dan constraint persis seperti skrip DDL. Hasil untuk Kopma adalah ERD pada Gambar 3.2 (Modul 3), yang kini dapat Anda cocokkan kolom demi kolom dengan skrip DDL. ERD fisik ini menjadi dokumentasi desain (KUK 2.4 dan 3.6) sekaligus acuan pemeriksaan skrip.

Tabel 5.2 Standar kelengkapan ERD fisik

Unsur	Ketentuan
Tipe data	Sama persis dengan DDL, termasuk panjang dan presisi (DECIMAL (12, 2))
Constraint	AI, UNIQUE, NOT NULL/NULL, CHECK, DEFAULT , dan nilai ENUM dicantumkan
Kunci tamu	Ditulis dengan tabel tujuannya (– petugas) dan aksi ON DELETE bila bukan bawaan
Data pribadi	Kolom data pribadi diberi tanda khusus sesuai kamus data
Nilai turunan	Tidak digambar sebagai kolom kecuali keputusan Modul 4 menyatakan disimpan

8. Buka phpMyAdmin, pilih **kopma_<3d>**, lalu buka menu **Designer**. Susun tabel agar mirip ERD Pertemuan 3, lalu catat setiap perbedaan antara desain fisik dan ERD.
9. Pastikan skrip dapat dijalankan ulang dari basis data kosong: tambahkan **DROP TABLE IF EXISTS** untuk semua tabel dengan urutan terbalik di awal skrip. Commit dengan pesan **p05: ddl kopma**.

Titik Analisis: 3

Mengapa **DROP TABLE IF EXISTS** harus dimulai dari **detail_pembelian** dan **detail_penjualan**, bukan dari **petugas**? Apa pesan galat yang muncul jika urutannya dibalik?

D.5 Membaca ERD Fisik: Fungsi Setiap Tabel

Sebelum mengisi data di Pertemuan 6, Anda harus memahami **untuk apa** setiap tabel ada, bukan hanya kolomnya. Delapan tabel Kopma terbagi menjadi tiga jenis. **Tabel master** menyimpan “siapa” dan “apa” yang relatif jarang berubah. **Tabel kepala transaksi** mencatat satu kejadian bisnis, misalnya satu nota. **Tabel detail transaksi** mencatat rincian kejadian itu baris demi baris.

Tabel 5.3 Pengelompokan tabel Kopma

Jenis	Tabel	Satu baris mewakili	Penanggung jawab data
Master	petugas	satu orang yang bekerja di Kopma	Ketua
Master	anggota	satu mahasiswa anggota koperasi	Ketua
Master	barang	satu jenis barang yang dijual	Petugas gudang
Master	pemasok	satu pihak penyuplai barang	Petugas gudang
Kepala transaksi	penjualan	satu nota penjualan	Kasir
Detail transaksi	detail_penjualan	satu jenis barang di dalam satu nota	Kasir
Kepala transaksi	pembelian	satu faktur pembelian dari pemasok	Petugas gudang
Detail transaksi	detail_pembelian	satu jenis barang di dalam satu faktur	Petugas gudang

1. Tabel petugas. Tabel ini mencatat setiap orang yang bekerja di Kopma: kasir, petugas gudang, dan ketua. Kolom **kode_petugas** berisi kode singkat tiga karakter (misalnya **K03**) yang tercetak di nota, sehingga pembeli tahu siapa yang melayaninya. Kolom **nama_petugas** menyimpan nama lengkap, sedangkan **peran_petugas** bertipe **ENUM** agar isinya hanya salah satu dari **kasir**, **gudang**, atau **ketua**, tidak bisa ditulis sembarangan. Tabel ini menjadi induk bagi **penjualan** melalui kolom **id_petugas**: satu petugas dapat melayani banyak nota, tetapi setiap nota wajib punya tepat satu petugas.

Karena catatan penjualan merujuk ke petugas, seorang petugas yang sudah pernah melayani transaksi **tidak boleh dihapus** (server menolaknya dengan **ERROR 1451**). Jika ia berhenti bekerja, akunnya dikunci dan perannya dicabut (Pertemuan 13), tetapi barisnya tetap ada sebagai jejak sejarah.

2. Tabel anggota. Tabel ini menyimpan data mahasiswa yang terdaftar sebagai anggota koperasi. Kolom **no_anggota** adalah nomor kartu (misalnya **A-0457**), sedangkan **nim_anggota** bersifat unik (AB-05) sehingga kasir tetap dapat menemukan anggota

yang lupa membawa kartu. Kolom **status_anggota** menentukan hak diskon 5%: hanya anggota berstatus **aktif** yang memperolehnya (AB-02). Kolom **tgl_daftar_anggota** dipakai untuk menghitung lama keanggotaan, dan **prodi_anggota** untuk laporan per program studi.

Kolom **no_hp_anggota** adalah **data pribadi**. Kolom ini boleh kosong, hanya boleh dilihat oleh ketua, dan disembunyikan dari kasir melalui view pada Pertemuan 13, sejalan dengan prinsip Undang-Undang Pelindungan Data Pribadi [17]. Relasi anggota ke penjualan bersifat opsional (0..1): nota pembeli umum mengisi **id_anggota** dengan **NULL**.

3. Tabel barang. Tabel ini adalah katalog sekaligus catatan persediaan. Setiap baris menyimpan satu jenis barang dengan kode unik **kode_barang** (misalnya **ATK-001**) dan kelompoknya di **kategori_barang**. Kolom **harga_jual_barang** berisi **harga yang berlaku saat ini**, bukan harga yang pernah dipakai di nota lama. Kolom **stok_barang** menyimpan jumlah tersedia dan dijaga **CHECK** agar tidak pernah negatif (AB-03), sedangkan **stok_min_barang** adalah batas yang memicu pemesanan ulang (AB-06) dan menjadi dasar laporan KI-03.

Tabel **barang** adalah satu-satunya tabel yang **diubah oleh dua arus berlawanan**: penjualan mengurangi stok, penerimaan barang dari pemasok menambahnya. Karena itu, setiap perubahan stok harus dilakukan di dalam transaksi yang sama dengan pencatatan nota atau faktur (Pertemuan 14), supaya stok tidak pernah berbeda dari catatan transaksinya.

4. Tabel pemasok. Tabel ini mencatat pihak yang memasok barang ke Kopma, lengkap dengan **nama_pemasok**, **telepon_pemasok**, dan **alamat_pemasok**. Satu pemasok dapat menerima banyak pesanan pembelian. Tabel ini sengaja **tidak** dihubungkan langsung ke barang; informasi “barang ini dipasok oleh siapa” diperoleh dari riwayat pembelian, sehingga Kopma tetap dapat berganti pemasok tanpa mengubah data barang (lihat Titik Analisis 3 Pertemuan 4).

5. Tabel penjualan. Tabel ini adalah **kepala nota**: satu baris untuk satu kali transaksi di kasir. Kolom **no_nota_penjualan** berformat **PJ-YYMM-nnnn** dan unik (AB-01). Kolom **tgl_penjualan** bertipe **DATETIME** karena laporan harian dan jam sibuk membutuhkan tanggal sekaligus jam (KI-01). Kolom **id_petugas** wajib diisi, **id_anggota** boleh kosong, dan **bayar_penjualan** mencatat uang yang diterima dari pembeli.

Perhatikan bahwa tabel ini **tidak menyimpan total belanja**. Total dihitung dari tabel detail, sesuai keputusan normalisasi di Pertemuan 4, sehingga total tidak mungkin berbeda dari rinciannya. Kembalian pun dihitung sebagai **bayar_penjualan** dikurangi total. Tabel ini menjadi induk **detail_penjualan** dengan aturan **ON DELETE CASCADE**: jika nota dibatalkan, rinciannya ikut terhapus.

6. Tabel detail_penjualan. Tabel ini mencatat **setiap barang di dalam satu nota** dan menjadi entitas asosiatif yang memecah relasi banyak-ke-banyak antara penjualan dan barang. Kunci primernya komposit, (**id_penjualan, id_barang**), sehingga satu jenis barang hanya muncul sekali per nota; pembelian tiga pulpen dicatat sebagai satu baris dengan **qty_detail_penjualan** bernilai 3, yang dijaga **CHECK** agar lebih dari nol.

Kolom terpenting di tabel ini adalah **harga_satuan_detail_penjualan**, yaitu **salinan harga pada saat transaksi** (AB-04). Ketika harga pulpen naik dari Rp4.000 menjadi Rp4.500 pada 15 Oktober 2026, nota-nota sebelumnya tetap menunjukkan Rp4.000 karena harganya tersimpan di sini, bukan dibaca dari tabel barang. Hampir semua laporan penjualan (omzet, barang terlaris, belanja anggota) bersumber dari tabel ini.

7. Tabel pembelian. Tabel ini adalah **kepala faktur pembelian** dari pemasok, pasangan dari tabel penjualan di sisi persediaan. Kolom **no_faktur_pembelian** menyimpan nomor faktur dari pemasok; kolom ini tidak dibuat unik karena dua pemasok berbeda bisa saja memakai nomor faktur yang sama. Kolom **tgl_pembelian** cukup bertipe **DATE**, dan **status_pembelian** mencatat tahap alur pengadaan: **dipesan** ketika petugas gudang memesan karena stok di bawah minimum (AB-06), lalu **diterima** ketika barang datang dan stok bertambah.

8. Tabel detail_pembelian. Tabel ini mencatat **setiap barang di dalam satu faktur pembelian**, dengan pola yang sama seperti **detail_penjualan**. Kolom **qty_detail_pembelian** menyimpan jumlah yang dibeli, dan **harga_beli_detail_pembelian** menyimpan harga modal per unit. Harga modal inilah yang kelak memungkinkan Kopma menghitung laba per barang, yaitu selisih harga jual saat transaksi dengan harga beli, yang akan dimanfaatkan di data warehouse pada Buku 2.

Perbedaan tabel penjualan dan pembelian. Kedua tabel sama-sama tabel kepala transaksi dengan pola kepala-detail yang serupa, sehingga mahasiswa sering menganggapnya kembar. Padahal keduanya mencatat arus yang **berlawanan**: penjualan mencatat barang **keluar** dari Kopma ke pembeli, sedangkan pembelian mencatat barang **masuk** dari pemasok ke Kopma.

Tabel 5.4 Perbedaan tabel penjualan dan pembelian

Aspek	penjualan	pembelian
Arus barang	Keluar ke pembeli	Masuk dari pemasok
Arus uang	Kopma menerima uang	Kopma mengeluarkan uang
Pihak lawan	Anggota atau pembeli umum (id_anggota , boleh NULL)	Pemasok (id_pemasok , wajib)
Pencatat	Kasir (id_petugas)	Petugas gudang
Pengaruh ke stok	Mengurangi stok_barang	Menambah stok_barang

Aspek	penjualan	pembelian
Kapan stok berubah	Langsung saat nota dibuat	Saat status berubah menjadi diterima
Status	Tidak ada; transaksi selesai seketika	status_pembelian : dipesan – diterima
Nomor dokumen	no_nota_penjualan dibuat Kopma, unik	no_faktur_pembelian dari pemasok, tidak unik
Waktu	DATETIME (jam dibutuhkan untuk laporan harian)	DATE (tanggal saja cukup)
Pembayaran	bayar_penjualan mencatat uang dari pembeli	Tidak ada kolom bayar
Harga di tabel detail	harga_satuan_detail_penjualan = harga jual saat transaksi	harga_beli_detail_pembelian = harga modal

Dari tabel di atas, ada tiga perbedaan yang paling penting dipahami:

1. **Arah stok.** Setiap nota penjualan mengurangi stok, sedangkan setiap faktur pembelian yang sudah diterima menambah stok. Tabel **barang** berada di tengah kedua arus ini, sehingga perubahan stok harus selalu dilakukan dalam satu transaksi bersama dokumennya (Pertemuan 14).
2. **Waktu terjadinya.** Penjualan terjadi seketika di kasir. Pembelian berlangsung bertahap: dipesan ketika stok di bawah minimum, lalu diterima beberapa hari kemudian. Karena itulah hanya tabel pembelian yang memerlukan kolom status.
3. **Makna harga.** Harga di detail penjualan adalah pendapatan, sedangkan harga di detail pembelian adalah modal. Selisih keduanya adalah laba per barang, yang akan dihitung di data warehouse pada Buku 2.

Catatan: alur data dalam satu hari di Kopma

Saat pembeli membayar: satu baris masuk ke **penjualan**, beberapa baris masuk ke **detail_penjualan**, lalu **stok_barang** di tabel **barang** dikurangi, semuanya dalam satu transaksi.

Saat stok menipis: petugas gudang membuat satu baris **pembelian** berstatus **dipesan** beserta rinciannya di **detail_pembelian**.

Saat barang datang: status pembelian diubah menjadi **diterima** dan **stok_barang** ditambah, juga dalam satu transaksi.

Saat laporan dibuat: data dibaca dari tabel transaksi dan digabungkan dengan tabel master untuk mendapatkan nama petugas, anggota, dan barang.

E. Latihan dan Modifikasi

1. Tambahkan constraint **CHECK** yang memastikan **stok_min_barang** tidak negatif, lalu buktikan dengan satu perintah yang ditolak.
2. Ubah **fk_penjualan_anggota** agar anggota yang dihapus membuat **id_anggota** pada penjualan menjadi **NULL** (**ON DELETE SET NULL**). Diskusikan apakah perubahan ini sesuai AB-02 dan UU PDP.

F. Tugas Mandiri: Milestone Proyek 5

- Tulis **p05_ddl_<NIM>.sql** untuk seluruh tabel proyek, dapat dijalankan ulang dari basis data kosong.
- Setiap tabel memiliki kunci primer; setiap relasi ERD menjadi kunci tamu; setiap aturan bisnis yang dapat dijaga struktur menjadi constraint, dengan komentar kode AB-xx.
- Aturan batas **P + 2** dari Modul 2 dianalisis: jika menyangkut satu baris (misalnya qty per item), wujudkan dengan **CHECK**; jika menyangkut jumlah baris per transaksi, catat bahwa **CHECK** tidak dapat menjaganya karena melibatkan banyak baris, lalu tandai untuk diselesaikan di Modul 14.
- Gambar ERD fisik proyek (**p05_erd_fisik_<NIM>.png** dan sumbernya) yang memenuhi standar kelengkapan ERD fisik.
- Dokumentasikan tabel pemetaan tipe data dan hasil perbandingan Designer dengan ERD.

Catatan: hubungan ke pertemuan berikutnya

Pertemuan 6 mengisi tabel ini dengan data sampel dan skenario uji. Constraint yang tidak diuji belum dianggap bekerja.

Troubleshooting Umum

Masalah 1: ERROR 1005 errno 150 “Foreign key constraint is incorrectly formed”

Cara membaca. Tabel anak gagal dibuat karena kunci tamu tidak cocok dengan kolom induknya. Penyebab tersering: tipe berbeda (misalnya **INT** merujuk **SMALLINT**), tabel induk belum ada, atau kolom induk bukan kunci. **Perbaikan:** samakan tipe dan tanda (**UNSIGNED**) persis, lalu buat tabel induk lebih dulu.

Masalah 2: ERROR 1451 saat menghapus data induk

Cara membaca. Pesan **Cannot delete or update a parent row** menyebut constraint yang menahan penghapusan. Server sedang menjaga integritas referensial, bukan rusak. **Perbaikan:** hapus atau pindahkan data anak dulu, atau tinjau ulang aksi **ON DELETE** jika aturan bisnis memang mengizinkannya.

Masalah 3: ERROR 1050 Table already exists

Perbaikan. Skrip dijalankan dua kali. Tambahkan blok **DROP TABLE IF EXISTS** di awal skrip dengan urutan anak ke induk, sesuai langkah D.4.

Checklist Laporan Pertemuan 5

Laporan disimpan sebagai `laporan/p05_laporan_<NIM>.md` dengan struktur baku, lalu di-push sebelum batas waktu. Pastikan semua **checklist umum** (lihat Pedoman dan Checklist Laporan Praktikum) dan checklist khusus berikut terpenuhi.

Tabel 5.5 Checklist khusus laporan Pertemuan 5

	Jenis	Yang harus ada
☐	Berkas wajib	<code>p05_ddl_<NIM>.sql</code> (dapat dijalankan ulang dari basis data kosong) dan ERD fisik <code>p05_erd_fisik_<NIM></code>
☐	Bukti tangkapan layar	SHOW CREATE TABLE tabel utama; galat saat constraint dilanggar (FK, UNIQUE, CHECK)
☐	Analisis wajib	Alasan pemilihan tipe data dan constraint; kaitan constraint dengan AB-xx

Perhatian: aturan keaslian

Nama constraint wajib mengikuti pola `pk_`, `fk_`, `uq_`, `ck_` ditambah nama tabel. Komentar AB-xx di setiap constraint harus merujuk dokumen kebutuhan data Anda sendiri.

Rubrik Penilaian Modul

Tabel 5.6 Rubrik penilaian Modul 5

Aspek	Bobot	Indikator
Pelaksanaan langkah D	25%	Skema Kopma lengkap, constraint terbukti menolak data salah
Titik Analisis dan latihan E	20%	Pemetaan galat ke AB tepat; aksi ON DELETE beralasan
Milestone Proyek 5	40%	DDL proyek dapat diulang, constraint sesuai AB, CHECK berbasis P
Laporan dan bukti Git	15%	Lengkap sesuai checklist

Sinkronisasi Kompetensi

Tabel 5.7 Unit kompetensi yang dilatih di Modul 5

Kode unit	Judul unit	Elemen yang dilatih
J.62DMS00.006.1	Merancang Basis Data	Elemen 3: kunci primer dan tamu, integritas referensial, aturan validasi, dokumentasi desain (KUK 3.1, 3.3, 3.4, 3.6); KUK 4.4 desain fisik dibandingkan dengan model konseptual
J.62DMS00.010.1	Membuat Basis Data	Elemen 1: membuat basis data

Modul 6: Mengisi, Mengubah, dan Menguji Data dengan DML

Cerita Pertemuan Ini

Lemari berlaci Kopma sudah berdiri, tetapi masih kosong. Kasir Rina ingin segera mencoba: memasukkan barang, mencatat nota, lalu dengan iseng mengetik stok “-5”. “Kalau yang ini diterima, berarti sistemnya belum bisa dipercaya,” katanya. Hari ini Anda mengisi basis data sekaligus membuktikan, dengan skenario uji tertulis, bahwa data yang salah memang ditolak.

A. Tujuan Praktikum

Setelah menyelesaikan modul ini, mahasiswa mampu:

1. menambah data dengan **INSERT** satu baris, banyak baris, dan **INSERT ... SELECT**;
2. mengubah dan menghapus data dengan **UPDATE** dan **DELETE** secara aman memakai mode **sql_safe_updates**;
3. menyusun skenario uji positif dan negatif serta mencatat hasilnya;
4. menyiapkan data sampel proyek yang dapat dibangun ulang dari skrip.

B. Alat dan Bahan

- XAMPP 8.2.12, basis data **kopma_<3d>** hasil Modul 5 (skrip DDL dapat dijalankan ulang).
- Parameter P dari Modul 2.
- Editor teks, repositori **basisdata-<NIM>**.

C. Dasar Teori

C.1 DML dan Urutan Pengisian

Data Manipulation Language (DML) terdiri atas **INSERT**, **UPDATE**, **DELETE**, dan **SELECT** [12]. Karena ada kunci tamu, urutan pengisian mengikuti urutan pembuatan tabel: induk dahulu, anak kemudian. Urutan penghapusan kebalikannya, kecuali relasi yang memakai **ON DELETE CASCADE**.

C.2 Bahaya UPDATE dan DELETE Tanpa WHERE

UPDATE atau **DELETE** tanpa **WHERE** mengenai **semua** baris. Kesalahan ini sering terjadi dan tidak memunculkan galat. Variabel **sql_safe_updates** membuat server menolak

perintah seperti itu kecuali **WHERE** memakai kolom kunci atau disertai **LIMIT** [13], seperti tampak pada Gambar 6.1.

```

Shell XAMPP - mode aman mencegah UPDATE tanpa syarat
MariaDB [kopma 123]> SET SESSION sql_safe_updates = 1;
MariaDB [kopma 123]> UPDATE barang SET harga_jual_barang = harga_jual_barang * 1.1;
ERROR 1175 (HY000): You are using safe update mode and you tried to update a table
without a WHERE that uses a KEY column

MariaDB [kopma 123]> UPDATE barang SET harga_jual_barang = harga_jual_barang * 1.1
-> WHERE kategori_barang = 'ATK' AND id_barang > 0;
Query OK, 2 rows affected (0.004 sec)
Rows matched: 2  Changed: 2  Warnings: 0
  
```

Ilustrasi disederhanakan; tampilan sebenarnya dapat sedikit berbeda.

Gambar 6.1 Mode aman menolak UPDATE tanpa syarat kolom kunci

C.3 Menguji Basis Data

Unit *Membuat Basis Data* tidak berhenti pada membuat tabel, tetapi mewajibkan **pengujian** [1]. Pengujian yang baik mencakup **uji positif** (data benar harus diterima) dan **uji negatif** (data salah harus ditolak dengan galat yang tepat). Setiap uji dicatat dalam format tetap (Gambar 6.2) agar dapat diulang oleh orang lain.



Gambar 6.2 Format skenario uji positif dan negatif

D. Langkah Percobaan

D.1 Mengisi Data Induk

1. Buat berkas **p06_data_kopma_<NIM>.sql**. Isi data induk dengan **INSERT** banyak baris sekaligus.

```

INSERT INTO petugas (kode_petugas, nama_petugas, peran_petugas) VALUES
  ('K01', 'Dimas', 'kasir'), ('K02', 'Sari', 'kasir'),
  ('K03', 'Rina', 'kasir'),
  ('G01', 'Joko', 'gudang'), ('P01', 'Ketua Kopma', 'ketua');
  
```

```

INSERT INTO barang (kode_barang, nama_barang, kategori_barang,
  harga_jual_barang, stok_barang, stok_min_barang) VALUES
  ('ATK-001', 'Pulpen Gel 0.5', 'ATK', 4000, 120, 20),
  ('ATK-002', 'Buku Tulis 58', 'ATK', 6500, 80, 20),
  ('MMN-001', 'Air Mineral 600', 'minuman', 4000, 48, 24),
  ('MKN-001', 'Roti Cokelat', 'makanan', 5000, 6, 10);
  
```

2. Isi **anggota** (minimal tiga, satu berstatus **nonaktif**) dan **pemasok** (minimal dua).

D.2 Mencatat Nota Penjualan

3. Catat nota contoh dari Modul 2 (PJ-2609-0142). Harga satuan disalin dari harga jual **saat itu**, sesuai AB-04.

```
INSERT INTO penjualan (no_nota_penjualan, tgl_penjualan, id_petugas,
id_anggota, bayar_penjualan)
VALUES ('PJ-2609-0142', '2026-09-24 10:15:00', 3, 1, 30000);

INSERT INTO detail_penjualan (id_penjualan, id_barang,
qty_detail_penjualan, harga_satuan_detail_penjualan)
SELECT 1, id_barang, 3, harga_jual_barang FROM barang WHERE kode_barang
= 'ATK-001';
```

4. Lengkapi dua barang lain pada nota yang sama, lalu kurangi stok setiap barang sesuai qty dengan **UPDATE ... WHERE id_barang =**

Titik Analisis: 1

Perintah **INSERT ... SELECT** di atas mengambil harga dari tabel barang. Mengapa cara ini lebih aman daripada mengetik angka 4000 secara manual? Setelah harga pulpen naik menjadi Rp4.500 minggu depan, apakah nota PJ-2609-0142 ikut berubah?

D.3 Mengubah dan Menghapus dengan Aman

5. Aktifkan mode aman, lalu coba menaikkan harga tanpa **WHERE**. Perintah pertama harus ditolak.

```
SET SESSION sql_safe_updates = 1;
UPDATE barang SET harga_jual_barang = harga_jual_barang * 1.1;
-- ditolak
UPDATE barang SET harga_jual_barang = harga_jual_barang * 1.1
WHERE kategori_barang = 'ATK' AND id_barang > 0;
-- diterima
```

6. Hapus nota uji kedua yang Anda buat, lalu buktikan bahwa baris detailnya ikut terhapus karena **ON DELETE CASCADE**. Setelah itu coba menghapus petugas yang pernah melayani penjualan, dan catat galatnya.

Titik Analisis: 2

Menghapus nota menghapus detailnya, tetapi menghapus petugas ditolak. Jelaskan kedua perilaku ini dari definisi constraint di Modul 5. Bagaimana cara yang benar jika seorang kasir berhenti bekerja?

D.4 Menjalankan Skenario Uji

7. Buat **p06_uji_kopma_<NIM>.md** berisi minimal delapan skenario seperti Tabel 6.1. Jalankan setiap masukan, salin hasil nyata dari layar, lalu tentukan statusnya.
8. Jalankan ulang seluruh skrip dari awal (DDL Modul 5 lalu data Modul 6). Hasilnya harus sama. Commit dengan pesan **p06: data dan uji kopma**.

Tabel 6.1 Contoh skenario uji Kopma

Kode	Masukan	Harapan	Aturan
UJ-01	Nota baru dengan anggota aktif	Diterima	AB-02
UJ-02	Nota tanpa anggota (umum)	Diterima, id_anggota NULL	AB-02
UJ-03	Stok diubah menjadi -5	Ditolak, ERROR 4025	AB-03
UJ-04	Nota dengan nomor yang sudah ada	Ditolak, ERROR 1062	AB-01
UJ-05	Detail dengan qty 0	Ditolak, ERROR 4025	ck_detail_qty
UJ-06	Anggota dengan NIM kembar	Ditolak, ERROR 1062	AB-05
UJ-07	Penjualan oleh petugas tidak dikenal	Ditolak, ERROR 1452	fk_penjualan_petugas
UJ-08	UPDATE tanpa WHERE di mode aman	Ditolak, ERROR 1175	kebijakan lab

Titik Analisis: 3

Semua uji di atas akan berstatus LULUS. Tuliskan satu uji yang Anda perkirakan **GAGAL** karena aturan bisnisnya belum dijaga server, misalnya penjualan yang qty-nya melebihi stok (AB-03). Mengapa constraint **CHECK** di tabel barang tidak mencegahnya?

E. Latihan dan Modifikasi

1. Pindahkan semua anggota berstatus **nonaktif** yang mendaftar sebelum 2025 ke tabel arsip **anggota_arsip** dengan **CREATE TABLE ... LIKE** dan **INSERT ... SELECT**, lalu hapus dari tabel asal. Pastikan anggota yang masih punya penjualan tidak ikut terhapus.
2. Tulis satu **UPDATE** yang menaikkan harga semua barang kategori makanan sebesar 5% dan dibulatkan ke ratusan terdekat.

F. Tugas Mandiri: Milestone Proyek 6

- Tulis **p06_data_<NIM>.sql**: setiap tabel induk minimal 5 baris, dan tabel transaksi utama minimal **10 × P** baris yang tersebar pada minimal tiga tanggal berbeda.
- Tulis **p06_uji_<NIM>.md** berisi minimal 10 skenario uji (sedikitnya 5 negatif), masing-masing merujuk AB-xx atau nama constraint.
- Sertakan satu uji yang **gagal** (aturan belum dapat dijaga) dan catat sebagai utang desain untuk Modul 14.
- Skrip DDL dan data harus dapat dijalankan ulang dari basis data kosong dengan hasil yang sama.

Catatan: hubungan ke pertemuan berikutnya

Data sampel ini menjadi bahan seluruh kueri Pertemuan 7–15 dan data awal migrasi di Buku 2. Pastikan datanya realistis, bukan “abc” atau “test1”.

Troubleshooting Umum

Masalah 1: ERROR 1175 safe update mode

Cara membaca. Server menolak **UPDATE** atau **DELETE** karena syaratnya tidak memakai kolom kunci. Ini perlindungan, bukan kesalahan server. **Perbaikan:** tambahkan syarat yang tepat pada kolom kunci. Jangan menonaktifkan mode aman hanya agar perintah lolos.

Masalah 2: ERROR 1452 saat mengisi tabel anak

Cara membaca. Nilai kunci tamu belum ada di tabel induk. Biasanya karena urutan pengisian salah atau nilai **AUTO_INCREMENT** induk tidak sesuai perkiraan. **Perbaikan:** isi induk dulu, lalu gunakan **INSERT ... SELECT** dengan kode alami (misalnya **kode_barang**) agar tidak menebak nilai **id**.

Masalah 3: Hasil berbeda saat skrip dijalankan ulang

Cara membaca. Nilai **AUTO_INCREMENT** tidak kembali ke 1 setelah **DELETE**. **Perbaikan:** bangun ulang dari DDL (yang sudah memuat **DROP TABLE IF EXISTS**), bukan menghapus isi tabel satu per satu.

Checklist Laporan Pertemuan 6

Laporan disimpan sebagai **laporan/p06_laporan_<NIM>.md** dengan struktur baku, lalu di-push sebelum batas waktu. Pastikan semua **checklist umum** (lihat Pedoman dan Checklist Laporan Praktikum) dan checklist khusus berikut terpenuhi.

Tabel 6.2 Checklist khusus laporan Pertemuan 6

	Jenis	Yang harus ada
☐	Berkas wajib	p06_data_<NIM>.sql (data sampel berbasis parameter P) dan p06_uji_<NIM>.md (skenario uji)
☐	Bukti tangkapan layar	Hasil skenario uji positif dan negatif; jumlah baris per tabel
☐	Analisis wajib	Tabel skenario uji: masukan, hasil yang diharapkan, hasil nyata, status

Perhatian: aturan keaslian

Jumlah baris transaksi proyek bergantung pada P milik Anda. Nama pelanggan, barang, atau entitas fiktif lain tidak boleh sama persis dengan milik mahasiswa lain.

Rubrik Penilaian Modul

Tabel 6.3 Rubrik penilaian Modul 6

Aspek	Bobot	Indikator
Pelaksanaan langkah D	25%	Data Kopma lengkap, delapan uji tercatat dengan hasil nyata
Titik Analisis dan latihan E	20%	Penjelasan CASCADE, uji gagal, dan pengarsipan tepat
Milestone Proyek 6	40%	Data berbasis P, 10 uji termasuk uji negatif dan uji gagal, dapat diulang
Laporan dan bukti Git	15%	Lengkap sesuai checklist

Sinkronisasi Kompetensi

Tabel 6.4 Unit kompetensi yang dilatih di Modul 6

Kode unit	Judul unit	Elemen yang dilatih
J.62DMS00.010.1	Membuat Basis Data	Elemen 1: membuat basis data (mengisi data); Elemen 2: menguji basis data

Modul 7: Menjawab Pertanyaan dengan SELECT dan Fungsi Skalar

Cerita Pertemuan Ini

Data Kopma kini terisi dan teruji. Ketua mulai datang dengan pertanyaan: “Barang apa yang stoknya hampir habis? Siapa anggota yang mendaftar bulan ini? Nota mana yang dibayar dengan uang pas?” Semua jawabannya ada di basis data, tetapi hanya dapat diambil dengan pertanyaan yang ditulis tepat. Hari ini Anda belajar “bertanya” kepada basis data.

A. Tujuan Praktikum

Setelah menyelesaikan modul ini, mahasiswa mampu:

1. menulis **SELECT** dengan proyeksi kolom, alias, dan **DISTINCT**;
2. menyaring baris dengan operator pembandingan, logika, **BETWEEN**, **IN**, **LIKE**, dan **IS NULL**;
3. mengurutkan dan membatasi hasil dengan **ORDER BY** dan **LIMIT**;
4. memakai fungsi skalar string, tanggal, numerik, dan kondisional untuk menyajikan hasil sesuai kebutuhan informasi.

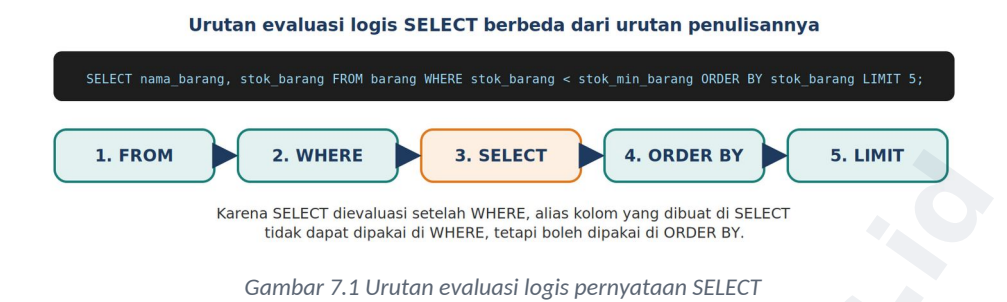
B. Alat dan Bahan

- XAMPP 8.2.12 dan **kopma_<3d>** berisi data Modul 6.
- Daftar kebutuhan informasi (KI) dari Pertemuan 2.
- Editor teks, repositori **basisdata-<NIM>**.

C. Dasar Teori

C.1 Anatomi dan Urutan Evaluasi SELECT

Pernyataan **SELECT** ditulis dengan urutan SELECT–FROM–WHERE–ORDER BY–LIMIT, tetapi dievaluasi secara logis dengan urutan berbeda (Gambar 7.1) [12]. Memahami urutan ini menjelaskan banyak galat pemula, misalnya alias kolom yang “tidak dikenal” di **WHERE**.



C.2 Kesalahan yang Tidak Memunculkan Galat

Galat sintaks mudah ditemukan karena server menolaknya. Yang lebih berbahaya adalah **kesalahan semantik**: kueri berjalan tanpa galat, tetapi hasilnya tidak menjawab pertanyaan. Penelitian pendidikan basis data menunjukkan jenis kesalahan ini sering terjadi pada mahasiswa dan sulit disadari [2], [3]. Karena itu, setiap kueri di buku ini selalu dipasangkan dengan **hasil yang diharapkan** sebelum dijalankan.

Tabel 7.1 Fungsi skalar yang dipakai di modul ini

Kelompok	Fungsi	Contoh pemakaian di Kopma
String	CONCAT, UPPER, LEFT, LENGTH	Label “nama (no_anggota)”
Tanggal	DATE_FORMAT, DATEDIFF, CURDATE, YEAR	Tanggal daftar format Indonesia, umur keanggotaan
Numerik	ROUND, FLOOR, ABS	Pembulatan harga ke ratusan
Kondisional	IF, IFNULL, CASE	Keterangan diskon; nomor HP kosong ditampilkan “-”

D. Langkah Percobaan

D.1 Proyeksi dan Penyaringan

1. Tulis setiap kueri di `p07_kueri_kopma_<NIM>.sql` dan beri komentar kode KI atau pertanyaan yang dijawabnya. Mulai dengan KI-03: barang di bawah stok minimum.

```
-- KI-03: barang yang perlu dipesan ulang
SELECT kode_barang, nama_barang, stok_barang, stok_min_barang,
       stok_min_barang - stok_barang AS kurang
FROM barang
WHERE stok_barang < stok_min_barang
ORDER BY kurang DESC;
```

```

Shell XAMPP - KI-03 barang di bawah stok minimum

MariaDB [kopma_123]> SELECT kode_barang, nama_barang, stok_barang, stok_min_barang,
->      stok_min_barang - stok_barang AS kurang
-> FROM barang WHERE stok_barang < stok_min_barang ORDER BY kurang DESC;
+-----+-----+-----+-----+-----+
| kode_barang | nama_barang | stok_barang | stok_min_barang | kurang |
+-----+-----+-----+-----+-----+
| MKN-001    | Roti Cokelat | 6          | 10              | 4      |
+-----+-----+-----+-----+-----+
1 row in set (0.001 sec)

```

Ilustrasi disederhanakan; tampilan sebenarnya dapat sedikit berbeda.

Gambar 7.2 Hasil kueri KI-03 barang di bawah stok minimum

2. Tulis tiga kueri penyaringan lain: barang kategori ATK atau minuman (**IN**), nota pada rentang tanggal tertentu (**BETWEEN**), dan anggota tanpa nomor HP (**IS NULL**).

Titik Analisis: 1

Ganti **ORDER BY kurang** menjadi **WHERE kurang > 0**. Catat galat yang muncul dan jelaskan dengan urutan evaluasi pada Gambar 7.1. Bagaimana menulis syarat yang setara tanpa memakai alias?

D.2 Pola Teks dan Nilai Kosong

3. Cari anggota yang namanya mengandung huruf tertentu dan tampilkan label yang mudah dibaca.

```

SELECT CONCAT(nama_anggota, ' (', no_anggota, ')') AS anggota,
       DATE_FORMAT(tgl_daftar_anggota, '%d-%m-%Y') AS
tgl_daftar_anggota,
       IFNULL(no_hp_anggota, '-') AS hp,
       CASE status_anggota WHEN 'aktif' THEN 'Diskon 5%'
       ELSE 'Tanpa diskon' END AS keterangan
FROM anggota
WHERE nama_anggota LIKE '%a%'
ORDER BY tgl_daftar_anggota DESC
LIMIT 5;

```

Titik Analisis: 2

Mengapa **WHERE no_hp_anggota = NULL** tidak pernah mengembalikan baris, padahal ada anggota tanpa nomor HP? Tulis kueri yang benar, lalu jelaskan perbedaan **NULL** dengan string kosong **''**.

D.3 Tanggal dan Perhitungan per Baris

4. Hitung lama keanggotaan setiap anggota dalam hari dan tampilkan kategori “baru” (kurang dari 180 hari) atau “lama”.
5. Tampilkan setiap baris detail nota PJ-2609-0142 beserta subtotalnya (**qty_detail_penjualan * harga_satuan_detail_penjualan**). Bandingkan jumlah manualnya dengan nota di Modul 2.
6. Commit dengan pesan **p07: kueri kopma**.

Titik Analisis: 3

Subtotal per baris dapat dihitung di modul ini, tetapi **total** nota belum. Mengapa? Fitur SQL apa yang dibutuhkan untuk menjumlahkan banyak baris menjadi satu nilai? (Jawabannya dipelajari di Pertemuan 9.)

E. Latihan dan Modifikasi

1. Tampilkan nama barang dalam huruf kapital dan harga dalam format “Rp4.000”.
Petunjuk: `FORMAT(harga_jual_barang, 0, 'id_ID')`.
2. Tulis kueri yang salah secara semantik tetapi tetap berjalan, misalnya syarat **OR** tanpa tanda kurung. Tunjukkan hasil salahnya, lalu perbaiki.

F. Tugas Mandiri: Milestone Proyek 7

- Tulis `p07_kueri_<NIM>.sql` berisi minimal **10 kueri satu tabel** yang mendukung KI proyek.
- Setiap kueri diberi komentar: kode KI, pertanyaan dalam bahasa sehari-hari, dan hasil yang diharapkan sebelum dijalankan.
- Gunakan minimal: **LIKE, BETWEEN, IN, IS NULL, CASE**, satu fungsi tanggal, dan satu fungsi string.
- Tandai KI yang belum dapat dijawab dengan satu tabel; KI itu akan dijawab di Pertemuan 9–12.

Catatan: hubungan ke pertemuan berikutnya

Pertemuan 8 adalah UTS Proyek Tahap 1. Milestone 1–7 harus terintegrasi dan dapat dipasang ulang dari skrip. Kueri milestone ini juga akan diukur kinerjanya di Buku 2 Modul 6.

Troubleshooting Umum

Masalah 1: ERROR 1054 Unknown column ... in 'where clause'

Cara membaca. Pesan menyebut nama kolom dan klausa tempat galat terjadi. Jika nama itu adalah alias, penyebabnya urutan evaluasi. Jika bukan alias, periksa ejaan kolom dan tabel pada **FROM**. **Perbaikan:** ulangi ekspresinya di **WHERE**, atau periksa `DESCRIBE nama_tabel;`.

Masalah 2: Hasil kosong padahal data ada

Cara membaca. Tidak ada galat, jadi ini kesalahan semantik. Periksa perbandingan dengan **NULL**, spasi berlebih pada data teks, dan batas **BETWEEN** untuk **DATETIME** (tanggal akhir dibaca sebagai pukul 00.00). **Perbaikan:** gunakan `tgl_penjualan >= '2026-09-01' AND tgl_penjualan < '2026-10-01'` untuk rentang bulanan.

Masalah 3: Urutan hasil berubah-ubah

Cara membaca. Tanpa **ORDER BY**, urutan baris tidak dijamin. **Perbaikan:** selalu tulis **ORDER BY** jika urutan penting, terutama bersama **LIMIT**.

Checklist Laporan Pertemuan 7

Laporan disimpan sebagai `laporan/p07_laporan_<NIM>.md` dengan struktur baku, lalu di-push sebelum batas waktu. Pastikan semua **checklist umum** (lihat Pedoman dan Checklist Laporan Praktikum) dan checklist khusus berikut terpenuhi.

Tabel 7.2 Checklist khusus laporan Pertemuan 7

	Jenis	Yang harus ada
☐	Berkas wajib	<code>p07_kueri_<NIM>.sql</code> berisi minimal 10 kueri satu tabel, masing-masing diberi komentar KI dan hasil yang diharapkan
☐	Bukti tangkapan layar	Hasil setiap kueri
☐	Analisis wajib	Pemetaan kueri – KI; alasan pemilihan fungsi skalar yang dipakai

Perhatian: aturan keaslian

Setiap kueri proyek wajib memuat komentar pertanyaan dan hasil yang diharapkan yang Anda tulis sendiri sebelum menjalankannya. Hasil yang diharapkan yang identik dengan hasil nyata tanpa satu pun perbaikan akan ditanyakan saat viva.

Rubrik Penilaian Modul

Tabel 7.3 Rubrik penilaian Modul 7

Aspek	Bobot	Indikator
Pelaksanaan langkah D	25%	Kueri Kopma benar dan hasilnya sesuai harapan
Titik Analisis dan latihan E	20%	Penjelasan urutan evaluasi, NULL, dan kesalahan semantik tepat
Milestone Proyek 7	40%	10 kueri berkomentar KI dan harapan, cakupan operator lengkap
Laporan dan bukti Git	15%	Lengkap sesuai checklist

Sinkronisasi Kompetensi

Tabel 7.4 Unit kompetensi yang dilatih di Modul 7

Kode unit	Judul unit	Elemen yang dilatih
J.62DMS00.019.1	Menggunakan Data	Elemen 2: kebutuhan akses data diidentifikasi, dipenuhi dengan tools yang ditentukan, dan didokumentasikan (KUK 2.1–2.3)
J.620100.020.02	Menggunakan SQL	Menulis perintah SQL untuk mengambil dan menyajikan data

Pertemuan 8: UTS Proyek Tahap 1

Cerita Pertemuan Ini

Setengah semester berlalu. Pengurus Kopma ingin melihat hasilnya: bukan presentasi slide, tetapi basis data yang benar-benar berjalan. Hari ini Anda berperan sebagai pengembang yang melakukan serah terima tahap pertama kepada klien. Klien berhak meminta perubahan kecil di tempat, dan Anda harus mampu menjelaskan setiap keputusan yang telah diambil sejak Pertemuan 1.

A. Tujuan dan Cakupan

UTS mengukur kemampuan mengintegrasikan milestone Pertemuan 1–7 menjadi satu basis data proyek yang utuh, dapat dipasang ulang, dan dapat dipertanggungjawabkan. Pendekatan evaluasi berbasis proyek dipilih karena menuntut penerapan konsep pada persoalan utuh, bukan hafalan sintaks [5].

Tabel 8.1 Cakupan UTS

Milestone	Yang diperiksa
1 – Lingkungan	Basis data dan akun proyek, README, <code>.gitignore</code>
2 – Kebutuhan data	PB, AB, KI, CRUD, kamus data, parameter P
3–4 – ERD dan normalisasi	ERD konsisten dengan skema 3NF; catatan tinjauan klien
5–6 – DDL dan data	Skrip dapat diulang; constraint teruji; data berbasis P
7 – Kueri	10 kueri satu tabel dengan komentar KI dan hasil yang diharapkan

B. Persiapan Mahasiswa

1. Tulis `install_uts_<NIM>.sql` yang memanggil DDL dan data secara berurutan, sehingga seluruh basis data proyek dapat dibangun dari nol dengan satu perintah.
2. Uji instalasi bersih di komputer lain atau pada basis data baru bernama `<kode_tema>_<3d>_uji`.
3. Siapkan `p08_uts_<NIM>.md` berisi ringkasan milestone 1–7 dan tautan setiap berkasnya.
4. Pastikan commit terakhir dibuat sebelum batas waktu. Dosen menilai isi repositori pada commit tersebut.

Contoh skrip instalasi

```
-- install_uts_2301010123.sql (dijalankan dari folder repositori)
SOURCE p05_ddl_2301010123.sql;
SOURCE p06_data_2301010123.sql;
```

C. Alur Pelaksanaan

Setiap mahasiswa menjalani empat tahap dalam kurang lebih 20 menit (Gambar 8.1).



Gambar 8.1 Alur ujian proyek

Tabel 8.2 Rincian tahap UTS

Tahap	Waktu	Yang dilakukan
Persiapan	3 menit	Menjalankan <code>install_uts_<NIM>.sql</code> di depan dosen pada basis data kosong
Demo	7 menit	Menunjukkan ERD, skema, uji constraint, dan kueri KI terpenting
Modifikasi langsung	5 menit	Mengerjakan satu permintaan perubahan dari bank modifikasi
Viva	5 menit	Menjawab pertanyaan tentang alasan desain

D. Bank Modifikasi Langsung

Dosen memilih satu permintaan secara acak. Permintaan dirancang agar dapat diselesaikan dalam lima menit oleh mahasiswa yang memahami proyeknya sendiri.

1. Tambahkan satu kolom baru yang wajib diisi dengan nilai bawaan pada tabel yang ditunjuk dosen.
2. Tambahkan constraint **CHECK** baru untuk aturan bisnis yang disebutkan dosen, lalu buktikan penolakannya.
3. Ubah satu kueri milestone 7 agar hasilnya diurutkan dan dibatasi berbeda.
4. Tulis kueri baru untuk pertanyaan sederhana dari dosen tentang data proyek.
5. Tambahkan satu data yang melanggar aturan, jelaskan galat yang muncul, lalu perbaiki datanya.
6. Ubah aksi **ON DELETE** pada satu kunci tamu dan jelaskan dampaknya.

E. Contoh Pertanyaan Viva

- Aturan bisnis mana yang menentukan kardinalitas relasi X, dan apa yang berubah jika aturannya diubah?

- Mengapa kolom Y memakai tipe data ini dan bukan tipe lain?
- Tunjukkan satu dependensi fungsional di proyek Anda dan bagaimana normalisasi menanganinya.
- Aturan bisnis mana yang belum dapat dijaga server, dan bagaimana rencana Anda?
- Bagian mana dari proyek yang dibantu asisten AI, dan bagaimana Anda memverifikasinya?

F. Rubrik Penilaian UTS

Tabel 8.3 Rubrik rinci UTS

Aspek (bobot)	Sangat baik	Cukup	Kurang
Produk berjalan (40%)	Instalasi bersih sukses; semua milestone konsisten	Berjalan dengan perbaikan kecil	Gagal dipasang atau milestone hilang
Demo, modifikasi, viva (30%)	Modifikasi selesai; alasan desain jelas	Modifikasi selesai dengan bantuan	Modifikasi gagal; tidak dapat menjelaskan
Dokumentasi dan Git (20%)	Commit rutin P1–P7, dokumen lengkap	Commit tidak rutin, dokumen cukup	Commit menumpuk di akhir
Ketepatan waktu (10%)	Sebelum batas waktu	–	Terlambat

Perhatian: ketentuan khusus

Mahasiswa yang tidak dapat memasang basis datanya dari skrip diberi kesempatan satu kali memperbaiki di tempat dalam 10 menit. Mahasiswa yang tidak mampu menjelaskan proyeknya sendiri saat viva diberi nilai aspek produk paling tinggi “cukup”, meskipun produknya berjalan.

Checklist Laporan Pertemuan 8

Laporan disimpan sebagai `laporan/p08_laporan_<NIM>.md` dengan struktur baku, lalu di-push sebelum batas waktu. Pastikan semua **checklist umum** (lihat Pedoman dan Checklist Laporan Praktikum) dan checklist khusus berikut terpenuhi.

Tabel 8.4 Checklist khusus laporan Pertemuan 8

	Jenis	Yang harus ada
☐	Berkas wajib	<code>p08_uts_<NIM>.md</code> (ringkasan milestone 1–7) dan <code>install_uts_<NIM>.sql</code>
☐	Bukti tangkapan layar	Bukti basis data berjalan dari skrip DDL dan data (instalasi bersih)

	Jenis	Yang harus ada
☐	Analisis wajib	Catatan modifikasi langsung saat demo; refleksi keputusan desain yang akan diperbaiki

Sinkronisasi Kompetensi

Tabel 8.5 Unit kompetensi yang diujikan pada UTS

Kode unit	Judul unit	Bukti
J.62DMS00.001.1	Mengidentifikasi Kebutuhan Pengelolaan Data	Dokumen kebutuhan data
J.62DMS00.006.1	Merancang Basis Data	ERD, normalisasi, DDL
J.62DMS00.010.1	Membuat Basis Data	Instalasi dan uji
J.62DMS00.019.1	Menggunakan Data	Kueri KI

Modul 9: Meringkas Data dengan Fungsi Agregat dan GROUP BY

Cerita Pertemuan Ini

Beberapa minggu setelah serah terima tahap pertama, Kopma sudah mencatat 75 nota sepanjang September dan Oktober. Menjelang rapat anggota, ketua meminta satu lembar ringkasan: berapa nota dan berapa omzet setiap hari, setiap bulan, dan totalnya. Pertanyaan yang di Pertemuan 7 belum bisa dijawab, yaitu total satu nota, kini menjadi kunci. Hari ini Anda belajar meringkas banyak baris menjadi satu angka yang bermakna.

A. Tujuan Praktikum

Setelah menyelesaikan modul ini, mahasiswa mampu:

1. memakai fungsi agregat **COUNT**, **SUM**, **AVG**, **MIN**, dan **MAX** dengan benar, termasuk perilakunya terhadap **NULL**;
2. mengelompokkan data dengan **GROUP BY** dan menyaring kelompok dengan **HAVING**;
3. menyusun laporan ringkasan bertingkat dengan **WITH ROLLUP**;
4. mencegah kesalahan semantik agregasi dengan mode **ONLY_FULL_GROUP_BY**.

B. Alat dan Bahan

- XAMPP 8.2.12 dan `kopma_<3d>` hasil DDL Modul 5.
- Berkas `kopma_data_lanjutan.sql` dari dosen: 12 barang, 12 anggota, dan 75 nota selama September–Oktober 2026 (termasuk kenaikan harga pulpen pada 15 Oktober). Berkas ini dipakai sampai Pertemuan 16.
- Daftar KI dari Pertemuan 2.

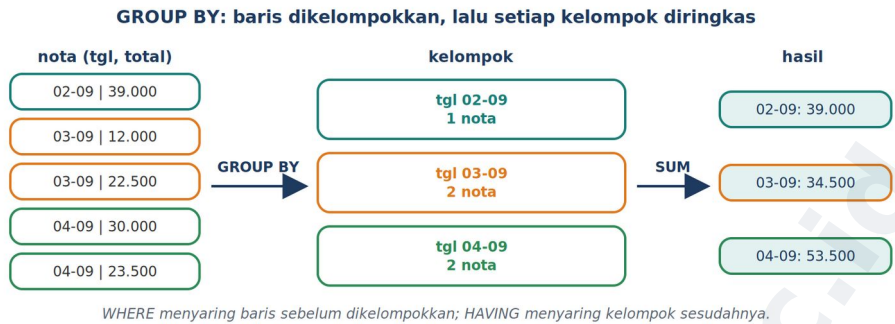
Catatan: memuat data lanjutan

Jalankan `mysql -u mhs_<3d> -p < kopma_data_lanjutan.sql` setelah mengganti `kopma_123` di baris **USE** dengan nama basis data Anda. Skrip mengosongkan tabel Kopma lalu mengisinya ulang, sehingga hasil Anda akan sama persis dengan contoh di buku.

C. Dasar Teori

C.1 Fungsi Agregat

Fungsi agregat menghitung satu nilai dari banyak baris [12]. Tanpa **GROUP BY**, seluruh baris menjadi satu kelompok. Dengan **GROUP BY**, baris dikelompokkan menurut nilai kolom tertentu, lalu setiap kelompok diringkas (Gambar 9.1).



Gambar 9.1 Cara kerja GROUP BY

Tabel 9.1 Fungsi agregat dan perilakunya terhadap NULL

Fungsi	Hasil	Terhadap NULL
COUNT (*)	Jumlah baris	Menghitung semua baris
COUNT(kolom)	Jumlah nilai bukan NULL	NULL diabaikan
COUNT(DISTINCT kolom)	Jumlah nilai berbeda	NULL diabaikan
SUM, AVG	Jumlah dan rata-rata	NULL diabaikan; AVG membagi dengan jumlah nilai bukan NULL
MIN, MAX	Nilai terkecil dan terbesar	NULL diabaikan

C.2 Agregasi yang Salah Tanpa Galat

Secara bawaan, MariaDB mengizinkan kolom yang tidak diagregasi dan tidak ada di **GROUP BY** muncul di **SELECT**. Nilainya diambil dari salah satu baris dalam kelompok, tanpa jaminan baris mana. Inilah contoh kesalahan semantik: kueri berjalan, tetapi hasilnya dapat menyesatkan [2]. Mode **ONLY_FULL_GROUP_BY** membuat server menolak kueri seperti itu dengan **ERROR 1055** [13]. Buku ini mewajibkan mode tersebut sejak modul ini.

D. Langkah Percobaan

D.1 Menyiapkan Mode dan Data

1. Muat `kopma_data_lanjutan.sql`, lalu aktifkan mode ketat pengelompokan untuk sesi Anda.

```
SET SESSION sql_mode = CONCAT(@@sql_mode, ',ONLY_FULL_GROUP_BY');
SELECT COUNT(*) AS nota, MIN(tgl_penjualan) AS awal, MAX(tgl_penjualan) AS akhir FROM penjualan;
```
2. Hasil yang diharapkan: 75 nota, dari 2 September sampai 31 Oktober 2026. Jika berbeda, muat ulang data.

D.2 Total Satu Nota

3. Jawab pertanyaan yang tertunda dari Pertemuan 7: total nota PJ-2609-0142 di Modul 2 (atau nota lain di data lanjutan).

```
SELECT p.no_nota_penjualan, SUM(d.qty_detail_penjualan *
d.harga_satuan_detail_penjualan) AS jumlah
FROM penjualan p JOIN detail_penjualan d ON d.id_penjualan =
p.id_penjualan
WHERE p.no_nota_penjualan = 'PJ-2609-0001'
GROUP BY p.id_penjualan, p.no_nota_penjualan;
```

Titik Analisis: 1

Hapus **p.no_nota_penjualan** dari **GROUP BY** (sisakan **p.id_penjualan**). Catat galat 1055 yang muncul. Secara logika satu **id_penjualan** hanya punya satu **no_nota_penjualan**, tetapi MariaDB tetap menolak. Mengapa lebih aman menuliskan semua kolom non-agregat di **GROUP BY**?

D.3 Laporan Omzet Harian dan Bulanan (KI-01)

4. Susun laporan omzet harian untuk minggu pertama September dengan baris total dari **WITH ROLLUP** (Gambar 9.2).

```
SELECT DATE(p.tgl_penjualan) AS tanggal,
COUNT(DISTINCT p.id_penjualan) AS nota,
SUM(d.qty_detail_penjualan * d.harga_satuan_detail_penjualan)
AS omzet
FROM penjualan p
JOIN detail_penjualan d ON d.id_penjualan = p.id_penjualan
WHERE p.tgl_penjualan >= '2026-09-01' AND p.tgl_penjualan < '2026-09-08'
GROUP BY DATE(p.tgl_penjualan) WITH ROLLUP;
```

```
Shell XAMPP - KI-01 omzet harian
MariaDB [kopma_123]> SELECT DATE(p.tgl_penjualan) AS tanggal,
-> COUNT(DISTINCT p.id_penjualan) AS nota,
-> SUM(d.qty_detail_penjualan * d.harga_satuan_detail_penjualan) AS omzet
-> FROM penjualan p JOIN detail_penjualan d ON d.id_penjualan = p.id_penjualan
-> WHERE p.tgl_penjualan >= '2026-09-01' AND p.tgl_penjualan < '2026-09-08'
-> GROUP BY DATE(p.tgl_penjualan) WITH ROLLUP;
+-----+-----+-----+
| tanggal | nota | omzet |
+-----+-----+-----+
| 2026-09-02 | 1 | 39000.00 |
| 2026-09-03 | 2 | 34500.00 |
| 2026-09-04 | 2 | 53500.00 |
| 2026-09-05 | 2 | 35000.00 |
| 2026-09-07 | 3 | 38000.00 |
| NULL | 10 | 200000.00 | <- baris total dari WITH ROLLUP
+-----+-----+-----+
```

Hasil nyata dari data kopma_data_lanjutan.sql.

Gambar 9.2 Laporan KI-01 omzet harian dengan baris total

5. Ubah pengelompokan menjadi per bulan dengan **DATE_FORMAT(p.tgl_penjualan, '%Y-%m')**. Hasil yang diharapkan: September 39 nota (Rp691.000), Oktober 36 nota (Rp507.500).

Titik Analisis: 2

Mengapa kueri memakai `COUNT(DISTINCT p.id_penjualan)`, bukan `COUNT(*)`? Jalankan keduanya pada data September dan jelaskan selisihnya dengan hubungan satu nota–banyak detail.

D.4 Menyaring Kelompok dengan HAVING

6. Tampilkan kasir yang melayani lebih dari 20 nota, lalu barang yang total qty terjualnya kurang dari 10.

```
SELECT pt.kode_petugas, pt.nama_petugas, COUNT(*) AS nota
FROM penjualan p JOIN petugas pt ON pt.id_petugas = p.id_petugas
GROUP BY pt.id_petugas, pt.kode_petugas, pt.nama_petugas
HAVING COUNT(*) > 20;
```

Titik Analisis: 3

Ketua meminta rata-rata belanja per nota **untuk anggota saja**. Syarat “anggota saja” harus ditulis di `WHERE` atau `HAVING`? Jelaskan dengan urutan evaluasi, lalu tulis kuerinya.

7. Simpan semua kueri di `p09_agregasi_kopma_<NIM>.sql` dengan komentar KI dan hasil yang diharapkan. Commit dengan pesan `p09: agregasi kopma`.

E. Latihan dan Modifikasi

1. Hitung total nota lengkap dengan diskon 5% untuk anggota aktif (AB-02). Bandingkan dengan kolom `bayar_penjualan`, lalu tampilkan nota yang kembaliannya lebih dari Rp4.000.
2. Tampilkan omzet per kategori barang per bulan dengan `WITH ROLLUP`, dan jelaskan arti setiap baris `NULL` yang muncul.

F. Tugas Mandiri: Milestone Proyek 9

- Tulis `p09_agregasi_<NIM>.sql` berisi minimal 5 laporan ringkasan proyek, masing-masing dipetakan ke KI.
- Minimal satu laporan memakai `HAVING` dan satu memakai `WITH ROLLUP`.
- Semua kueri dijalankan dengan `ONLY_FULL_GROUP_BY` aktif.
- Tambahkan data transaksi proyek bila perlu, sehingga setiap laporan memiliki minimal tiga kelompok.

Troubleshooting Umum**Masalah 1: ERROR 1055 ... isn't in GROUP BY**

Cara membaca. Pesan menyebut kolom lengkap dengan nama basis data dan alias tabel. Kolom itu muncul di `SELECT` tanpa agregat dan tanpa ada di `GROUP BY`. **Perbaikan:** tambahkan kolom ke `GROUP BY`, atau bungkus dengan agregat bila memang nilai ringkasan yang dimaksud.

Masalah 2: Angka omzet berlipat

Cara membaca. Tidak ada galat, tetapi omzet terlalu besar. Biasanya karena **JOIN** ke tabel lain yang juga banyak baris per nota, sehingga setiap detail terhitung berulang. **Perbaikan:** periksa jumlah baris sebelum dikelompokkan, dan agregasikan dulu di tingkat nota.

Masalah 3: AVG tidak sesuai hitungan manual

Cara membaca. **AVG** mengabaikan **NULL**. Jika maksud Anda “NULL dihitung nol”, gunakan **AVG(IFNULL(kolom, 0))** dan tuliskan alasannya di komentar.

Checklist Laporan Pertemuan 9

Laporan disimpan sebagai `laporan/p09_laporan_<NIM>.md` dengan struktur baku, lalu di-push sebelum batas waktu. Pastikan semua **checklist umum** (lihat Pedoman dan Checklist Laporan Praktikum) dan checklist khusus berikut terpenuhi.

Tabel 9.2 Checklist khusus laporan Pertemuan 9

	Jenis	Yang harus ada
☐	Berkas wajib	<code>p09_agregasi_<NIM>.sql</code> berisi kueri laporan ringkasan
☐	Bukti tangkapan layar	Hasil laporan ringkasan (misalnya omzet per hari/bulan)
☐	Analisis wajib	Kaitan setiap laporan dengan KI; penjelasan pemakaian HAVING

Perhatian: aturan keaslian

Laporan ringkasan proyek memakai data berbasis parameter P Anda. Angka hasil yang identik dengan mahasiswa lain dengan tema berbeda adalah tanda data tidak dibuat sendiri.

Rubrik Penilaian Modul

Tabel 9.3 Rubrik penilaian Modul 9

Aspek	Bobot	Indikator
Pelaksanaan langkah D	25%	Laporan KI-01 harian dan bulanan cocok dengan hasil yang diharapkan
Titik Analisis dan latihan E	20%	Penjelasan 1055, COUNT DISTINCT, WHERE vs HAVING tepat
Milestone Proyek 9	40%	Lima laporan ringkasan, HAVING dan ROLLUP, mode ketat
Laporan dan bukti Git	15%	Lengkap sesuai checklist

Sinkronisasi Kompetensi

Tabel 9.4 Unit kompetensi yang dilatih di Modul 9

Kode unit	Judul unit	Elemen yang dilatih
J.62DMS00.019.1	Menggunakan Data	Elemen 2: mengakses data sesuai kebutuhan pengguna dan mendokumentasikannya
J.620100.020.02	Menggunakan SQL	Fungsi agregat, pengelompokan, dan penyaringan kelompok

Modul 10: Menghubungkan Tabel dengan JOIN

Cerita Pertemuan Ini

Laporan omzet sudah dibacakan di rapat anggota. Lalu muncul pertanyaan yang lebih tajam: “Barang apa yang paling laku bulan ini? Siapa anggota yang paling banyak berbelanja? Adakah barang yang sama sekali belum pernah terjual?” Jawabannya tersebar di beberapa tabel. Hari ini Anda menyatukan kembali data yang dipisahkan oleh normalisasi.

A. Tujuan Praktikum

Setelah menyelesaikan modul ini, mahasiswa mampu:

1. menggabungkan dua tabel atau lebih dengan **INNER JOIN** berdasarkan kunci primer dan kunci tamu;
2. memakai **LEFT JOIN** untuk menemukan data yang tidak memiliki pasangan;
3. menggabungkan JOIN dengan agregasi untuk menjawab KI lintas tabel;
4. mengenali dan mencegah hasil perkalian Kartesius yang tidak disengaja.

B. Alat dan Bahan

- XAMPP 8.2.12 dan `kopma_<3d>` berisi `kopma_data_lanjutan.sql`.
- ERD Kopma (Pertemuan 3) sebagai peta jalur JOIN.
- Mode **ONLY_FULL_GROUP_BY** aktif.

C. Dasar Teori

C.1 JOIN Mengikuti Garis di ERD

Normalisasi memecah data ke beberapa tabel; JOIN menyatukannya kembali melalui pasangan kunci tamu–kunci primer [11]. Karena itu, setiap JOIN yang benar dapat ditunjuk sebagai garis relasi di ERD. JOIN tanpa dasar relasi hampir selalu kesalahan.



Gambar 10.1 Perbedaan INNER JOIN dan LEFT JOIN

Tabel 10.1 Jenis JOIN yang dipakai di modul ini

Jenis	Baris yang dihasilkan	Pertanyaan Kopma
INNER JOIN	Hanya baris yang punya pasangan di kedua tabel	Barang apa yang terjual bulan ini?
LEFT JOIN	Semua baris tabel kiri; pasangan kosong bernilai NULL	Barang apa yang belum pernah terjual?
RIGHT JOIN	Kebalikan LEFT JOIN	Jarang dipakai; tulis ulang sebagai LEFT JOIN
Self join	Tabel digabung dengan dirinya sendiri	Pasangan anggota dari prodi yang sama

C.2 Perkalian Kartesius

Jika syarat penggabungan terlupa, setiap baris tabel kiri dipasangkan dengan setiap baris tabel kanan. Pada data Kopma, **penjualan** (75 baris) dan **detail_penjualan** (146 baris) tanpa syarat menghasilkan 10.950 baris. Tidak ada galat, tetapi semua agregat di atasnya salah.

D. Langkah Percobaan

D.1 INNER JOIN: Barang Terlaris (KI-02)

1. Telusuri jalur di ERD: barang – detail_penjualan – penjualan. Tulis kuerinya:

```
SELECT b.kode_barang, b.nama_barang, SUM(d.qty_detail_penjualan) AS
terjual
FROM detail_penjualan d
JOIN barang      b ON b.id_barang      = d.id_barang
JOIN penjualan p ON p.id_penjualan = d.id_penjualan
WHERE p.tgl_penjualan >= '2026-10-01' AND p.tgl_penjualan < '2026-11-
01'
GROUP BY b.id_barang, b.kode_barang, b.nama_barang
ORDER BY terjual DESC
LIMIT 5;
```

- Hasil yang diharapkan: Roti Cokelat (18) berada di urutan teratas, disusul Penghapus (15).

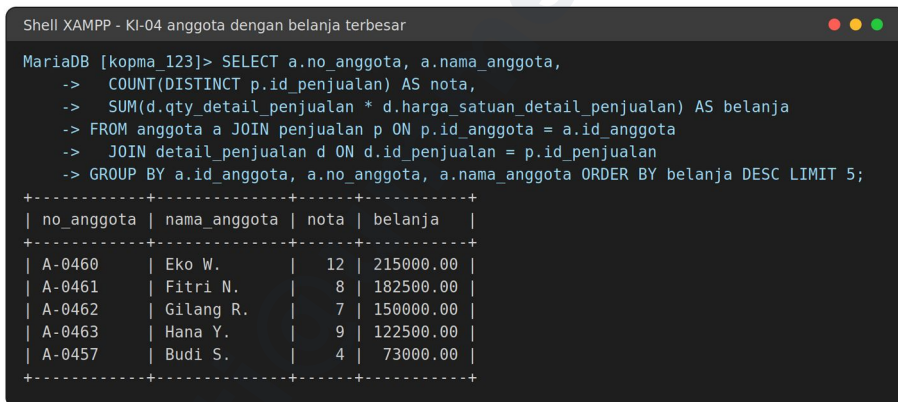
Titik Analisis: 1

Roti Cokelat paling laku di Oktober, padahal stoknya di bawah batas minimum (KI-03 di Pertemuan 7). Informasi apa yang sebaiknya disampaikan kepada petugas gudang? Kueri gabungan apa yang Anda usulkan?

D.2 JOIN Tiga Tabel: Anggota Belanja Terbesar (KI-04)

- Tulis kueri KI-04, lalu cocokkan hasilnya dengan Gambar 10.2.

```
SELECT a.no_anggota, a.nama_anggota,
       COUNT(DISTINCT p.id_penjualan) AS nota,
       SUM(d.qty_detail_penjualan * d.harga_satuan_detail_penjualan)
AS belanja
FROM anggota a
JOIN penjualan p ON p.id_anggota = a.id_anggota
JOIN detail_penjualan d ON d.id_penjualan = p.id_penjualan
GROUP BY a.id_anggota, a.no_anggota, a.nama_anggota
ORDER BY belanja DESC
LIMIT 5;
```



```
Shell XAMPP - KI-04 anggota dengan belanja terbesar

MariaDB [kopma_123]> SELECT a.no_anggota, a.nama_anggota,
->   COUNT(DISTINCT p.id_penjualan) AS nota,
->   SUM(d.qty_detail_penjualan * d.harga_satuan_detail_penjualan) AS belanja
-> FROM anggota a JOIN penjualan p ON p.id_anggota = a.id_anggota
->   JOIN detail_penjualan d ON d.id_penjualan = p.id_penjualan
-> GROUP BY a.id_anggota, a.no_anggota, a.nama_anggota ORDER BY belanja DESC LIMIT 5;

+-----+-----+-----+-----+
| no_anggota | nama_anggota | nota | belanja |
+-----+-----+-----+-----+
| A-0460     | Eko W.       | 12   | 215000.00 |
| A-0461     | Fitri N.     | 8    | 182500.00 |
| A-0462     | Gilang R.    | 7    | 150000.00 |
| A-0463     | Hana Y.      | 9    | 122500.00 |
| A-0457     | Budi S.      | 4    | 73000.00  |
+-----+-----+-----+-----+
```

Hasil nyata dari data kopma_data_lanjutan.sql.

Gambar 10.2 Hasil KI-04 anggota dengan belanja terbesar

D.3 LEFT JOIN: Data Tanpa Pasangan

- Cari barang yang belum pernah terjual. Syaratnya ditulis di **HAVING** karena yang dicari adalah kelompok dengan hitungan nol.

```
SELECT b.kode_barang, b.nama_barang, COUNT(d.id_barang) AS
baris_terjual
FROM barang b
LEFT JOIN detail_penjualan d ON d.id_barang = b.id_barang
GROUP BY b.id_barang, b.kode_barang, b.nama_barang
HAVING COUNT(d.id_barang) = 0;
```

- Hasil yang diharapkan: MKN-004 Wafer Cokelat. Ganti **COUNT(d.id_barang)** menjadi **COUNT(*)**, lalu amati perubahannya.

Titik Analisis: 2

Dengan `COUNT(*)`, Wafer Cokelat justru bernilai 1 dan hilang dari hasil. Mengapa? Kaitkan dengan tabel perilaku `COUNT` terhadap `NULL` di Modul 9.

D.4 Membuktikan Perkalian Kartesius

6. Jalankan `SELECT COUNT(*) FROM penjualan, detail_penjualan;` lalu bandingkan dengan jumlah baris masing-masing tabel. Tambahkan syarat penggabungan dan jalankan ulang.
7. Simpan semua kueri di `p10_join_kopma_<NIM>.sql`, lalu commit dengan pesan `p10: join kopma`.

Titik Analisis: 3

Sebutkan satu cara cepat mendeteksi perkalian Kartesius sebelum menjalankan agregasi, tanpa menghitung manual.

E. Latihan dan Modifikasi

1. Tampilkan setiap nota Oktober beserta nama kasir dan nama anggota. Nota pembeli umum harus tetap tampil dengan nama anggota "Umum".
2. Buktikan AB-04 dengan data: tampilkan nota yang memuat Pulpen Gel 0.5 sebelum dan sesudah 15 Oktober beserta `harga_satuan_detail_penjualan` dan `harga_jual_barang` saat ini.

F. Tugas Mandiri: Milestone Proyek 10

- Tulis `p10_join_<NIM>.sql` berisi minimal 5 kueri lintas tabel yang menjawab KI proyek.
- Minimal satu kueri tiga tabel, satu `LEFT JOIN` untuk data tanpa pasangan, dan satu self join atau `JOIN` dengan agregasi.
- Untuk setiap kueri, tuliskan jalur ERD yang dilalui di komentar.

Troubleshooting Umum**Masalah 1: ERROR 1052 Column ... in field list is ambiguous**

Cara membaca. Nama kolom yang sama ada di lebih dari satu tabel. Standar penamaan buku ini (`nama_barang`, `nama_anggota`) mencegah hal itu pada kolom biasa, tetapi kolom kunci seperti `id_barang` tetap muncul di tabel induk dan anak. **Perbaikan:** selalu awali kolom dengan alias tabel (`d.id_barang`).

Masalah 2: LEFT JOIN berubah menjadi INNER JOIN

Cara membaca. Syarat pada tabel kanan ditulis di `WHERE`, sehingga baris bernilai `NULL` tersaring. **Perbaikan:** pindahkan syarat untuk tabel kanan ke klausa `ON`.

Masalah 3: Angka terlalu besar setelah JOIN

Cara membaca. Tanda perkalian Kartesius atau JOIN ke dua tabel anak sekaligus.
Perbaikan: hitung jumlah baris sebelum agregasi dan periksa setiap syarat **ON**.

Checklist Laporan Pertemuan 10

Laporan disimpan sebagai `laporan/p10_laporan_<NIM>.md` dengan struktur baku, lalu di-push sebelum batas waktu. Pastikan semua **checklist umum** (lihat Pedoman dan Checklist Laporan Praktikum) dan checklist khusus berikut terpenuhi.

Tabel 10.2 Checklist khusus laporan Pertemuan 10

	Jenis	Yang harus ada
☐	Berkas wajib	<code>p10_join_<NIM>.sql</code> berisi kueri relasional
☐	Bukti tangkapan layar	Hasil INNER dan LEFT JOIN, termasuk kasus data tanpa pasangan
☐	Analisis wajib	Perbedaan hasil antarjenis JOIN pada data proyek

Perhatian: aturan keaslian

Komentar jalur ERD pada setiap kueri proyek harus merujuk ERD milik Anda sendiri.

Rubrik Penilaian Modul

Tabel 10.3 Rubrik penilaian Modul 10

Aspek	Bobot	Indikator
Pelaksanaan langkah D	25%	KI-02, KI-04, dan barang tak laku sesuai hasil yang diharapkan
Titik Analisis dan latihan E	20%	Penjelasan COUNT dan Kartesius tepat; AB-04 terbukti
Milestone Proyek 10	40%	Lima kueri relasional dengan jalur ERD
Laporan dan bukti Git	15%	Lengkap sesuai checklist

Sinkronisasi Kompetensi

Tabel 10.4 Unit kompetensi yang dilatih di Modul 10

Kode unit	Judul unit	Elemen yang dilatih
J.62DMS00.019.1	Menggunakan Data	Elemen 2: mengakses data lintas tabel sesuai kebutuhan pengguna
J.620100.020.02	Menggunakan SQL	Menggabungkan tabel dengan JOIN

Modul 11: Subquery dan Operasi Himpunan

Cerita Pertemuan Ini

Setelah rapat anggota, bendahara datang dengan daftar pertanyaan yang lebih rumit: “Barang mana yang harganya di atas rata-rata? Siapa anggota yang terdaftar tetapi tidak pernah berbelanja? Barang apa yang kita beli dari pemasok tetapi belum pernah terjual?” Pertanyaan seperti ini adalah pertanyaan di dalam pertanyaan. Hari ini Anda juga akan bertemu jebakan SQL yang paling diam-diam.

A. Tujuan Praktikum

Setelah menyelesaikan modul ini, mahasiswa mampu:

- 1. menulis subquery skalar, subquery daftar dengan **IN**, dan subquery berkorelasi dengan **EXISTS**;
- 2. memakai tabel turunan (*derived table*) di klausa **FROM**;
- 3. menggabungkan hasil kueri dengan **UNION**, **UNION ALL**, **INTERSECT**, dan **EXCEPT**;
- 4. mengenali jebakan **NOT IN** terhadap **NULL** dan memilih bentuk kueri yang benar.

B. Alat dan Bahan

- XAMPP 8.2.12 dan `kopma_<3d>` berisi `kopma_data_lanjutan.sql`.
- Kueri JOIN dari Pertemuan 10 sebagai pembandingan.

C. Dasar Teori

C.1 Jenis Subquery

Subquery adalah kueri di dalam kueri lain [12]. Bentuknya ditentukan oleh apa yang dikembalikan dan hubungannya dengan kueri luar.

Tabel 11.1 Jenis subquery

Jenis	Mengembalikan	Contoh Kopma
Skalar	Satu nilai	Harga rata-rata semua barang
Daftar (IN)	Satu kolom, banyak baris	Daftar id barang yang pernah dibeli dari pemasok
Berkorelasi (EXISTS)	Benar/salah per baris luar	Apakah anggota ini pernah punya nota?
Tabel turunan	Tabel sementara di FROM	Total per nota, lalu dirata-ratakan

C.2 Jebakan NOT IN dan NULL

NOT IN membandingkan nilai dengan setiap anggota daftar. Jika daftar mengandung **NULL**, perbandingan dengan **NULL** bernilai UNKNOWN, sehingga syarat **NOT IN** tidak pernah TRUE (Gambar 11.1). Kolom **penjualan.id_anggota** memang boleh NULL untuk pembeli umum (AB-02), sehingga jebakan ini pasti muncul di Kopma. Kesalahan jenis ini tidak memunculkan galat sama sekali [2].

Jebakan NOT IN ketika subquery mengandung NULL

```
WHERE id_anggota NOT IN
  (SELECT id_anggota FROM penjualan)
-- hasil: 0 baris
```

```
WHERE NOT EXISTS (SELECT 1 FROM
  penjualan p WHERE p.id_anggota = a.id_anggota)
-- hasil: 2 baris (A-0459, A-0466)
```

Daftar subquery berisi {1, 4, NULL, ...}. Untuk anggota 3: 3 <> 1 benar, 3 <> 4 benar, tetapi 3 <> NULL bernilai UNKNOWN, sehingga seluruh syarat NOT IN tidak pernah TRUE. Tidak ada galat; hasilnya salah diam-diam. Gunakan NOT EXISTS, atau saring NULL di dalam subquery.

Gambar 11.1 Jebakan NOT IN ketika subquery mengandung NULL

C.3 Operasi Himpunan

Operasi himpunan menggabungkan **hasil** dua kueri yang kolomnya setara (Gambar 11.2). **UNION** menghapus duplikat, **UNION ALL** mempertahankannya, sedangkan **INTERSECT** dan **EXCEPT** tersedia di MariaDB sejak versi 10.3 [13].

Operasi himpunan pada id_barang yang terjual dan yang dibeli



Gambar 11.2 Operasi himpunan UNION, INTERSECT, dan EXCEPT

D. Langkah Percobaan

D.1 Subquery Skalar

1. Tampilkan barang yang harganya di atas rata-rata semua barang. Hasil yang diharapkan: enam barang, dipimpin Kopi Susu Kaleng.

```
SELECT kode_barang, nama_barang, harga_jual_barang
FROM barang
WHERE harga_jual_barang > (SELECT AVG(harga_jual_barang) FROM barang)
ORDER BY harga_jual_barang DESC;
```

D.2 NOT IN dibandingkan NOT EXISTS

2. Cari anggota yang tidak pernah berbelanja dengan dua cara, lalu bandingkan hasilnya.

```
-- Cara 1: NOT IN (hasil: 0 baris)
SELECT no_anggota, nama_anggota FROM anggota
WHERE id_anggota NOT IN (SELECT id_anggota FROM penjualan);

-- Cara 2: NOT EXISTS (hasil: A-0459 dan A-0466)
SELECT a.no_anggota, a.nama_anggota, a.status_anggota FROM anggota a
WHERE NOT EXISTS (SELECT 1 FROM penjualan p WHERE p.id_anggota =
a.id_anggota);
```

Titik Analisis: 1

Perbaiki Cara 1 tanpa mengganti **NOT IN**, sehingga hasilnya sama dengan Cara 2. Lalu jelaskan mengapa buku ini menyarankan **NOT EXISTS** sebagai kebiasaan.

D.3 Tabel Turunan

3. Hitung rata-rata total per nota. Total per nota dihitung dulu di tabel turunan, baru dirata-ratakan.

```
SELECT ROUND(AVG(t.total), 0) AS rata_nota
FROM (SELECT d.id_penjualan, SUM(d.qty_detail_penjualan *
d.harga_satuan_detail_penjualan) AS total
FROM detail_penjualan d GROUP BY d.id_penjualan) AS t;
```

Titik Analisis: 2

Mengapa $\text{AVG}(\text{d.qty_detail_penjualan} * \text{d.harga_satuan_detail_penjualan})$ langsung pada tabel detail tidak menghasilkan rata-rata per nota? Apa yang sebenarnya dirata-ratakan?

D.4 Operasi Himpunan

4. Tampilkan barang yang pernah dibeli dari pemasok **dan** pernah terjual (**INTERSECT**), lalu barang yang dibeli tetapi belum pernah terjual (**EXCEPT**). Hasil **EXCEPT** yang diharapkan: **id_barang 12** (Wafer Cokelat).

```
SELECT id_barang FROM detail_pembelian
INTERSECT
SELECT id_barang FROM detail_penjualan;

SELECT id_barang FROM detail_pembelian
EXCEPT
SELECT id_barang FROM detail_penjualan;
```

5. Simpan semua kueri di **p11_subquery_kopma_<NIM>.sql** dan commit dengan pesan **p11: subquery kopma**.

Titik Analisis: 3

Hasil **EXCEPT** sama dengan hasil **LEFT JOIN** di Pertemuan 10 (barang yang belum terjual). Kapan Anda memilih operasi himpunan, dan kapan **JOIN**?

E. Latihan dan Modifikasi

1. Tulis ulang kueri INTERSECT di atas memakai **IN** dan memakai **EXISTS**. Pastikan ketiganya menghasilkan baris yang sama.
2. Tampilkan daftar kontak gabungan anggota dan pemasok (nama, telepon/HP, jenis) dengan **UNION ALL**,urut berdasarkan nama.

F. Tugas Mandiri: Milestone Proyek 11

- Tulis **p11_subquery_<NIM>.sql** berisi minimal 5 kueri analitis proyek: satu subquery skalar, satu **EXISTS/NOT EXISTS**, satu tabel turunan, dan dua operasi himpunan.
- Uji jebakan **NOT IN** pada data proyek Anda. Jika tidak ada kolom yang boleh NULL, jelaskan mengapa proyek Anda aman atau tambahkan satu kasus uji.
- Untuk satu kebutuhan yang sama, bandingkan versi subquery dan versi JOIN di laporan.

Troubleshooting Umum

Masalah 1: ERROR 1242 Subquery returns more than 1 row

Cara membaca. Subquery dipakai sebagai nilai tunggal, tetapi mengembalikan banyak baris. **Perbaikan:** gunakan **IN/EXISTS**, atau pastikan subquery memang menghasilkan satu nilai (agregat atau kunci unik).

Masalah 2: ERROR 1064 ... near " pada tabel turunan

Cara membaca. MariaDB mewajibkan setiap tabel turunan diberi alias. Jika alias terlupa, server tidak memberi pesan khusus, tetapi galat sintaks umum 1064 dengan keterangan **near ''**, artinya bagian yang bermasalah berada di akhir perintah. **Perbaikan:** tambahkan alias setelah kurung penutup tabel turunan, misalnya **AS t**.

Masalah 3: ERROR 1222 The used SELECT statements have a different number of columns

Perbaikan. Samakan jumlah dan urutan kolom pada kedua sisi operasi himpunan. Tipe data yang berbeda juga dapat menghasilkan perbandingan tak terduga.

Checklist Laporan Pertemuan 11

Laporan disimpan sebagai **laporan/p11_laporan_<NIM>.md** dengan struktur baku, lalu di-push sebelum batas waktu. Pastikan semua **checklist umum** (lihat Pedoman dan Checklist Laporan Praktikum) dan checklist khusus berikut terpenuhi.

Tabel 11.2 Checklist khusus laporan Pertemuan 11

	Jenis	Yang harus ada
☐	Berkas wajib	p11_subquery_<NIM>.sql
☐	Bukti tangkapan layar	Hasil subquery berkorelasi dan operasi himpunan
☐	Analisis wajib	Perbandingan subquery dengan JOIN untuk kebutuhan yang sama

Perhatian: aturan keaslian

Kasus uji jebakan NOT IN pada proyek wajib memakai data Anda sendiri, lengkap dengan tangkapan layar kedua versi kueri.

Rubrik Penilaian Modul

Tabel 11.3 Rubrik penilaian Modul 11

Aspek	Bobot	Indikator
Pelaksanaan langkah D	25%	Hasil subquery dan himpunan sesuai harapan
Titik Analisis dan latihan E	20%	Jebakan NOT IN dijelaskan dan diperbaiki; tiga versi setara
Milestone Proyek 11	40%	Lima kueri analitis dengan variasi yang diminta
Laporan dan bukti Git	15%	Lengkap sesuai checklist

Sinkronisasi Kompetensi

Tabel 11.4 Unit kompetensi yang dilatih di Modul 11

Kode unit	Judul unit	Elemen yang dilatih
J.620100.020.02	Menggunakan SQL	Subquery dan operasi himpunan
J.62DMS00.019.1	Menggunakan Data	Elemen 2: memenuhi kebutuhan akses data yang kompleks

Modul 12: CTE dan Window Function untuk Peringkat dan Tren

Cerita Pertemuan Ini

Pengurus ingin menyusun strategi stok untuk semester depan. Mereka tidak lagi bertanya “berapa”, tetapi “peringkat berapa” dan “naik atau turun”: barang nomor satu di setiap kategori, omzet kumulatif sepanjang bulan, dan hari-hari ketika Kopma tidak mencatat penjualan sama sekali. Kueri bersarang mulai sulit dibaca. Hari ini Anda belajar menulis kueri panjang yang tetap jelas.

A. Tujuan Praktikum

Setelah menyelesaikan modul ini, mahasiswa mampu:

1. menyusun kueri bertahap yang mudah dibaca dengan *Common Table Expression* (**WITH**);
2. membangkitkan deret tanggal dengan CTE rekursif untuk menemukan data yang tidak ada;
3. menghitung peringkat dengan **ROW_NUMBER**, **RANK**, dan **DENSE_RANK**;
4. menghitung total berjalan dan perubahan antarperiode dengan **SUM() OVER** dan **LAG**.

B. Alat dan Bahan

- XAMPP 8.2.12 (MariaDB 10.4 mendukung CTE sejak 10.2.1 dan window function sejak 10.2.0) [13].
- `kopma_<3d>` berisi `kopma_data_lanjutan.sql`.

C. Dasar Teori

C.1 CTE: Memberi Nama pada Langkah

CTE memberi nama pada hasil antara, sehingga kueri panjang dapat dibaca dari atas ke bawah seperti langkah-langkah [12]. Bagi pemula, kebiasaan ini juga membantu mengenali pola kueri yang berulang [4]. CTE **rekursif** memanggil dirinya sendiri dan cocok untuk membangkitkan deret, misalnya daftar tanggal.

C.2 Window Function

Berbeda dengan **GROUP BY**, window function menghitung nilai untuk setiap baris **tanpa** meringkas baris (Gambar 12.1). Klausula **OVER** menentukan “jendela” baris: **PARTITION BY** membagi jendela, dan **ORDER BY** menentukan urutan di dalamnya.

Window function: menghitung tanpa meringkas baris

kategori_barang	nama_barang	omzet	RANK()
ATK	Buku Tulis 58	117.000	1
ATK	Pulpen Gel 0.5	89.000	2
ATK	Pensil 2B	75.000	3
makanan	Keripik Singkong	189.000	1
makanan	Roti Cokelat	135.000	2
minuman	Kopi Susu Kaleng	184.000	1
minuman	Air Mineral 600	104.000	2

PARTITION BY kategori_barang memulai ulang peringkat di setiap kategori; jumlah baris tetap.

Gambar 12.1 Peringkat omzet per kategori dengan window function

Tabel 12.1 Fungsi peringkat saat ada nilai kembar

Fungsi	Nilai 100, 90, 90, 80	Kapan dipakai
ROW_NUMBER()	1, 2, 3, 4	Memilih tepat satu baris per kelompok
RANK()	1, 2, 2, 4	Peringkat lomba (ada lompatan)
DENSE_RANK()	1, 2, 2, 3	Peringkat tanpa lompatan

D. Langkah Percobaan

D.1 CTE dan Peringkat per Kategori

1. Hitung omzet per barang di CTE, lalu beri peringkat per kategori. Hasil yang diharapkan: peringkat 1 ATK = Buku Tulis 58, makanan = Keripik Singkong, minuman = Kopi Susu Kaleng.

```
WITH omzet_barang AS (  
    SELECT b.kategori_barang, b.nama_barang, SUM(d.qty_detail_penjualan *  
    d.harga_satuan_detail_penjualan) AS omzet  
    FROM detail_penjualan d JOIN barang b ON b.id_barang = d.id_barang  
    GROUP BY b.id_barang, b.kategori_barang, b.nama_barang)  
SELECT kategori_barang, nama_barang, omzet,  
    RANK() OVER (PARTITION BY kategori_barang ORDER BY omzet DESC)  
AS peringkat  
FROM omzet_barang  
ORDER BY kategori_barang, peringkat;
```

Titik Analisis: 1

Ketua hanya ingin **satu** barang teratas per kategori. Mengapa **WHERE peringkat = 1** tidak dapat ditulis langsung di kueri yang sama? Tulis versi yang benar dengan CTE kedua.

D.2 CTE Rekursif: Hari Tanpa Penjualan

2. Bangkitkan semua tanggal 1–14 Oktober, lalu cari tanggal yang tidak memiliki nota (Gambar 12.2).

```

WITH RECURSIVE kalender AS (
    SELECT DATE('2026-10-01') AS tgl_kalender
    UNION ALL
    SELECT tgl_kalender + INTERVAL 1 DAY FROM kalender WHERE tgl_kalender
    < '2026-10-14')
SELECT k.tgl_kalender, COUNT(p.id_penjualan) AS nota
FROM kalender k
LEFT JOIN penjualan p ON DATE(p.tgl_penjualan) = k.tgl_kalender
GROUP BY k.tgl_kalender
HAVING nota = 0;

```

```

Shell XAMPP - hari tanpa penjualan

MariaDB [kopma_123]> WITH RECURSIVE kalender AS (
-> SELECT DATE('2026-10-01') AS tgl_kalender
-> UNION ALL SELECT tgl_kalender + INTERVAL 1 DAY FROM kalender
-> WHERE tgl_kalender < '2026-10-14')
-> SELECT k.tgl_kalender, COUNT(p.id_penjualan) AS nota FROM kalender k
-> LEFT JOIN penjualan p ON DATE(p.tgl_penjualan) = k.tgl_kalender
-> GROUP BY k.tgl_kalender HAVING nota = 0;

+-----+-----+
| tgl_kalender | nota |
+-----+-----+
| 2026-10-01   | 0    |
| 2026-10-02   | 0    |
| 2026-10-03   | 0    |
| 2026-10-04   | 0    |
| 2026-10-07   | 0    |
| 2026-10-09   | 0    |
| 2026-10-11   | 0    |
+-----+-----+

```

Hasil nyata dari data kopma_data_lanjutan.sql.

Gambar 12.2 CTE rekursif menemukan hari tanpa penjualan

Titik Analisis: 2

Tanggal 4 dan 11 Oktober adalah hari Minggu ketika Kopma tutup. Bagaimana Anda membedakan “tutup” dari “buka tetapi tidak ada penjualan”? Petunjuk: `DAYOFWEEK()`. Mengapa kueri tanpa CTE rekursif tidak akan pernah menampilkan tanggal-tanggal ini?

D.3 Total Berjalan dan Perubahan Harian

3. Hitung omzet harian 1-10 Oktober, omzet kumulatif, dan selisih terhadap hari penjualan sebelumnya.

```

WITH harian AS (
    SELECT DATE(p.tgl_penjualan) AS tanggal,
           SUM(d.qty_detail_penjualan * d.harga_satuan_detail_penjualan)
    AS omzet
    FROM penjualan p JOIN detail_penjualan d ON d.id_penjualan =
    p.id_penjualan
    WHERE p.tgl_penjualan >= '2026-10-01' AND p.tgl_penjualan < '2026-10-
    11'
    GROUP BY DATE(p.tgl_penjualan))
SELECT tanggal, omzet,
       SUM(omzet) OVER (ORDER BY tanggal) AS kumulatif,
       omzet - LAG(omzet) OVER (ORDER BY tanggal) AS selisih
FROM harian;

```

- Hasil yang diharapkan: empat hari penjualan dengan kumulatif akhir Rp64.000. Simpan semua kueri di `p12_cte_window_kopma_<NIM>.sql` dan commit dengan pesan `p12: cte window kopma`.

Titik Analisis: 3

Nilai `selisih` pada baris pertama adalah NULL. Mengapa? Dan apakah “selisih terhadap hari sebelumnya” di sini benar-benar hari sebelumnya menurut kalender? Gabungkan dengan hasil D.2 untuk menjawabnya.

E. Latihan dan Modifikasi

- Pilih tepat satu nota dengan belanja terbesar untuk setiap anggota memakai `ROW_NUMBER()`.
- Hitung pangsa omzet setiap kategori terhadap total bulanan dalam persen dengan `SUM() OVER (PARTITION BY bulan)`.

F. Tugas Mandiri: Milestone Proyek 12

- Tulis `p12_cte_window_<NIM>.sql` berisi minimal 4 laporan: satu peringkat per kelompok, satu total berjalan, satu perubahan antarperiode dengan `LAG` atau `LEAD`, dan satu CTE rekursif.
- Setiap laporan ditulis dengan CTE bernama yang menjelaskan maknanya (bukan `t1, t2`).
- Jelaskan di laporan pilihan antara `ROW_NUMBER`, `RANK`, dan `DENSE_RANK` untuk kasus Anda.

Troubleshooting Umum

Masalah 1: ERROR 1054 saat menyaring hasil window function

Cara membaca. Window function dievaluasi setelah `WHERE`, sehingga aliasnya belum dikenal di `WHERE`. **Perbaikan:** bungkus kueri dengan CTE atau tabel turunan, lalu saring di kueri luar.

Masalah 2: CTE rekursif tidak berhenti

Cara membaca. Server menghentikan rekursi yang melampaui batas `max_recursive_iterations` dan melaporkannya. **Perbaikan:** pastikan bagian rekursif memiliki syarat berhenti yang pasti tercapai, misalnya `WHERE tgl_kalender < '2026-10-14'`.

Masalah 3: Total berjalan melompat

Cara membaca. Nilai kembar pada kolom **ORDER BY** di dalam **OVER** diperlakukan sebagai satu kelompok. **Perbaikan:** tambahkan kolom pembeda ke **ORDER BY**, misalnya **ORDER BY tgl_penjualan, id_penjualan**.

Checklist Laporan Pertemuan 12

Laporan disimpan sebagai **laporan/p12_laporan_<NIM>.md** dengan struktur baku, lalu di-push sebelum batas waktu. Pastikan semua **checklist umum** (lihat Pedoman dan Checklist Laporan Praktikum) dan checklist khusus berikut terpenuhi.

Tabel 12.2 Checklist khusus laporan Pertemuan 12

	Jenis	Yang harus ada
☐	Berkas wajib	p12_cte_window_<NIM>.sql
☐	Bukti tangkapan layar	Hasil laporan peringkat dan tren
☐	Analisis wajib	Penjelasan PARTITION BY dan ORDER BY pada window function yang dipakai

Perhatian: aturan keaslian

Nama CTE, kasus peringkat, dan periode tren proyek harus mencerminkan KI proyek Anda.

Rubrik Penilaian Modul

Tabel 12.3 Rubrik penilaian Modul 12

Aspek	Bobot	Indikator
Pelaksanaan langkah D	25%	Peringkat, kalender, dan kumulatif sesuai hasil yang diharapkan
Titik Analisis dan latihan E	20%	Penyaringan hasil window dan makna LAG dijelaskan tepat
Milestone Proyek 12	40%	Empat laporan dengan CTE bernama dan pilihan fungsi beralasan
Laporan dan bukti Git	15%	Lengkap sesuai checklist

Sinkronisasi Kompetensi

Tabel 12.4 Unit kompetensi yang dilatih di Modul 12

Kode unit	Judul unit	Elemen yang dilatih
J.620100.020.02	Menggunakan SQL	CTE, CTE rekursif, dan window function
J.62DMS00.006.1	Merancang Basis Data	KUK 4.3: laporan keluaran dirancang berdasarkan kebutuhan

Modul 13: View dan Perancangan Kueri per Peran

Cerita Pertemuan Ini

Seorang anggota mengeluh: nomor HP-nya dihubungi orang tak dikenal yang mengaku “dapat dari kasir Kopma”. Ketua langsung bertanya, “Mengapa kasir bisa melihat nomor HP anggota?” Selama ini setiap orang memakai akses yang sama ke semua tabel. Hari ini Anda membuat “jendela” yang berbeda untuk setiap peran, sehingga setiap orang hanya melihat data yang ia butuhkan.

A. Tujuan Praktikum

Setelah menyelesaikan modul ini, mahasiswa mampu:

1. membuat view untuk menyederhanakan kueri laporan yang sering dipakai;
2. merancang view per peran yang menyembunyikan kolom data pribadi;
3. memberi hak akses melalui role ke view, bukan ke tabel dasar;
4. menjaga perubahan data lewat view dengan **WITH CHECK OPTION**.

B. Alat dan Bahan

- XAMPP 8.2.12, akun **root** (untuk membuat role dan akun) dan akun **mhs_<3d>** (untuk membuat view).
- Kamus data Pertemuan 2 (kolom penanggung jawab dan penanda data pribadi).
- **kopma_<3d>** berisi **kopma_data_lanjutan.sql**.

C. Dasar Teori

C.1 View sebagai Jendela

View adalah kueri bernama yang diperlakukan seperti tabel [12]. View tidak menyimpan data sendiri; setiap kali dibaca, kuerinya dijalankan ulang. View dipakai untuk dua tujuan: menyederhanakan kueri yang rumit, dan membatasi apa yang dapat dilihat pengguna (Gambar 13.1).

View sebagai jendela: setiap peran melihat bagian data yang berbeda



Hak akses diberikan ke view melalui role `r_kasir_<3d>`, bukan ke tabel dasar.

Gambar 13.1 View per peran menyembunyikan kolom data pribadi

Unit *Merancang Basis Data* memasukkan perancangan kueri, tampilan, dan laporan berdasarkan kebutuhan sebagai satu elemen tersendiri [1]. Pembatasan kolom seperti nomor HP juga sejalan dengan prinsip meminimalkan akses ke data pribadi dalam UU PDP [17].

C.2 Role dan DEFINER

Role adalah kumpulan hak akses yang diberi nama, lalu diberikan kepada akun. Jika seorang kasir berganti, cukup cabut role dari akun lama dan berikan ke akun baru. Setiap view juga menyimpan atribut **DEFINER**, yaitu akun pembuatnya. View dijalankan dengan hak DEFINER, sehingga kasir dapat membaca view tanpa punya hak atas tabel dasarnya [13].

Tabel 13.1 Rancangan view per peran Kopma

Peran	View	Isi	Yang disembunyikan
Kasir	v_kasir_anggota	no_anggota, nim_anggota, nama_anggota, status_anggota	no_hp_anggota, tgl_daftar_anggota
Kasir	v_kasir_barang	kode_barang, nama_barang, harga_jual_barang, stok_barang	stok_min_barang
Gudang	v_barang_menis	barang dengan stok_barang < stok_min_barang	harga_jual_barang
Ketua	v_omzet_harian	omzet dan jumlah nota per hari	rincian nota

D. Langkah Percobaan

D.1 Membuat View (sebagai mhs_<3d>)

1. Masuk sebagai **mhs_<3d>** dan buat view kasir serta view gudang.

```
CREATE OR REPLACE VIEW v_kasir_anggota AS
  SELECT id_anggota, no_anggota, nim_anggota, nama_anggota,
  status_anggota FROM anggota;

CREATE OR REPLACE VIEW v_barang_menipis AS
  SELECT kode_barang, nama_barang, stok_barang, stok_min_barang
  FROM barang WHERE stok_barang < stok_min_barang
  WITH CHECK OPTION;
```

2. Buat **v_kasir_barang** dan **v_omzet_harian** sesuai tabel rancangan. **v_omzet_harian** memakai kueri KI-01 dari Pertemuan 9.
3. Periksa DEFINER dengan **SHOW CREATE VIEW v_kasir_anggota;**. Catat akun yang tertulis.

Titik Analisis: 1

DEFINER view Anda adalah **mhs_<3d>**. Di Buku 2, basis data ini akan dipindah ke server baru. Apa yang terjadi pada view ini jika akun **mhs_<3d>** belum dibuat di server baru? (Buku 2 Modul 1 membahasnya sebagai ERROR 1449.)

D.2 Membuat Role dan Akun Kasir (sebagai root)

4. Masuk sebagai **root**. Buat role, beri hak hanya ke view, lalu buat akun kasir dan jadikan role tersebut bawaannya.

```
CREATE ROLE r_kasir_123;
GRANT SELECT ON kopma_123.v_kasir_anggota TO r_kasir_123;
GRANT SELECT ON kopma_123.v_kasir_barang TO r_kasir_123;

CREATE USER 'kasir_123'@'localhost' IDENTIFIED BY '<password_kasir>';
GRANT r_kasir_123 TO 'kasir_123'@'localhost';
SET DEFAULT ROLE r_kasir_123 FOR 'kasir_123'@'localhost';
```

D.3 Membuktikan Pembatasan

5. Masuk sebagai **kasir_<3d>**, lalu coba membaca tabel dasar dan view. Hasil yang diharapkan tampak pada Gambar 13.2.

```
SELECT CURRENT_ROLE();
SELECT no_hp_anggota FROM anggota LIMIT 1;           -- ditolak ERROR
1142
SELECT no_hp_anggota FROM v_kasir_anggota LIMIT 1;   -- ditolak ERROR
1054
SELECT no_anggota, nama_anggota, status_anggota FROM v_kasir_anggota
LIMIT 2;
```

```

Shell XAMPP - akses kasir lewat view

C:\xampp> mysql -u kasir_123 -p kopma_123
MariaDB [kopma_123]> SELECT CURRENT_ROLE();
+-----+
| r_kasir_123 |
+-----+
MariaDB [kopma_123]> SELECT no_hp_anggota FROM anggota LIMIT 1;
ERROR 1142 (42000): SELECT command denied to user 'kasir_123'@'localhost'
for table 'kopma_123'.'anggota'
MariaDB [kopma_123]> SELECT no_anggota, nama_anggota, status_anggota
-> FROM v_kasir_anggota LIMIT 2;
+-----+-----+-----+
| A-0457 | Budi S. | aktif |
| A-0458 | Citra A. | aktif |
+-----+-----+-----+

```

Ilustrasi disederhanakan; tampilan sebenarnya dapat sedikit berbeda.

Gambar 13.2 Kasir ditolak mengakses tabel dasar tetapi dapat membaca view

Titik Analisis: 2

Dua galat berbeda muncul: 1142 untuk tabel dan 1054 untuk kolom `no_hp_anggota` di view. Apa arti masing-masing? Mana yang membuktikan bahwa kasir tidak punya hak, dan mana yang membuktikan bahwa kolom memang tidak ada di jendela kasir?

D.4 WITH CHECK OPTION

- Sebagai `mhs_<3d>`, coba ubah stok Roti Cokelat menjadi 50 melalui `v_barang_menipis`. Perubahan itu akan membuat baris keluar dari view, sehingga ditolak dengan `ERROR 1369 CHECK OPTION failed`.
- Simpan skrip view di `p13_view_kopma_<NIM>.sql` dan skrip role di `p13_role_kopma_<NIM>.sql` (tanpa password asli). Commit dengan pesan `p13: view per peran_petugas kopma`.

Titik Analisis: 3

Mengapa `WITH CHECK OPTION` penting untuk petugas gudang yang hanya boleh mengurus barang menipis? Apa risikonya jika opsi itu tidak ada?

E. Latihan dan Modifikasi

- Buat role `r_ketua_<3d>` yang hanya dapat membaca `v_omzet_harian` dan `v_barang_menipis`, lalu buktikan dengan akun `ketua_<3d>`.
- Seorang kasir berhenti bekerja. Cabut role dari akunnya dengan `REVOKE` dan kunci akunnya dengan `ALTER USER ... ACCOUNT LOCK`. Buktikan bahwa akun itu tidak dapat masuk.

F. Tugas Mandiri: Milestone Proyek 13

- Tulis `p13_view_<NIM>.sql` berisi minimal satu view per peran proyek (minimal tiga peran), dan `p13_role_<NIM>.sql` berisi role serta akunnya.
- Setiap kolom data pribadi di kamus data proyek hanya muncul di view peran yang berhak; buat matriks peran × view di laporan.
- Minimal satu view memakai `WITH CHECK OPTION`, dan buktikan penolakannya.

- Buktikan untuk setiap peran: akses ke tabel dasar ditolak, akses ke view berhasil.

Catatan: hubungan ke Buku 2

Role dan view ini menjadi fondasi Buku 2 Modul 9 (keamanan data dan PDP), sedangkan DEFINER-nya diuji saat migrasi di Buku 2 Modul 1.

Troubleshooting Umum

Masalah 1: ERROR 1142 ... command denied

Cara membaca. Pesan menyebut perintah yang ditolak (**SELECT**), akun ('kasir_123'@'localhost'), dan tabel lengkap dengan nama basis datanya ('kopma_123.anggota'). ****Perbaikan:**** jika penolakan ini memang dirancang, galat ini adalah bukti keberhasilan. Jika tidak, periksa **SHOW GRANTS FOR CURRENT_USER;** dan pastikan role aktif dengan **SELECT CURRENT_ROLE();**.

Masalah 2: Role sudah diberikan tetapi hak tidak berlaku

Cara membaca. **CURRENT_ROLE()** bernilai NULL, artinya role belum aktif di sesi. **Perbaikan:** jalankan **SET DEFAULT ROLE** untuk akun tersebut, lalu keluar dan masuk ulang, atau aktifkan sementara dengan **SET ROLE r_kasir_<3d>;**.

Masalah 3: ERROR 1227 saat membuat role sebagai mhs_<3d>

Perbaikan. Membuat role dan akun memerlukan hak administrasi global. Lakukan langkah D.2 sebagai **root**, sesuai pembagian tanggung jawab di laporan.

Checklist Laporan Pertemuan 13

Laporan disimpan sebagai **laporan/p13_laporan_<NIM>.md** dengan struktur baku, lalu di-push sebelum batas waktu. Pastikan semua **checklist umum** (lihat Pedoman dan Checklist Laporan Praktikum) dan checklist khusus berikut terpenuhi.

Tabel 13.2 Checklist khusus laporan Pertemuan 13

	Jenis	Yang harus ada
☐	Berkas wajib	p13_view_<NIM>.sql (view per peran) dan p13_role_<NIM>.sql (role dan akun, tanpa password asli)
☐	Bukti tangkapan layar	Akses view berhasil dan akses tabel dasar ditolak untuk setiap peran
☐	Analisis wajib	Matriks peran × view; alasan pembatasan kolom data pribadi

Perhatian: aturan keaslian

Nama role dan akun wajib memuat **<3d>**. Matriks peran × view harus mengikuti peran dan data pribadi di kamus data proyek Anda.

Rubrik Penilaian Modul

Tabel 13.3 Rubrik penilaian Modul 13

Aspek	Bobot	Indikator
Pelaksanaan langkah D	25%	View, role, dan pembatasan terbukti; CHECK OPTION menolak
Titik Analisis dan latihan E	20%	Makna galat 1142, 1054, 1369 dan DEFINER tepat
Milestone Proyek 13	40%	View per peran, data pribadi terlindungi, bukti akses per peran
Laporan dan bukti Git	15%	Lengkap sesuai checklist

Sinkronisasi Kompetensi

Tabel 13.4 Unit kompetensi yang dilatih di Modul 13

Kode unit	Judul unit	Elemen yang dilatih
J.62DMS00.006.1	Merancang Basis Data	Elemen 4: merancang query, tampilan, dan laporan (KUK 4.2, 4.3)
J.62DMS00.019.1	Menggunakan Data	Elemen 3: penggunaan data dimonitor sesuai hak akses (KUK 3.1)

Modul 14: Transaksi dan Integritas Data

Cerita Pertemuan Ini

Siang itu listrik kampus padam tepat saat Rina mencatat nota. Setelah listrik menyala, nota tercatat tetapi detail barangnya hanya separuh, dan stok sudah terpotong. Tidak ada yang tahu berapa yang benar-benar terjual. Di Pertemuan 3 dan 6 Anda mencatat dua “utang desain”: nota wajib punya minimal satu barang, dan penjualan tidak boleh melebihi stok. Hari ini keduanya dilunasi dengan transaksi.

A. Tujuan Praktikum

Setelah menyelesaikan modul ini, mahasiswa mampu:

1. menjelaskan sifat ACID dan alasan satu penjualan harus menjadi satu transaksi;
2. menulis transaksi dengan `START TRANSACTION`, `COMMIT`, `ROLLBACK`, dan `SAVEPOINT`;
3. memeriksa kondisi bisnis di dalam transaksi (stok cukup, minimal satu detail) dan membatalkan bila gagal;
4. mengenali perintah yang menutup transaksi secara otomatis.

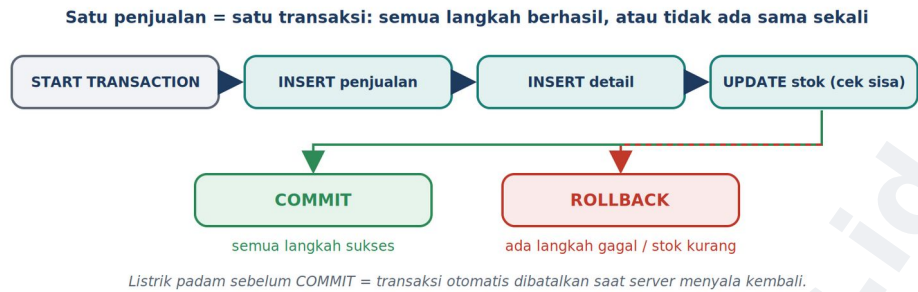
B. Alat dan Bahan

- XAMPP 8.2.12, `kopma_<3d>` berisi `kopma_data_lanjutan.sql`.
- Dua jendela klien `mysql` untuk mengamati sesi berbeda.
- Catatan utang desain dari Pertemuan 5–6.

C. Dasar Teori

C.1 Transaksi dan ACID

Transaksi adalah rangkaian operasi yang diperlakukan sebagai satu kesatuan. Sifatnya diringkas dengan **ACID**: *atomicity* (semua atau tidak sama sekali), *consistency* (aturan tetap terpenuhi), *isolation* (transaksi tidak saling mengganggu), dan *durability* (yang sudah di-commit bertahan) [11], [12]. Satu penjualan Kopma terdiri atas empat langkah yang harus berhasil bersama (Gambar 14.1).



Gambar 14.1 Satu penjualan sebagai satu transaksi

C.2 Autocommit dan Commit Implisit

Secara bawaan, setiap perintah langsung di-commit (*autocommit*). **START TRANSACTION** menahan commit sampai **COMMIT** atau **ROLLBACK**. Perintah DDL seperti **CREATE TABLE** memicu **commit implisit**: transaksi yang sedang berjalan langsung disimpan dan tidak lagi dapat dibatalkan [13].

Tabel 14.1 Perilaku penting yang diuji di modul ini

Situasi	Yang terjadi	Akibat
Satu perintah gagal karena constraint	Hanya perintah itu yang ditolak; transaksi tetap terbuka	Harus diputuskan sendiri: ROLLBACK atau lanjut
Klien terputus sebelum COMMIT	Server membatalkan transaksi	Tidak ada data setengah jadi
CREATE TABLE di tengah transaksi	Commit implisit	Perubahan sebelumnya tidak dapat di-rollback
ROLLBACK TO SAVEPOINT	Membatalkan sebagian	Perubahan sebelum savepoint tetap ada

D. Langkah Percobaan

D.1 Transaksi Penjualan dengan Pemeriksaan Stok

1. Catat stok Roti Cokelat (**MKN-001**), lalu jalankan transaksi penjualan 10 roti. Kunci pemeriksaannya: **UPDATE** hanya mengurangi stok jika sisa mencukupi, lalu **ROW_COUNT ()** memberi tahu apakah baris itu benar-benar berubah.

```

START TRANSACTION;
INSERT INTO penjualan (no_nota_penjualan, tgl_penjualan, id_petugas,
id_anggota, bayar_penjualan)
VALUES ('PJ-2611-0900', '2026-11-01 09:00:00', 3, 4, 50000);
SET @id := LAST_INSERT_ID();
INSERT INTO detail_penjualan (id_penjualan, id_barang,
qty_detail_penjualan, harga_satuan_detail_penjualan)
SELECT @id, id_barang, 10, harga_jual_barang FROM barang WHERE
kode_barang = 'MKN-001';
UPDATE barang SET stok_barang = stok_barang - 10
WHERE kode_barang = 'MKN-001' AND stok_barang >= 10;
SELECT ROW_COUNT() AS baris_stok_berubah; -- 0 berarti stok tidak
cukup
ROLLBACK; -- karena hasilnya 0

```

2. Pastikan nota PJ-2611-0900 tidak ada di tabel dan stok tidak berubah. Ulangi dengan qty 2; kali ini `ROW_COUNT()` bernilai 1, jadi akhiri dengan `COMMIT`.

Titik Analisis: 1

Mengapa pemeriksaan stok ditulis sebagai syarat `AND stok_barang >= 10` di dalam `UPDATE`, bukan dengan `SELECT stok_barang` terlebih dahulu lalu dibandingkan? Pikirkan dua kasir yang menjual barang terakhir pada detik yang sama. (Buku 2 Modul 5 mengujinya secara nyata.)

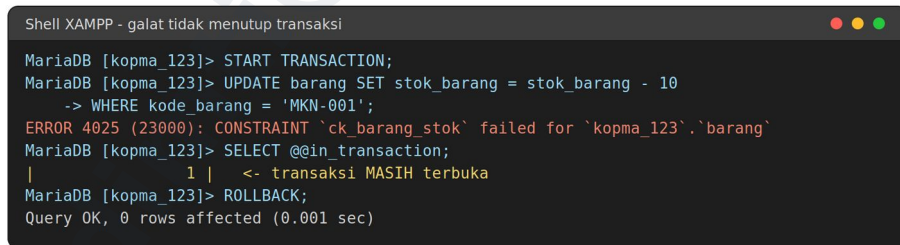
D.2 Galat Tidak Menutup Transaksi

3. Mulai transaksi, lalu kurangi stok Roti Cokelat 10 **tanpa** syarat stok. Constraint `CHECK` menolak, tetapi periksa status transaksi (Gambar 14.2).

```

START TRANSACTION;
UPDATE barang SET stok_barang = stok_barang - 10 WHERE kode_barang =
'MKN-001'; -- ERROR 4025
SELECT @@in_transaction; --
hasil: 1
ROLLBACK;

```



```

Shell XAMPP - galat tidak menutup transaksi

MariaDB [kopma_123]> START TRANSACTION;
MariaDB [kopma_123]> UPDATE barang SET stok_barang = stok_barang - 10
-> WHERE kode_barang = 'MKN-001';
ERROR 4025 (23000): CONSTRAINT `ck_barang_stok` failed for `kopma_123`.`barang`
MariaDB [kopma_123]> SELECT @@in_transaction;
+-----+
| 1 |
+-----+
MariaDB [kopma_123]> ROLLBACK;
Query OK, 0 rows affected (0.001 sec)

```

Ilustrasi disederhanakan; tampilan sebenarnya dapat sedikit berbeda.

Gambar 14.2 Galat constraint tidak menutup transaksi

Titik Analisis: 2

Jika setelah galat itu pengguna tetap mengetik `COMMIT`, apa yang tersimpan? Tuliskan aturan kerja untuk kasir (atau aplikasi kasir) agar transaksi yang pernah menerima galat selalu diakhiri dengan benar.

D.3 Mensimulasikan Listrik Padam

4. Buka jendela klien pertama, jalankan **START TRANSACTION**, sisipkan satu nota dan satu detail, lalu **tutup jendela** tanpa **COMMIT**.
5. Di jendela kedua, periksa apakah nota tersebut ada. Hasil yang diharapkan: tidak ada. Bandingkan dengan cerita pembuka modul ini.

D.4 Commit Implisit dan Savepoint

6. Mulai transaksi, sisipkan satu nota uji, lalu jalankan **CREATE TABLE uji_commit (x INT);** dan **ROLLBACK;**. Periksa: nota uji ternyata **tetap ada**. Hapus nota uji dan tabel **uji_commit** setelahnya.
7. Gunakan **SAVEPOINT**: tambah stok pulpen 1, buat savepoint, tambah lagi 100, lalu **ROLLBACK TO SAVEPOINT** dan **COMMIT**. Stok akhir hanya bertambah 1.
8. Simpan semua skenario di **p14_transaksi_kopma_<NIM>.sql**, commit dengan pesan **p14: transaksi kopma**.

Titik Analisis: 3

Kembali ke utang desain Pertemuan 3: “nota wajib memiliki minimal satu barang”. Tunjukkan bagaimana transaksi di D.1 menjamin aturan ini, padahal tidak ada constraint yang dapat menjaganya.

E. Latihan dan Modifikasi

1. Tulis transaksi penerimaan barang dari pemasok: ubah status pembelian menjadi **diterima** dan tambahkan stok semua barangnya sekaligus. Batalkan jika status pembelian bukan **dipesan**.
2. Tulis transaksi nota tiga barang dengan aturan batas item **P + 2** (Modul 2). Jika jumlah baris detail melebihi batas, transaksi dibatalkan.

F. Tugas Mandiri: Milestone Proyek 14

- Tulis **p14_transaksi_<NIM>.sql** untuk transaksi bisnis utama proyek, lengkap dengan pemeriksaan kondisi dan keputusan **COMMIT/ROLLBACK**.
- Minimal dua skenario: satu berakhir **COMMIT**, satu berakhir **ROLLBACK** karena melanggar aturan bisnis.
- Lunasi semua utang desain yang dicatat sejak Pertemuan 5–6; untuk setiap utang, tunjukkan bagian transaksi yang menjaganya.
- Petakan setiap langkah transaksi ke AB-xx.

Troubleshooting Umum

Masalah 1: Perubahan hilang padahal tidak ada galat

Cara membaca. Sesi ditutup sebelum **COMMIT**, sehingga server membatalkan transaksi. **Perbaikan:** biasakan menutup setiap transaksi secara eksplisit, dan periksa **@@in_transaction** sebelum menutup klien.

Masalah 2: ROLLBACK tidak membatalkan apa pun

Cara membaca. Ada perintah DDL di tengah transaksi, atau **START TRANSACTION** terlupa sehingga *autocommit* aktif. **Perbaikan:** pisahkan DDL dari transaksi data, dan selalu mulai dengan **START TRANSACTION**.

Masalah 3: ERROR 1205 Lock wait timeout exceeded

Cara membaca. Sesi lain memegang kunci baris yang sama dan belum mengakhiri transaksinya, biasanya jendela klien lain yang terlupa. **Perbaikan:** akhiri transaksi di jendela lain. Penguncian dibahas mendalam di Buku 2 Modul 5.

Checklist Laporan Pertemuan 14

Laporan disimpan sebagai **laporan/p14_laporan_<NIM>.md** dengan struktur baku, lalu di-push sebelum batas waktu. Pastikan semua **checklist umum** (lihat Pedoman dan Checklist Laporan Praktikum) dan checklist khusus berikut terpenuhi.

Tabel 14.2 Checklist khusus laporan Pertemuan 14

	Jenis	Yang harus ada
☐	Berkas wajib	p14_transaksi_<NIM>.sql
☐	Bukti tangkapan layar	Skenario COMMIT dan ROLLBACK , termasuk transaksi yang dibatalkan karena melanggar aturan
☐	Analisis wajib	Pemetaan transaksi ke AB-xx; alasan batas awal dan akhir transaksi

Perhatian: aturan keaslian

Skenario transaksi proyek wajib memakai aturan bisnis dan parameter P milik Anda. Tangkapan layar harus memperlihatkan hasil **ROW_COUNT()** atau **@@in_transaction** sebelum keputusan **COMMIT/ROLLBACK**.

Rubrik Penilaian Modul

Tabel 14.3 Rubrik penilaian Modul 14

Aspek	Bobot	Indikator
Pelaksanaan langkah D	25%	Empat perilaku transaksi terbukti dengan hasil yang diharapkan
Titik Analisis dan latihan E	20%	Utang desain dijelaskan; transaksi penerimaan dan batas P + 2 benar
Milestone Proyek 14	40%	Transaksi utama dengan COMMIT dan ROLLBACK, utang desain lunas
Laporan dan bukti Git	15%	Lengkap sesuai checklist

Sinkronisasi Kompetensi

Tabel 14.4 Unit kompetensi yang dilatih di Modul 14

Kode unit	Judul unit	Elemen yang dilatih
J.62DMS00.010.1	Membuat Basis Data	Elemen 2: menguji basis data terhadap skenario transaksi
J.62DMS00.006.1	Merancang Basis Data	KUK 3.4: aturan validasi data dikembangkan sesuai kebutuhan

Modul 15: Mengelola Kualitas Data: Validasi dan Pembersihan

Cerita Pertemuan Ini

Panitia penerimaan anggota baru, yang mengumpulkan pendaftaran lewat formulir daring sejak awal semester, akhirnya menyerahkan hasilnya dalam satu spreadsheet. Isinya kacau: NIM bertanda strip, nama huruf kecil semua, prodi ditulis “ilkom”, “IK”, dan “Ilmu Komputer”, nomor HP diawali +62, bahkan ada tanggal 31 September. Constraint akan menolak sebagian data, tetapi bukan itu tujuannya. Hari ini Anda mengukur, memperbaiki, dan memasukkan data itu dengan benar.

A. Tujuan Praktikum

Setelah menyelesaikan modul ini, mahasiswa mampu:

1. menetapkan dimensi dan parameter kualitas data sesuai kebutuhan organisasi;
2. mengukur masalah kualitas data dengan kueri profil;
3. membersihkan data di tabel penampung (*staging*) dengan fungsi string, **REGEXP**, dan validasi tanggal;
4. memasukkan data bersih ke tabel utama tanpa duplikat, lalu mengukur ulang kualitasnya.

B. Alat dan Bahan

- XAMPP 8.2.12, **kopma_<3d>** berisi **kopma_data_lanjutan.sql**.
- Berkas **kopma_impor_kotor.sql** dari dosen: tabel penampung **anggota_impor** berisi 10 baris pendaftaran bermasalah.
- Kamus data Pertemuan 2 sebagai acuan format baku.

C. Dasar Teori

C.1 Dimensi Kualitas Data

Kualitas data diukur dari beberapa **dimensi**. Penelitian pengukuran kualitas data di instansi pemerintah Indonesia memakai kerangka *Total Data Quality Management* dan menilai dimensi seperti kelengkapan dan akurasi [7]. Dimensi yang dipakai di Kopma dirangkum berikut.

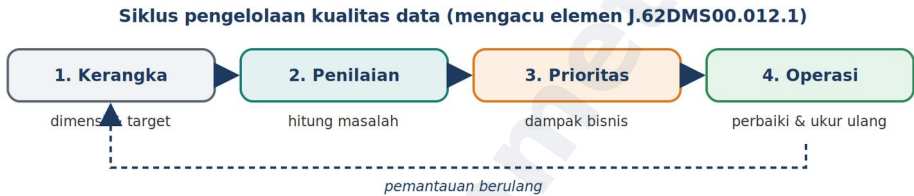
Tabel 15.1 Dimensi kualitas data dan contoh aturannya di Kopma

Dimensi	Pertanyaan	Aturan Kopma
Kelengkapan	Apakah nilai wajib terisi?	Nama tidak boleh kosong

Dimensi	Pertanyaan	Aturan Kopma
Validitas	Apakah nilai sesuai format dan rentang?	NIM 10 digit; HP diawali 08; tanggal benar-benar ada
Keunikan	Apakah ada entitas tercatat ganda?	Satu NIM hanya satu anggota (AB-05)
Konsistensi	Apakah nilai yang sama ditulis sama?	Nama prodi baku sesuai daftar prodi

C.2 Siklus Pengelolaan Kualitas

Unit *Mengelola Kualitas Data* menyusun pekerjaan ini sebagai siklus: membuat kerangka kerja kualitas, menilai kualitas awal, menentukan prioritas perbaikan berdasarkan dampak bisnis, lalu menjalankan operasi perbaikan dan pemantauan (Gambar 15.1) [1].



Gambar 15.1 Siklus pengelolaan kualitas data

C.3 Tabel Penampung

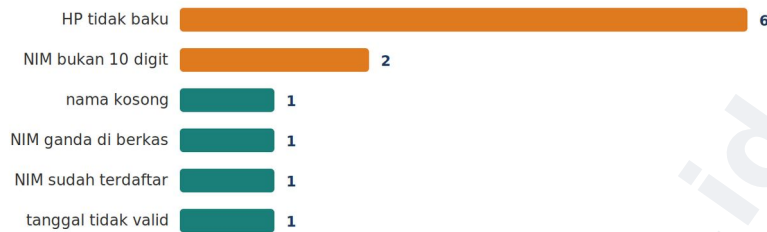
Data dari luar tidak dimasukkan langsung ke tabel utama. Data dimuat dulu ke tabel penampung yang longgar (semua kolom teks, tanpa constraint), diperiksa dan dibersihkan di sana, baru kemudian dipindahkan. Dengan cara ini constraint tabel utama tetap menjadi penjaga terakhir, bukan satu-satunya penjaga.

D. Langkah Percobaan

D.1 Menilai Kualitas Awal

1. Muat `kopma_impор_kotor.sql`, lalu jalankan kueri profil. Hasil yang diharapkan tampak pada Gambar 15.2.

```
SELECT 'nama kosong' AS masalah, COUNT(*) AS jumlah
FROM anggota_impор WHERE TRIM(nama) = ''
UNION ALL SELECT 'NIM bukan 10 digit', COUNT(*)
FROM anggota_impор WHERE nim NOT REGEXP '^0-9]{10}$'
UNION ALL SELECT 'NIM sudah terdaftar', COUNT(*)
FROM anggota_impор i JOIN anggota a ON a.nim_anggota =
REGEXP_REPLACE(i.nim, '^0-9]', '')
UNION ALL SELECT 'HP tidak baku', COUNT(*)
FROM anggota_impор WHERE no_hp IS NULL OR no_hp NOT REGEXP '^080-9]
{8,11}$';
```

Profil masalah data impor calon anggota (10 baris)

Hasil nyata kueri profil pada `kopma_imporkotor.sql`: satu baris dapat memiliki lebih dari satu masalah.

Gambar 15.2 Profil masalah data impor calon anggota

- Lengkapi profil dengan dua dimensi lain: NIM ganda di dalam berkas (**GROUP BY** NIM yang sudah dibersihkan, **HAVING COUNT(*) > 1**) dan tanggal tidak valid.

Titik Analisis: 1

Coba `SELECT STR_TO_DATE('31/09/2026', '%d/%m/%Y');`. Hasilnya bukan NULL, melainkan `2026-09-31`, tanggal yang tidak pernah ada. Namun saat hasil itu disimpan ke kolom `DATE` dalam mode ketat, server menolaknya. Apa pelajarannya tentang “validasi” yang hanya mengandalkan fungsi konversi?

D.2 Menentukan Prioritas

- Urutkan masalah berdasarkan dampaknya bagi Kopma. Contoh: NIM ganda dan NIM sudah terdaftar berdampak tinggi (diskon ganda, pelanggaran AB-05), sedangkan huruf kapital nama berdampak rendah. Tulis tabel prioritas beserta tindakan: **perbaiki otomatis**, **tolak dan kembalikan ke panitia**, atau **terima dengan catatan**.

D.3 Membersihkan di Tabel Penampung

- Buat tabel `anggota_bersih` dari `anggota_imporkotor`. Validasi tanggal dilakukan **sebelum** konversi, sehingga tanggal mustahil menjadi NULL alih-alih galat.

```
CREATE TABLE anggota_bersih AS
SELECT baris,
       REGEXP_REPLACE(nim, '[^0-9]', '') AS nim_anggota,
       CONCAT(UPPER(LEFT(TRIM(nama), 1)), LOWER(SUBSTRING(TRIM(nama), 2)))
AS nama_anggota,
       CASE WHEN UPPER(REGEXP_REPLACE(prodi, '+', ' ')) IN ('ILMU
KOMPUTER', 'ILKOM', 'IK')
            THEN 'Ilmu Komputer'
            WHEN UPPER(prodi) IN ('SISTEM INFORMASI', 'SI') THEN 'Sistem
Informasi'
            WHEN UPPER(prodi) IN ('TEKNIK INFORMATIKA', 'TI') THEN 'Teknik
Informatika'
            WHEN UPPER(prodi) IN ('PENDIDIKAN MATEMATIKA', 'PEND.
MATEMATIKA')
            THEN 'Pendidikan Matematika' END AS prodi_anggota,
       CASE WHEN REGEXP_REPLACE(no_hp, '[^0-9]', '') REGEXP '^628'
            THEN CONCAT('0', SUBSTRING(REGEXP_REPLACE(no_hp, '[^0-
9]', ''), 3))
```

```

        WHEN REGEXP_REPLACE(no_hp, '[^0-9]', '') REGEXP '^8'
        THEN CONCAT('0', REGEXP_REPLACE(no_hp, '[^0-9]', ''))
        WHEN REGEXP_REPLACE(no_hp, '[^0-9]', '') REGEXP '^08'
        THEN REGEXP_REPLACE(no_hp, '[^0-9]', '') END AS
no_hp_anggota,
CASE WHEN tgl_daftar REGEXP '^[0-9]{2}/[0-9]{2}/[0-9]{4}$'
AND CAST(LEFT(tgl_daftar, 2) AS UNSIGNED) BETWEEN 1 AND
DAY(LAST_DAY(STR_TO_DATE(CONCAT('01', SUBSTRING(tgl_daftar,
3)), '%d/%m/%Y')))
THEN STR_TO_DATE(tgl_daftar, '%d/%m/%Y') END AS
tgl_daftar_anggota
FROM anggota_imp;

```

Titik Analisis: 2

Pembersihan nama hanya mengubah huruf pertama menjadi kapital (“Nadia putri”). Mengapa ini dianggap berdampak rendah di langkah D.2? Tuliskan satu risiko jika Anda mencoba “memperbaiki” nama secara berlebihan, misalnya pada nama bermarga atau bergelar.

D.4 Memindahkan Data Bersih dan Mengukur Ulang

5. Masukkan baris yang lolos semua aturan ke tabel **anggota**. Duplikat di dalam berkas disaring dengan **ROW_NUMBER()**, dan NIM yang sudah terdaftar disaring dengan **NOT EXISTS**. Nomor anggota baru dilanjutkan dari nomor terbesar.

```

INSERT INTO anggota (no_anggota, nim_anggota, nama_anggota,
prodi_anggota,
no_hp_anggota, status_anggota, tgl_daftar_anggota)
SELECT CONCAT('A-', LPAD(maks.n + ROW_NUMBER() OVER (ORDER BY k.baris),
4, '0')),
k.nim_anggota, k.nama_anggota, k.prodi_anggota, k.no_hp_anggota,
'aktif', k.tgl_daftar_anggota
FROM (SELECT b.*, ROW_NUMBER() OVER (PARTITION BY b.nim_anggota ORDER
BY b.baris) AS urut
FROM anggota_bersih b
WHERE b.nama_anggota <> '' AND b.tgl_daftar_anggota IS NOT NULL
AND b.prodi_anggota IS NOT NULL
AND b.nim_anggota REGEXP '^[0-9]{10}$'
AND NOT EXISTS (SELECT 1 FROM anggota a WHERE a.nim_anggota =
b.nim_anggota)) AS k
CROSS JOIN (SELECT MAX(CAST(SUBSTRING(no_anggota, 3) AS UNSIGNED)) AS n
FROM anggota) AS maks
WHERE k.urut = 1;

```

6. Hasil yang diharapkan: 6 anggota baru, A-0469 sampai A-0474. Baris nama kosong, tanggal 31 September, dan NIM yang sudah terdaftar tidak masuk. Susun daftar baris yang ditolak untuk dikembalikan ke panitia.
7. Jalankan ulang kueri profil pada data yang dimasukkan, tulis hasil sebelum-sesudah di **p15_audit_kualitas_kopma_<NIM>.md**, lalu commit dengan pesan **p15: kualitas data kopma**.

Titik Analisis: 3

Dua anggota baru tidak memiliki nomor HP yang valid dan tetap diterima dengan HP kosong. Apakah keputusan ini benar? Kaitkan dengan dimensi kelengkapan, aturan bisnis Kopma, dan prinsip meminimalkan data pribadi.

E. Latihan dan Modifikasi

1. Tulis kueri yang mengubah setiap kata nama menjadi berhuruf awal kapital (misalnya “Nadia Putri”) dengan CTE rekursif atau pemisahan kata sederhana, lalu diskusikan batasannya.
2. Tambahkan dimensi **akurasi**: tandai calon anggota yang dua digit awal NIM-nya tidak cocok dengan tahun pendaftaran. Tentukan tindakannya.

F. Tugas Mandiri: Milestone Proyek 15

- Buat tabel penampung berisi minimal 10 baris data impor bermasalah untuk tema proyek Anda, dengan minimal empat jenis masalah dari dimensi berbeda.
- Tulis `p15_kualitas_<NIM>.sql`: kueri profil, pembersihan, pemindahan ke tabel utama, dan pengukuran ulang.
- Tulis `p15_audit_kualitas_<NIM>.md`: kerangka (dimensi dan target), profil awal, tabel prioritas dan tindakan, hasil sesudah perbaikan, dan daftar baris yang ditolak.
- Minimal satu masalah harus bisa lolos constraint tabel utama tetapi tetap salah secara bisnis, sehingga hanya tertangkap oleh audit Anda.

Catatan: hubungan ke Buku 2

Aturan kualitas yang Anda tulis manual di sini akan diotomatiskan dengan trigger di Buku 2 Modul 4.

Troubleshooting Umum

Masalah 1: ERROR 1292 Incorrect date value

Cara membaca. Pesan menyebut nilai tanggal dan kolom tujuannya. Dalam mode ketat, tanggal mustahil ditolak saat disimpan meskipun fungsi konversi tampak berhasil.

Perbaikan: validasi format dan rentang hari terlebih dahulu, lalu konversi hanya baris yang lolos, seperti langkah D.3.

Masalah 2: ERROR 1062 saat memindahkan data bersih

Cara membaca. Ada duplikat yang lolos penyaringan, biasanya karena NIM dibersihkan setelah pemeriksaan duplikat. **Perbaikan:** pastikan pemeriksaan keunikan memakai nilai yang sudah dibersihkan, dan saring duplikat di dalam berkas dengan `ROW_NUMBER()`.

Masalah 3: REGEXP tidak menemukan pola yang jelas ada

Cara membaca. Ada spasi atau karakter tak terlihat di awal dan akhir data. **Perbaikan:** terapkan `TRIM()` sebelum `REGEXP`, dan periksa panjang dengan `LENGTH()`.

Checklist Laporan Pertemuan 15

Laporan disimpan sebagai `laporan/p15_laporan_<NIM>.md` dengan struktur baku, lalu di-push sebelum batas waktu. Pastikan semua **checklist umum** (lihat Pedoman dan Checklist Laporan Praktikum) dan checklist khusus berikut terpenuhi.

Tabel 15.2 Checklist khusus laporan Pertemuan 15

	Jenis	Yang harus ada
☐	Berkas wajib	<code>p15_kualitas_<NIM>.sql</code> dan <code>p15_audit_kualitas_<NIM>.md</code>
☐	Bukti tangkapan layar	Profil masalah sebelum dan sesudah, daftar baris yang ditolak
☐	Analisis wajib	Tabel temuan: dimensi kualitas, jumlah kasus, tindakan perbaikan

Perhatian: aturan keaslian

Data impor bermasalah proyek dirancang sendiri sesuai tema. Daftar masalah, prioritas, dan keputusan penolakan harus dapat dijelaskan saat viva.

Rubrik Penilaian Modul

Tabel 15.3 Rubrik penilaian Modul 15

Aspek	Bobot	Indikator
Pelaksanaan langkah D	25%	Profil, pembersihan, dan pemindahan Kopma sesuai hasil yang diharapkan
Titik Analisis dan latihan E	20%	Validasi tanggal dan keputusan data pribadi beralasan
Milestone Proyek 15	40%	Siklus kualitas lengkap dengan bukti sebelum-sesudah
Laporan dan bukti Git	15%	Lengkap sesuai checklist

Sinkronisasi Kompetensi

Tabel 15.4 Unit kompetensi yang dilatih di Modul 15

Kode unit	Judul unit	Elemen yang dilatih
J.62DMS00.012.1	Mengelola Kualitas Data	Elemen 1–4: kerangka kerja kualitas, penilaian awal, prioritas perbaikan, operasi koreksi dan pengukuran kualitas data

Pertemuan 16: UAS Proyek Final dan Serah Terima

Cerita Pertemuan Ini

Kepengurusan Kopma akan berganti. Pengurus lama meminta satu hal sebelum lengser: basis data yang dapat diserahkan kepada pengurus baru tanpa kehadiran pembuatnya. Artinya, basis data harus dapat dipasang ulang dari skrip, setiap keputusan terdokumentasi, dan setiap peran tahu apa yang boleh dilihatnya. Hari ini Anda melakukan serah terima akhir, dan cerita Kopma berlanjut di Buku 2.

A. Tujuan dan Cakupan

UAS mengukur kemampuan menyerahkan basis data operasional (OLTP) yang utuh: seluruh milestone Pertemuan 1–15 terintegrasi, dapat dipasang dari nol, menjawab semua KI, aman per peran, transaksional, dan berkualitas datanya.

Tabel 16.1 Cakupan UAS

Milestone	Yang diperiksa
1–7 (sudah diuji di UTS)	Tetap harus berjalan; perbaikan pascaUTS dicatat di README
9–12 – Kueri	Laporan ringkasan, relasional, analitis, peringkat dan tren menjawab semua KI
13 – View per peran	Pembatasan akses dan perlindungan data pribadi terbukti
14 – Transaksi	Transaksi utama dengan COMMIT/ROLLBACK; utang desain lunas
15 – Kualitas data	Audit sebelum–sesudah dan data impor bersih

B. Paket Serah Terima

Paket serah terima adalah isi repositori pada commit terakhir sebelum batas waktu:

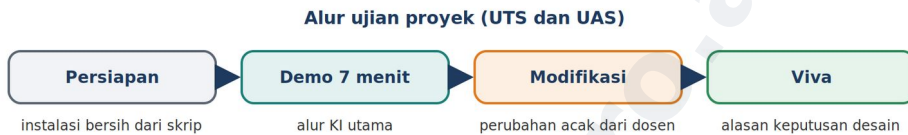
- `install_uas_<NIM>.sql`: memanggil DDL, data, view, role (tanpa password asli), dan skrip kualitas secara berurutan.
- `README.md` final: identitas proyek, cara instalasi, daftar peran dan aksesnya, serta daftar perubahan sejak UTS.
- `kamus_data_<NIM>.md` final yang sesuai dengan struktur tabel terakhir.
- `p16_uas_<NIM>.md`: peta KI → kueri/view, daftar aturan bisnis → constraint/transaksi, dan refleksi P1–P16.

Contoh skrip instalasi UAS

```
-- install_uas_2301010123.sql
SOURCE p05_ddl_2301010123.sql;
SOURCE p06_data_2301010123.sql;
SOURCE p13_view_2301010123.sql;
SOURCE p15_kualitas_2301010123.sql;
-- p13_role_*.sql dijalankan terpisah sebagai root
```

C. Alur Pelaksanaan

Alurnya sama dengan UTS (Gambar 16.1), tetapi demo berfokus pada fitur Pertemuan 9–15 dan bank modifikasi lebih luas.



Total ± 20 menit per mahasiswa; seluruh berkas diambil dari commit terakhir sebelum batas waktu.

Gambar 16.1 Alur ujian proyek UAS

D. Bank Modifikasi Langsung

1. Tambahkan satu KI baru dari dosen dan jawab dengan kueri JOIN dan agregasi.
2. Ubah satu laporan peringkat agar memakai **DENSE_RANK** per kelompok lain.
3. Buat peran baru dengan satu view dan buktikan pembatasannya.
4. Tambahkan satu pemeriksaan bisnis ke transaksi utama dan tunjukkan skenario ROLLBACK-nya.
5. Masukkan tiga baris data impor bermasalah baru dan jalankan ulang siklus kualitas.
6. Temukan dan perbaiki kesalahan semantik yang sengaja disisipkan dosen pada satu kueri.

E. Contoh Pertanyaan Viva

- Tunjukkan satu KI dan telusuri jalurnya dari kebutuhan (P2), ERD (P3), tabel (P5), sampai kueri (P9–12).
- Data pribadi apa yang ada di proyek Anda, dan bagaimana setiap peran dibatasi?
- Aturan bisnis mana yang dijaga constraint, mana yang dijaga transaksi, dan mana yang dijaga audit kualitas? Mengapa dibagi demikian?
- Apa yang akan Anda ubah jika proyek ini dimulai ulang?

F. Rubrik Penilaian UAS

Tabel 16.2 Rubrik rinci UAS

Aspek (bobot)	Sangat baik	Cukup	Kurang
Produk berjalan (40%)	Instalasi bersih sukses; semua KI terjawab; peran, transaksi, kualitas terbukti	Sebagian fitur P9–15 belum lengkap	Gagal dipasang atau banyak fitur hilang
Demo, modifikasi, viva (30%)	Modifikasi selesai; penelusuran P2–P12 lancar	Modifikasi selesai dengan bantuan	Tidak dapat menjelaskan proyek sendiri
Dokumentasi dan Git (20%)	Paket serah terima lengkap; commit rutin P1–P16	Dokumen cukup; commit tidak rutin	Dokumen minim; commit menumpuk
Ketepatan waktu (10%)	Sebelum batas waktu	–	Terlambat

Catatan: hubungan ke Buku 2

Repositori dan basis data proyek ini **jangan dihapus**. Pada Pertemuan 1 Buku 2, basis data ini dimigrasikan ke MariaDB 12.3 dan dikelola sampai akhir semester berikutnya.

Checklist Laporan Pertemuan 16

Laporan disimpan sebagai `laporan/p16_laporan_<NIM>.md` dengan struktur baku, lalu di-push sebelum batas waktu. Pastikan semua **checklist umum** (lihat Pedoman dan Checklist Laporan Praktikum) dan checklist khusus berikut terpenuhi.

Tabel 16.3 Checklist khusus laporan Pertemuan 16

	Jenis	Yang harus ada
☐	Berkas wajib	<code>p16_uas_<NIM>.md</code> , <code>install_uas_<NIM>.sql</code> , <code>README.md</code> dan <code>kamus_data_<NIM>.md</code> final
☐	Bukti tangkapan layar	Instalasi bersih dari nol dan demo seluruh KI
☐	Analisis wajib	Catatan modifikasi langsung; refleksi perjalanan proyek P1–P16

Sinkronisasi Kompetensi

Tabel 16.4 Unit kompetensi yang diujikan pada UAS

Kode unit	Judul unit	Bukti
J.62DMS00.006.1	Merancang Basis Data	View, laporan, desain terdokumentasi

Kode unit	Judul unit	Bukti
J.62DMS00.010.1	Membuat Basis Data	Instalasi, uji, dan transaksi
J.62DMS00.012.1	Mengelola Kualitas Data	Audit kualitas
J.62DMS00.019.1	Menggunakan Data	Kueri KI dan akses per peran
J.620100.020.02	Menggunakan SQL	Seluruh kueri P7–P15

Daftar Pustaka

Draf sementara: memuat referensi yang telah dikutip pada bagian yang sudah ditulis. Daftar akan bertambah seiring penulisan modul berikutnya.

- [1] Kementerian Ketenagakerjaan Republik Indonesia, *Keputusan Menteri Ketenagakerjaan Nomor 268 Tahun 2020 tentang Penetapan Standar Kompetensi Kerja Nasional Indonesia Kategori Informasi dan Komunikasi Golongan Pokok Aktivitas Pemrograman, Konsultasi Komputer, dan Kegiatan yang Berhubungan dengan Itu (YBDI) Bidang Data Management System Sub Bidang Data Management*. Jakarta, Indonesia: Kemnaker, 2020.
- [2] T. Taipalus, "Explaining causes behind SQL query formulation errors," in *Proc. IEEE Frontiers Educ. Conf. (FIE)*, Uppsala, Sweden, 2020, pp. 1–9, doi: 10.1109/FIE44824.2020.9274114.
- [3] T. Taipalus, "Query execution plans and semantic errors: Usability and educational opportunities," in *Ext. Abstr. CHI Conf. Human Factors Comput. Syst.*, Hamburg, Germany, 2023, pp. 1–6, doi: 10.1145/3544549.3585794.
- [4] Z. Hatami and P. Wolcott, "Understanding students' identification and use of patterns while writing SQL queries," in *Proc. 21st Annu. Conf. Inf. Technol. Educ. (SIGITE)*, Virtual Event, USA, 2020, pp. 20–25, doi: 10.1145/3368308.3415367.
- [5] A. Borodzhieva, "Using project-based learning in the course 'Databases' on the topic 'Relational algebra'," in *Proc. 45th Jubilee Int. Conv. Inf., Commun. Electron. Technol. (MIPRO)*, Opatija, Croatia, 2022, pp. 1369–1374, doi: 10.23919/MIPRO55190.2022.9803494.
- [6] A. Subhan and W. H. Haji, "Analisa dan perancangan sistem informasi manajemen validasi data pembangunan fiber optik," *J. Teknol. Inf. Ilmu Komput.*, vol. 8, no. 6, pp. 1107–1116, Nov. 2021, doi: 10.25126/jtiik.2021862884.
- [7] E. Cahyaningsih, A. Rinjatmoko, and W. P. Sari, "Pengukuran kualitas data menggunakan framework total data quality management: Studi kasus Kementerian Hukum dan Hak Asasi Manusia Rutan Klas I Jakarta Pusat," *J. Teknol. Inf. Ilmu Komput.*, vol. 12, no. 1, pp. 121–132, Feb. 2025, doi: 10.25126/jtiik.2025129178.
- [8] Kementerian Ketenagakerjaan Republik Indonesia, *Keputusan Menteri Ketenagakerjaan Nomor 282 Tahun 2016 tentang Penetapan Standar Kompetensi Kerja Nasional Indonesia Bidang Software Development Subbidang Pemrograman*. Jakarta, Indonesia: Kemnaker, 2016.
- [9] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Berkeley, CA, USA: Apress, 2014. [Online]. Available: <https://git-scm.com/book/en/v2>
- [10] Apache Friends, "Download XAMPP," [apachefriends.org](https://www.apachefriends.org). Accessed: Sep. 24, 2026. [Online]. Available: <https://www.apachefriends.org/download.html>

- [11] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.
- [12] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill Education, 2020.
- [13] MariaDB Foundation, "MariaDB Server documentation," mariadb.com. Accessed: Sep. 24, 2026. [Online]. Available: <https://mariadb.com/docs/server>
- [14] MariaDB Foundation, "MariaDB 10.4.32 release notes," mariadb.com. Accessed: Sep. 24, 2026. [Online]. Available: <https://mariadb.com/docs/release-notes/community-server/old-releases/10.4/10.4.32>
- [15] T. Connolly and C. Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*, 6th ed. Harlow, U.K.: Pearson, 2015.
- [16] R. Abraham, J. Schneider, and J. vom Brocke, "Data governance: A conceptual framework, structured review, and research agenda," *Int. J. Inf. Manage.*, vol. 49, pp. 424–438, Dec. 2019, doi: 10.1016/j.ijinfomgt.2019.07.008.
- [17] Republik Indonesia, *Undang-Undang Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi*. Jakarta, Indonesia: Sekretariat Negara, 2022.

Glosarium

Tabel G.1 Istilah penting di buku ini

Istilah	Arti singkat	Pertemuan
Agregat	Fungsi yang meringkas banyak baris menjadi satu nilai (SUM, COUNT)	9
Aturan bisnis (AB)	Batasan dari organisasi yang harus dipatuhi data, diberi kode AB-xx	2
Atribut turunan	Nilai yang dapat dihitung dari atribut lain, misalnya subtotal	3–4
CTE	Hasil antara bernama yang ditulis dengan WITH	12
DBMS	Perangkat lunak pengelola basis data, di buku ini MariaDB	1
DDL / DML	Perintah pembentuk struktur / perintah pengolah isi data	5 / 6
DEFINER	Akun pembuat view yang haknya dipakai saat view dijalankan	13
Dependensi fungsional	$A \rightarrow B$: nilai A selalu menentukan tepat satu nilai B	4
Entitas asosiatif	Tabel pemecah relasi banyak-ke-banyak yang punya atribut sendiri	3
ERD	Diagram entitas, atribut, dan relasi	3
Galat semantik	Kueri yang berjalan tanpa galat tetapi hasilnya salah	7, 9, 11
Kardinalitas	Jumlah minimum dan maksimum pasangan dalam satu relasi	3
Kebutuhan informasi (KI)	Pertanyaan yang harus bisa dijawab basis data, diberi kode KI-xx	2
Kunci primer / tamu	Kolom identitas baris / kolom yang merujuk kunci primer tabel lain	3, 5
Mode ketat	Mode SQL yang menolak data tidak valid (STRICT_TRANS_TABLES)	1, 15
Normalisasi	Pemecahan tabel untuk menghilangkan anomali	4
Role	Kumpulan hak akses bernama yang diberikan ke akun	13
Tabel penampung	Tabel sementara untuk data impor sebelum dibersihkan	15
Transaksi	Rangkaian perintah yang berhasil semua atau batal semua (ACID)	14

Istilah	Arti singkat	Pertemuan
View	Kueri bernama yang dipakai seperti tabel	13
Window function	Fungsi yang menghitung per baris tanpa meringkas, memakai OVER	12

Lampiran A: Ringkasan Sintaks SQL

Rujukan cepat saat praktikum; penjelasan lengkap ada di pertemuan (P) yang disebutkan.

Tabel A.1 Ringkasan sintaks SQL yang dipakai di buku ini

Keperluan	Bentuk umum	P
Basis data dan akun	CREATE DATABASE nama ...; / CREATE USER ...; GRANT ... ON db.* TO ...;	1
Membuat tabel	CREATE TABLE t (kolom tipe [NOT NULL] [DEFAULT ...], PRIMARY KEY (...), CONSTRAINT ...);	5
Mengubah struktur	ALTER TABLE t ADD COLUMN ... DROP COLUMN ... ADD CONSTRAINT ...;	5
Menambah data	INSERT INTO t (k1, k2) VALUES (...), (...); atau INSERT INTO t SELECT ...;	6
Mengubah / menghapus	UPDATE t SET k = ... WHERE ...; / DELETE FROM t WHERE ...;	6
Mengambil data	SELECT ... FROM t WHERE ... ORDER BY ... LIMIT n;	7
Meringkas	SELECT k, SUM(x) FROM t GROUP BY k HAVING ... [WITH ROLLUP];	9
Menggabungkan	FROM a JOIN b ON b.id_a = a.id_a LEFT JOIN c ON ...	10
Subquery	WHERE x > (SELECT AVG(x) FROM t); WHERE NOT EXISTS (SELECT 1 FROM ...)	11
Operasi himpunan	SELECT ... UNION [ALL] INTERSECT EXCEPT SELECT ...	11
CTE	WITH nama AS (SELECT ...) SELECT ... FROM nama; (tambah RECURSIVE untuk rekursif)	12
Window function	RANK() OVER (PARTITION BY k ORDER BY x DESC); SUM(x) OVER (ORDER BY tgl)	12
View dan role	CREATE VIEW v AS SELECT ...; / CREATE ROLE r; GRANT r TO akun;	13
Transaksi	START TRANSACTION; ... COMMIT; atau ROLLBACK;	14

Lampiran B: Daftar Kode Galat

Pesan di bawah adalah pesan asli MariaDB 10.4.32 yang muncul pada praktikum di buku ini. Bagian dalam tanda . . . berisi nama objek yang berbeda sesuai perintah Anda. Cara membaca pesan galat dijelaskan di Modul 1.

Tabel B.1 Kode galat MariaDB yang dibahas di buku ini

Kode	Pesan (disingkat)	Arti dan tindakan	P
1005 (errno 150)	Can't create table ... Foreign key constraint is incorrectly formed	Tipe kunci tamu tidak sama dengan kunci yang dirujuk, atau tabel induk belum ada	5
1044	Access denied for user ... to database ...	Akun tidak berhak atas basis data itu	1
1045	Access denied for user ... (using password: NO/YES)	Password tidak dikirim (NO) atau salah (YES)	1
1050	Table ... already exists	Skrip dijalankan dua kali; pakai DROP TABLE IF EXISTS	5
1052	Column ... in field list is ambiguous	Kolom ada di lebih dari satu tabel; beri alias tabel	10
1054	Unknown column ... in 'where clause' / 'field list'	Nama kolom salah, atau alias dipakai sebelum dievaluasi	7, 13
1055	... isn't in GROUP BY	Kolom non-agregat belum masuk GROUP BY (mode ONLY_FULL_GROUP_BY)	9
1062	Duplicate entry ... for key ...	Nilai melanggar PRIMARY KEY atau UNIQUE	5–6
1064	You have an error in your SQL syntax ... near ...	Galat sintaks; lihat teks setelah near . near ' ' berarti di akhir perintah	11
1142	SELECT command denied to user ... for table db.t	Akun atau role tidak berhak atas tabel	13
1146	Table ... doesn't exist	Nama tabel salah atau basis data aktif keliru	13
1175	You are using safe update mode ...	UPDATE/DELETE tanpa syarat kolom kunci di mode aman	6
1205	Lock wait timeout exceeded	Sesi lain memegang kunci baris; akhiri transaksinya	14

Kode	Pesan (disingkat)	Arti dan tindakan	P
1222	The used SELECT statements have a different number of columns	Jumlah kolom operasi himpunan tidak sama	11
1227	Access denied; you need ... the CREATE USER privilege(s)	Perintah administrasi; jalankan sebagai root	13
1242	Subquery returns more than 1 row	Subquery skalar menghasilkan banyak baris	11
1292	Incorrect date value ... for column ...	Tanggal tidak valid ditolak di mode ketat	15
1369	CHECK OPTION failed ...	Perubahan lewat view akan mengeluarkan baris dari view	13
1451	Cannot delete or update a parent row ...	Data induk masih dirujuk data anak	5–6
1452	Cannot add or update a child row ...	Nilai kunci tamu belum ada di tabel induk	5–6
4025	CONSTRAINT ... failed for ...	Nilai melanggar constraint CHECK	5, 14
4151	Access denied, this account is locked	Akun sudah dikunci dengan ACCOUNT LOCK	13

Lampiran C: Skema Kanonik Kopma dan Berkas Data

Skema berikut adalah acuan tunggal kasus Kopma untuk seluruh modul Buku 1 dan Buku 2. Skrip ini hanya berisi struktur; datanya dimuat dari berkas terpisah.

Tabel C.1 Berkas pendukung dari dosen

Berkas	Isi	Dipakai pada
<code>kopma_skema_kanonik.sql</code>	Struktur 8 tabel dan data contoh minimal	Pertemuan 5–8 (pembanding)
<code>kopma_data_lanjutan.sql</code>	12 anggota, 12 barang, 75 nota September–Oktober 2026	Pertemuan 9–16
<code>kopma_impор_kotor.sql</code>	Tabel penampung <code>anggota_impор</code> berisi 10 baris bermasalah	Pertemuan 15

`kopma_skema_kanonik.sql` (bagian struktur)

```

CREATE DATABASE IF NOT EXISTS kopma_123
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE kopma_123;

CREATE TABLE petugas (
  id_petugas      SMALLINT          NOT NULL AUTO_INCREMENT,
  kode_petugas    CHAR(3)           NOT NULL,
  -- mis. K03
  nama_petugas    VARCHAR(100)      NOT NULL,
  peran_petugas   ENUM('kasir','gudang','ketua') NOT NULL,
  PRIMARY KEY (id_petugas),
  UNIQUE KEY uq_petugas_kode (kode_petugas)
) ENGINE=InnoDB;

CREATE TABLE anggota (
  id_anggota      INT               NOT NULL AUTO_INCREMENT,
  no_anggota      CHAR(6)           NOT NULL,
  A-0457
  nim_anggota     CHAR(10)          NOT NULL,
  nama_anggota    VARCHAR(100)     NOT NULL,
  prodi_anggota   VARCHAR(60)      NOT NULL,
  no_hp_anggota   VARCHAR(15)      NULL,
  data_pribadi (PDP)
  status_anggota   ENUM('aktif','nonaktif') NOT NULL DEFAULT 'aktif',
  tgl_daftar_anggota DATE          NOT NULL,
  PRIMARY KEY (id_anggota),
  UNIQUE KEY uq_anggota_no (no_anggota),
  UNIQUE KEY uq_anggota_nim (nim_anggota)
  -- AB-05
) ENGINE=InnoDB;

CREATE TABLE barang (
  id_barang      INT               NOT NULL
  AUTO_INCREMENT,

```

```

    kode_barang      VARCHAR(10)          NOT NULL,
    nama_barang      VARCHAR(100)         NOT NULL,
    kategori_barang  ENUM('ATK','makanan','minuman') NOT NULL,
    harga_jual_barang DECIMAL(12,2)       NOT NULL,
    stok_barang      INT                  NOT NULL DEFAULT 0,
    stok_min_barang  INT                  NOT NULL DEFAULT 5,
    PRIMARY KEY (id_barang),
    UNIQUE KEY uq_barang_kode (kode_barang),
    CONSTRAINT ck_barang_stok CHECK (stok_barang >= 0), -- AB-03
    CONSTRAINT ck_barang_harga CHECK (harga_jual_barang >= 0)
) ENGINE=InnoDB;

CREATE TABLE pemasok (
    id_pemasok      INT          NOT NULL AUTO_INCREMENT,
    nama_pemasok    VARCHAR(100) NOT NULL,
    telepon_pemasok VARCHAR(15)  NULL,
    alamat_pemasok  VARCHAR(200) NULL,
    PRIMARY KEY (id_pemasok)
) ENGINE=InnoDB;

CREATE TABLE penjualan (
    id_penjualan     INT          NOT NULL AUTO_INCREMENT,
    no_nota_penjualan CHAR(12)    NOT NULL, -- PJ-2609-0142,
    AB-01
    tgl_penjualan    DATETIME     NOT NULL,
    id_petugas       SMALLINT     NOT NULL,
    id_anggota       INT          NULL, -- NULL =
    pembeli umum, AB-02
    bayar_penjualan  DECIMAL(12,2) NOT NULL,
    PRIMARY KEY (id_penjualan),
    UNIQUE KEY uq_penjualan_nota (no_nota_penjualan),
    CONSTRAINT fk_penjualan_petugas FOREIGN KEY (id_petugas) REFERENCES
    petugas (id_petugas),
    CONSTRAINT fk_penjualan_anggota FOREIGN KEY (id_anggota) REFERENCES
    anggota (id_anggota)
) ENGINE=InnoDB;

CREATE TABLE detail_penjualan (
    id_penjualan     INT          NOT NULL,
    id_barang        INT          NOT NULL,
    qty_detail_penjualan SMALLINT NOT NULL,
    harga_satuan_detail_penjualan DECIMAL(12,2) NOT NULL, --
    harga saat transaksi, AB-04
    PRIMARY KEY (id_penjualan, id_barang),
    CONSTRAINT fk_detail_penjualan FOREIGN KEY (id_penjualan) REFERENCES
    penjualan (id_penjualan) ON DELETE CASCADE,
    CONSTRAINT fk_detail_barang FOREIGN KEY (id_barang) REFERENCES
    barang (id_barang),
    CONSTRAINT ck_detail_qty CHECK (qty_detail_penjualan > 0)
) ENGINE=InnoDB;

CREATE TABLE pembelian (
    id_pembelian     INT          NOT NULL
    AUTO_INCREMENT,
    no_faktur_pembelian VARCHAR(20) NOT NULL,
    tgl_pembelian    DATE          NOT NULL,
    id_pemasok       INT          NOT NULL,

```

```
status_pembelian    ENUM('dipesan','diterima') NOT NULL DEFAULT
'dipesan', -- AB-06
PRIMARY KEY (id_pembelian),
CONSTRAINT fk_pembelian_pemasok FOREIGN KEY (id_pemasok) REFERENCES
pemasok (id_pemasok)
) ENGINE=InnoDB;

CREATE TABLE detail_pembelian (
  id_pembelian        INT          NOT NULL,
  id_barang           INT          NOT NULL,
  qty_detail_pembelian INT          NOT NULL,
  harga_beli_detail_pembelian DECIMAL(12,2) NOT NULL,
  PRIMARY KEY (id_pembelian, id_barang),
  CONSTRAINT fk_dbeli_pembelian FOREIGN KEY (id_pembelian) REFERENCES
pembelian (id_pembelian) ON DELETE CASCADE,
  CONSTRAINT fk_dbeli_barang    FOREIGN KEY (id_barang)    REFERENCES
barang (id_barang),
  CONSTRAINT ck_dbeli_qty CHECK (qty_detail_pembelian > 0)
) ENGINE=InnoDB;
```

Tentang Penulis

Dedi Irawan, S.Kom., M.T.I. adalah dosen pada Program Studi Ilmu Komputer, Fakultas Ilmu Komputer, Universitas Muhammadiyah Metro. Penulis menempuh pendidikan Sarjana Komputer (S.Kom.) dan Magister Teknologi Informasi (M.T.I.).

Buku ini disusun untuk mendukung pembelajaran praktikum basis data yang selaras dengan kebutuhan dunia kerja dan Standar Kompetensi Kerja Nasional Indonesia (SKKNI) bidang Data Management. Buku ini merupakan bagian pertama dari dua buku; pembahasan berlanjut pada *Buku Praktikum Basis Data Lanjut* yang mengangkat administrasi, kinerja, keamanan, dan tata kelola data.

Kritik, saran, dan pertanyaan tentang buku ini dapat disampaikan kepada penulis melalui Program Studi Ilmu Komputer, Fakultas Ilmu Komputer, Universitas Muhammadiyah Metro.